

modo que vale la pena echarle un vistazo. Nos concentraremos en el prototipo de 64 CPU que realmente se construyó, pero es fácil cambiar la escala del diseño a máquinas más grandes.

En la figura 8-31(a) se muestra un diagrama ligeramente simplificado del prototipo DASH. Consiste en 16 cúmulos, cada uno de los cuales contiene un bus, cuatro CPU MIPS R3000, 16 MB de la memoria global, y algo de equipo de E/S (discos, etc.), que no se muestran. Cada CPU espía en su bus local, pero no en ningún otro bus. La coherencia local se mantiene espionando; la coherencia global requiere un mecanismo distinto porque no hay espionaje global.

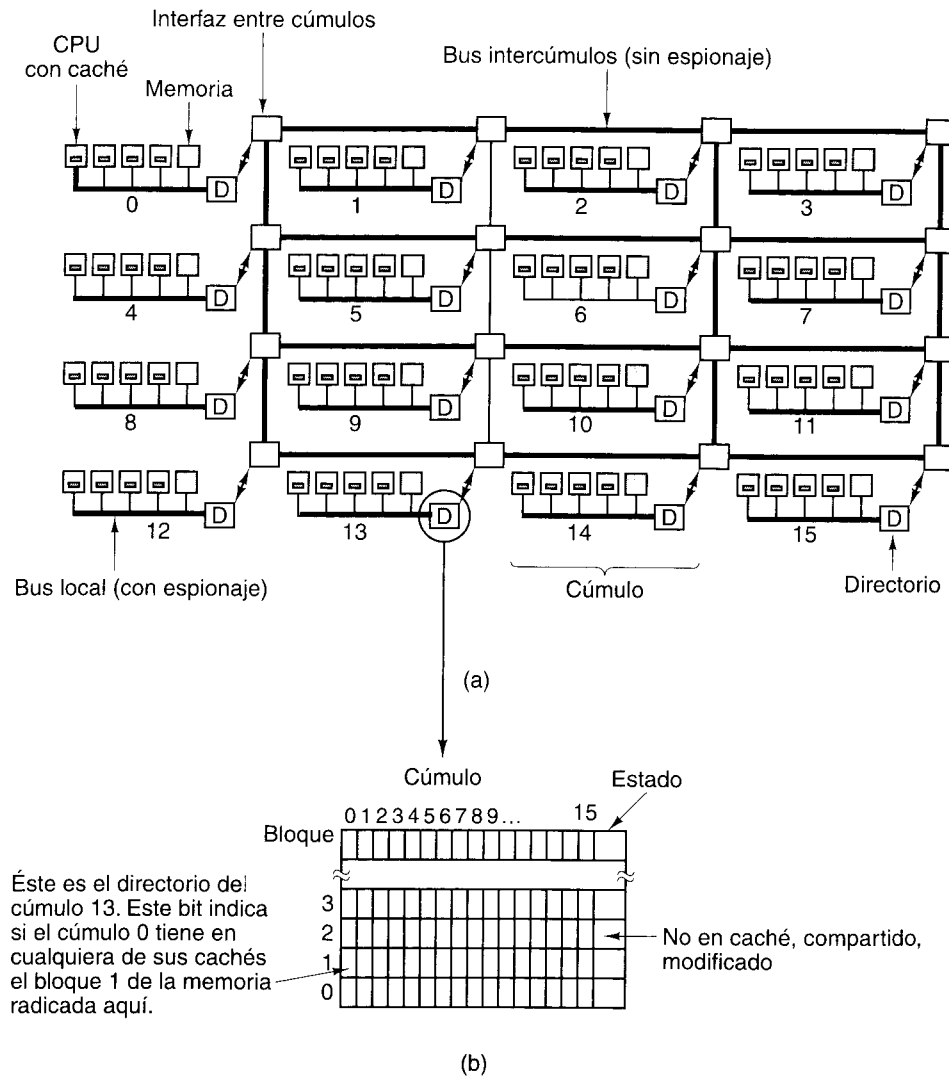


Figura 8-31. (a) La arquitectura DASH. (b) Un directorio DASH.

El espacio de direcciones total disponible en el prototipo es de 256 MB, dividido en 16 regiones de 16 MB cada una. La memoria global del cúmulo 0 contiene las direcciones 0 a 16M. La memoria global del cúmulo 1 contiene las direcciones 16M a 32M, etc. La memoria se coloca en caché y se transfiere en unidades de 16 bytes (líneas), de modo que cada cúmulo tiene 1M líneas de memoria dentro de su espacio de direcciones.

Cada cúmulo tiene un directorio que se mantiene al tanto de cuáles cúmulos tienen actualmente copias de sus líneas. Puesto que cada cúmulo posee 1M líneas de memoria, tiene 1M entradas en su directorio, una por línea. Cada entrada contiene un mapa de bits con un bit por cúmulo que indica si ese cúmulo tiene o no actualmente en caché la línea. La entrada también tiene un campo de dos bits que indica el estado de la línea.

Tener 1M entradas de 18 bits cada una implica que el tamaño total de cada directorio es de más de 2 MB. Con 16 cúmulos, la memoria de directorio total es de poco más de 36 MB, o sea cerca del 14% de los 256 MB. Si se incrementa el número de CPU por cúmulo, la cantidad de memoria de directorio no cambia. Así, tener más CPU por cúmulo permite amortizar el costo de la memoria de directorio, y también del controlador de bus, entre un número mayor de CPU, lo que reduce el costo por CPU. Es por esto que cada cúmulo tiene varias CPU.

Cada cúmulo de DASH está conectado a una interfaz que permite al cúmulo comunicarse con otros. Las interfaces están conectadas por enlaces intercúmulo (buses primitivos) en una cuadrícula rectangular, como se muestra en la figura 8-31(a). A medida que se añaden más cúmulos al sistema, también se añaden más enlaces entre cúmulos, por lo que el ancho de banda aumenta y el sistema es escalable. El sistema de enlaces intercúmulos usa enrutamiento por túnel, de modo que la primera parte de un paquete se reenvía antes de recibirse todo el paquete, con lo que se reduce el retraso en cada brinco. Aunque no se muestra en la figura, en realidad hay dos conjuntos de enlaces intercúmulos, uno para paquetes de solicitud y uno para paquetes de respuesta. Los enlaces intercúmulos no pueden espionarse.

Cada línea de caché puede estar en uno de los tres estados siguientes:

1. **NO EN CACHÉ** – La única copia de la línea está en esta memoria.
2. **COMPARTIDA** – La memoria está actualizada; la línea puede estar en varias cachés.
3. **MODIFICADA** – La memoria es incorrecta; sólo una caché contiene la línea.

El estado de cada línea de caché se guarda en el campo *Estado* de su entrada de directorio, como se muestra en la figura 8-31(b).

Los protocolos DASH se basan en propiedad e invalidación. En cada instante, cada línea de caché tiene un dueño único. En el caso de líneas **NO EN CACHÉ** o **COMPARTIDAS**, el cúmulo base de la línea es el dueño. En el caso de líneas **MODIFICADAS**, el cúmulo que contiene la única copia es el dueño. Para escribir en una línea **COMPARTIDA** primero es necesario encontrar e invalidar todas las copias existentes. Es aquí donde entran los directorios.

Para ver cómo funciona este mecanismo, consideremos primero la forma en que una CPU lee una palabra de memoria. Primero examina su propio caché. Si la caché no tiene la palabra, se emite una solicitud por el bus de cúmulo local para ver si otra CPU del cúmulo

tiene la línea que la contiene. Si así es, se ejecuta una transferencia de caché a caché para colocar la línea en la caché de la CPU solicitante. Si la línea es **COMPARTIDA**, se hace una copia; si es **MODIFICADA**, se informa al directorio base que la línea ahora es **COMPARTIDA**. De cualquier manera, un acierto de uno de los cachés satisface la instrucción pero no afecta el mapa de bits de ningún directorio (porque el directorio tiene un bit por cúmulo, no un bit por CPU).

Si la línea no está presente en ninguno de los cachés del cúmulo, se envía un paquete de solicitud al cúmulo base de la línea, el cual puede determinarse examinando los cuatro bits altos de la dirección de memoria. El cúmulo base bien podría ser el cúmulo del solicitante, en cuyo caso el mensaje no se envía físicamente. El hardware de gestión de directorio del cúmulo base examina sus tablas para ver en qué estado está la línea. Si está **NO EN CACHÉ** o **COMPARTIDA**, el hardware trae la línea de su memoria global y la devuelve al cúmulo solicitante; luego actualiza su directorio para indicar que la línea está en caché en el cúmulo del solicitante.

En cambio, si la línea requerida está **MODIFICADA**, el hardware de directorio busca la identidad del cúmulo que tiene la línea y reenvía ahí la solicitud. El cúmulo que tiene la línea la envía entonces al cúmulo solicitante y marca su propia copia como **COMPARTIDA** porque ahora hay varias copias; también envía una copia de vuelta al cúmulo base para que esa memoria pueda actualizarse y el estado de la línea se cambie a **COMPARTIDA**.

Las escrituras funcionan de otra manera. Antes de poder efectuar una escritura, la CPU que quiere escribir debe estar segura de que es la dueña de la única copia de la línea de caché en el sistema. Si ya tiene la línea en su caché y la línea está **MODIFICADA**, la escritura puede efectuarse de inmediato. Si tiene la línea pero está **COMPARTIDA**, primero se envía un paquete al cúmulo base para solicitar que se busquen todas las demás copias y se invaliden.

Si la CPU solicitante no tiene la línea de caché, emite una solicitud por el bus local para ver si cualquiera de sus vecinas la tiene. Si así es, se efectúa una transferencia de caché a caché (o de memoria a caché). Si la línea es **COMPARTIDA**, el cúmulo base deberá invalidar todas las demás copias, si existen.

Si la difusión local no logra encontrar una copia y la línea tiene su base en otro lado, se envía un paquete al cúmulo base. Aquí podemos distinguir tres casos. Si la línea está **NO EN CACHÉ**, se marca como **MODIFICADA** y se envía al solicitante. Si la línea está **COMPARTIDA**, todas las copias se invalidan y luego se sigue el procedimiento para **NO EN CACHÉ**. Si la línea está **MODIFICADA**, la solicitud se remite al cúmulo remoto que actualmente es dueño de la línea (si es necesario). Este cúmulo satisface la solicitud y luego invalida su propia copia.

Mantener la consistencia de la memoria en DASH es relativamente complejo y lento. Un solo acceso a la memoria podría requerir el envío de un número considerable de paquetes. Además, a fin de mantener la consistencia de la memoria, el acceso por lo regular no puede llevarse a cabo antes de que se haya reconocido la recepción de todos los paquetes, lo que puede tener un efecto notable sobre el desempeño. Para superar estos problemas, DASH utiliza diversas técnicas especiales, como dos conjuntos de enlaces intercúmulos, escrituras por filas de procesamiento y el uso de consistencia de liberación en lugar de consistencia secuencial como modelo de memoria.

El multiprocesador Sequent NUMA-Q

Aunque la máquina DASH tuvo una influencia notable, nunca fue un producto comercial. En vista de la importancia que están adquiriendo los multiprocesadores CC-NUMA, vale la pena examinar también un producto comercial. Como ejemplo usaremos la máquina Sequent NUMA-Q 2000 porque utiliza un protocolo de coherencia de cachés interesante e importante llamado **interfaz coherente escalable** (SCI, *Scalable Coherent Interface*). Este protocolo se estandarizó como IEEE 1596 y se usa también en varias otras máquinas CC-NUMA.

El NUMA-Q se basa en la tarjeta **quad estándar** vendida por Intel que contiene cuatro chips de CPU Pentium Pro y hasta 4 GB de RAM. Cada CPU tiene una caché de nivel 1 y una caché de nivel 2. Todas estas cachés se mantienen coherentes espiando en el bus local de 534 MB/s de la tarjeta quad con el protocolo MESI. Las líneas de caché son de 64 bytes. El NUMA-Q se ilustra en la figura 8-32.

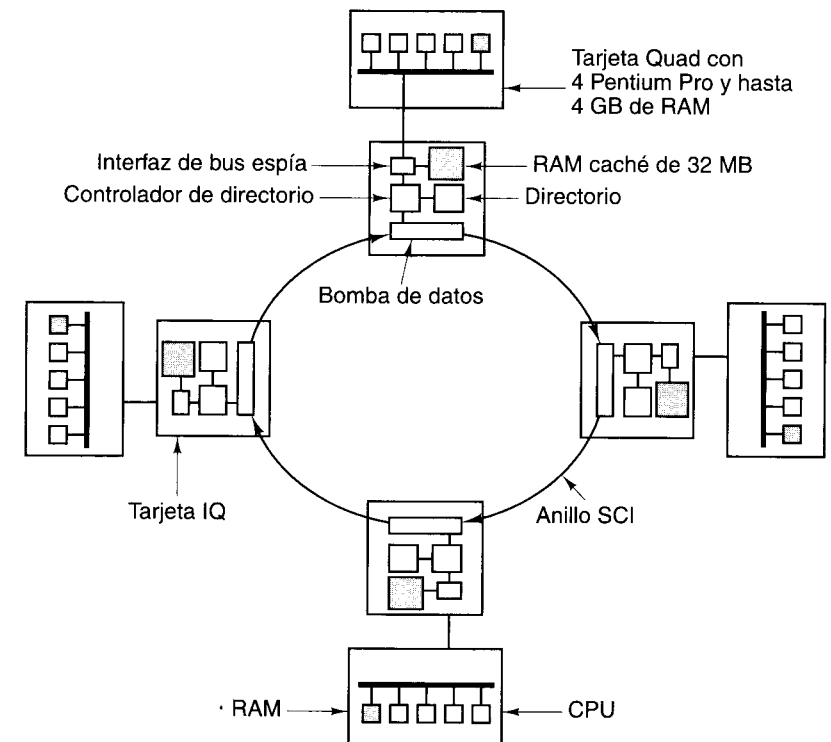


Figura 8-32. El multiprocesador NUMA-Q.

El tamaño del sistema se expande más allá de cuatro CPU insertando una tarjeta de controlador de red en una ranura de la tarjeta quad diseñada para controladores de red. Dicha tarjeta, llamada **tarjeta IQ-Link**, es el adhesivo que convierte todas las tarjetas quad en un solo multiprocesador. Su labor primaria es implementar el protocolo SCI. Cada tarjeta IQ-Link

contiene 32 MB de caché, un directorio que se mantiene al tanto de lo que está en la caché, una interfaz de espionaje con el bus local de la tarjeta quad, y un chip a la medida llamado **bomba de datos** que conecta la tarjeta IQ-Link con otras tarjetas IQ-Link. Básicamente, el chip “bombea” datos del lado de la entrada al lado de la salida, quedándose con los datos que van dirigidos a su nodo y dejando pasar los demás datos sin modificación.

Juntas, todas las tarjetas IQ-Link forman un anillo, como se muestra en la figura 8-32. El protocolo SCI mantiene la coherencia de todas las cachés de tarjeta IQ-Link usando el anillo; no tiene conocimiento del protocolo MESI que se usa para mantener la coherencia entre las cuatro CPU y la caché de 32 MB en cada nodo, así que este diseño tiene dos niveles de protocolo de coherencia, igual que DASH.

Sequent optó por usar SCI como interconexión de las tarjetas quad porque SCI se diseñó para sustituir al bus en sistemas multiprocesador y multicomputadora grandes (como el NUMA-Q). SCI apoya la coherencia de caché que se requiere en los multiprocesadores, pero también la transferencia rápida de bloques que se necesita en las multicomputadoras. SCI puede manejar hasta 64K nodos, cada uno con un espacio de direcciones de hasta 2^{48} bytes. El sistema NUMA-Q más grande consiste en 63 tarjetas quad, que contienen 252 CPU y casi 2^{38} bytes de memoria física, muy por debajo de los límites de SCI.

El anillo que conecta las tarjetas IQ-Link cumple con el protocolo SCI. En realidad, no se trata realmente de un anillo, sino de cables individuales punto a punto. Su anchura es de 18 bits que incluyen un bit de reloj, un bit indicador y 16 bits de datos, todos enviados en paralelo. Los enlaces operan a 500 MHz, lo que da una tasa de datos de 1 GB/s. El tráfico por los enlaces consiste en paquetes, cada uno de los cuales tiene una cabecera de 14 bytes, 0, 16, 64 o 256 bytes de datos, y una suma de verificación de dos bytes. El tráfico de paquetes consiste en solicitudes y respuestas.

La memoria física del NUMA-Q 2000 se divide entre los nodos, de modo que cada página de memoria tiene una máquina base. Cada tarjeta quad puede contener hasta 4 GB de RAM y las líneas de caché son de 64 bytes, de modo que cada tarjeta quad guarda 2^{26} líneas de caché. Cuando una línea de caché no se está usando, se encuentra en sólo un lugar, la memoria base.

Sin embargo, las líneas de caché a menudo se encuentran en una o más cachés remotas, por lo que cada nodo necesita una **tabla de memoria local** de 2^{26} entradas para poder localizar todas sus líneas de caché, dondequiera que se encuentren. Una posibilidad habría sido tener un mapa de bits por cada entrada, que indique cuáles tarjetas IQ-Link tienen la línea. DASH opera de este modo. Sin embargo, SCI evita este mapa de bits porque no funciona bien cuando cambia la escala. (Recuerde que SCI puede manejar 64K nodos; tener 2^{26} entradas de directorio, cada una de 64K bits, sería demasiado costoso.)

En lugar de usar un mapa de bits, todas las copias de una línea de caché se “ensartan” en una lista doblemente enlazada. La entrada de la tabla de memoria local del nodo base indica cuál nodo tiene la cabeza de la lista. En la NUMA-Q 2000 un número de 6 bits es suficiente, porque hay cuando más 63 nodos. Para el sistema SCI máximo, un número de 16 bits es lo bastante grande. De cualquier modo, este esquema funciona mucho mejor en sistemas grandes que un mapa de bits al estilo DASH con 63 o 64K bits. Esta propiedad es lo que hace que SCI sea mucho más escalable que un diseño estilo DASH.

Además de la tabla de memoria local, cada tarjeta IQ-Link tiene un directorio con una entrada por cada línea de caché que tiene actualmente. Puesto que la caché tiene una capacidad de 32 MB y las líneas de caché son de 64 bytes, cada tarjeta IQ-Link puede contener hasta 2^{19} líneas de caché. Así pues, cada directorio tiene 2^{19} entradas, una para cada línea de caché.

Si una línea está en una sola caché, la tabla de memoria local del nodo base apunta al nodo en el que se encuentra la línea. Si después la línea se coloca en caché en un segundo nodo, el protocolo SCI hará que el directorio base apunte a la nueva entrada, que a su vez apunta a la entrada vieja. De este modo se forma una lista de dos elementos. A medida que nuevos nodos comparten la misma línea, se agregan, por turno, a la cabeza de la lista. Todos los nodos que contienen la línea de caché se enlazan así en una cadena arbitrariamente larga. Este encadenamiento se ilustra en la figura 8-33 para el caso de una línea de caché que se encuentra en los nodos 4, 9 y 22.

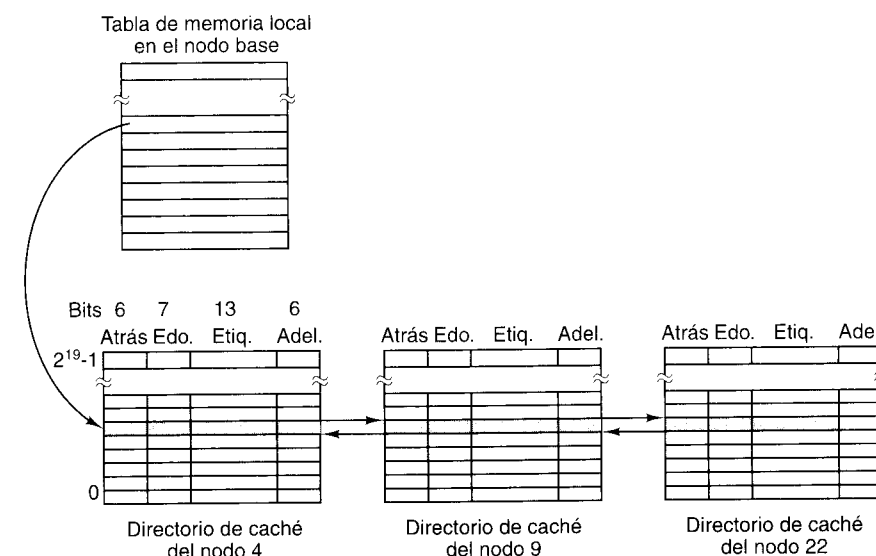


Figura 8-33. SCI encadena todos los contenedores de una línea de caché dada formando una lista doblemente enlazada. En este ejemplo se muestra una línea en caché en tres nodos.

Cada entrada de directorio consta de 36 bits. Seis de ellos apuntan al nodo que tiene la línea anterior de la cadena. Así mismo, seis bits apuntan al nodo que tiene la siguiente línea de la cadena. Un cero indica el fin de la cadena, y es por ello que el sistema máximo es de 63 nodos, no de 64. Se necesitan siete bits para registrar el estado de la línea.

Por último, se requieren 13 bits para el campo de etiqueta que identifica la línea. Recuerde que el sistema NUMA-Q 2000 más grande contiene $63 \times 2^{32} \approx 2^{38}$ bytes de RAM, por lo que hay casi 2^{32} líneas de caché. Los cachés son de correspondencia directa, así que con 2^{32} líneas de caché que corresponden a 2^{19} entradas de caché hay 2^{13} líneas que corresponden a cada entrada. Por tanto, se necesita una etiqueta de 13 bits para saber cuál es la que está ahí.

Toda línea de caché tiene una localidad fija en una y sólo una memoria que es su base. Estas líneas pueden estar en uno de tres estados: **NO EN CACHÉ**, **COMPARTIDA** y **MODIFICADA**. La terminología SCI para estos estados es **HOME**, **FRESH** y **GONE**, respectivamente, pero a fin de evitar complicaciones innecesarias seguiremos con la terminología anterior, pues ya hemos estado usándola. El estado **NO EN CACHÉ** implica que la línea no está en la caché de ninguna tarjeta IQ-Link, aunque podría estar en una caché local de la misma tarjeta quad. El estado **COMPARTIDA** implica que la línea está en al menos una caché de tarjeta IQ-Link, posiblemente en más, y que la memoria está actualizada. Por último, el estado **MODIFICADA** implica que la línea está en caché en alguna tarjeta IQ-Link y podría haberse modificado allí con lo que la memoria no estaría al día.

El la tarjeta IQ-Link los bloques pueden estar en cualquiera de 29 estados estables o en cualquiera de un gran número de estados transitorios. De hecho, se necesitan 7 bits para registrar el estado de cada línea de caché, como se muestra en la figura 8-33. Cada estado estable se compone de dos campos. El primero indica si la línea es la cabeza de su lista, el final de su lista, un miembro intermedio de su lista o la única entrada de la lista. El segundo contiene información acerca de si la línea está limpia, es exclusiva, etc.

El protocolo SCI define tres operaciones de gestión de listas: añadir un nodo a una lista, eliminar un nodo de una lista y eliminar todos los nodos excepto uno. La última operación se necesita cuando una línea compartida se modifica y por tanto se vuelve exclusiva.

El protocolo SCI tiene tres opciones de complejidad. El protocolo mínimo sólo permite una copia de cada línea en una caché, como en la figura 8-30. El protocolo típico permite poner en caché cada línea en un número ilimitado de nodos. El protocolo completo contiene varias características extra que mejoran el desempeño. La NUMA-Q implementa el protocolo típico, así que ése es el que explicaremos a continuación.

Consideremos el manejo de una instrucción **READ**. Si la CPU que ejecuta la instrucción no puede encontrar la línea de caché en su propia tarjeta, la tarjeta IQ-Link captura la solicitud en su nodo. La tarjeta envía un paquete de solicitud a la tarjeta IQ-Link base, que entonces busca en tablas el estado de la línea. Si el estado es **NO EN CACHÉ**, el estado se vuelve **COMPARTIDO** y la línea se devuelve de la memoria. Luego se actualiza la tabla de memoria local en el nodo base para que contenga una lista de un elemento que apunta al nodo en cuyo caché la línea está ahora.

Suponga ahora que el estado es **COMPARTIDO**. También se devuelve la línea de la memoria y el nuevo nodo se convierte en la cabeza de la lista, es decir, su número se coloca en la entrada de directorio en el nodo base. La entrada de directorio en el nodo solicitante se ajusta de modo que apunte al antiguo nodo cabeza. Este procedimiento alarga la lista en un elemento, y la nueva caché es la primera entrada.

Por último, imagine que la línea solicitada está en el estado **MODIFICADO**. El directorio base no puede devolver la línea de la memoria porque no tiene una copia actualizada. En vez de ello, le dice a la tarjeta IQ-Link solicitante en qué caché está la línea y le ordena traerla de ahí. También se modifica la tabla de memoria local de modo que apunte a la nueva ubicación.

Desde luego, el procesamiento de una instrucción **WRITE** es un poco diferente. Si el nodo solicitante ya está en la lista, deberá eliminar todas las demás entradas para convertirse en la única; si no está en la lista, deberá eliminar todas las entradas y luego insertarse en la

lista como única entrada. En todos los casos, el nodo termina siendo la única entrada válida de la lista, y es al que el directorio base apunta. Los pasos reales del manejo de instrucciones tanto **READ** como **WRITE** son muy complicados porque el protocolo debe funcionar correctamente aun cuando varias máquinas están realizando operaciones incompatibles con la misma línea. La necesidad de que el protocolo sea correcto aun durante operaciones concurrentes es la que da lugar a todos los estados transitorios. Lamentablemente, no tenemos espacio en este libro para una exposición completa del protocolo. Si desea conocer toda la historia, vea IEEE 1596.

8.3.7 Multiprocesadores COMA

Las máquinas NUMA y CC-NUMA tienen la desventaja de que las referencias a una memoria remota son mucho más lentas que las referencias a la memoria local. En CC-NUMA, el uso de cachés oculta hasta cierto punto esta diferencia en el desempeño. No obstante, si la cantidad de datos remotos requeridos excede considerablemente la capacidad de caché, ocurrirán constantemente fallos de caché y el desempeño será deficiente.

Así, tenemos la situación de que las máquinas UMA, como la Sun Enterprise 10000, tienen excelente desempeño pero su tamaño es limitado y su costo es considerable. Las máquinas NUMA se pueden escalar a tamaños un poco mayores pero requieren colocación de páginas manual o semiautomática, a menudo con resultados mixtos. El problema es que es difícil predecir cuáles páginas se necesitarán dónde y, en cualquier caso, las páginas suelen ser demasiado grandes como para estarlas moviendo de aquí para allá. Las máquinas CC-NUMA, como la Sequent NUMA-Q, podrían tener un desempeño pobre si muchas CPU necesitan muchos datos remotos. En síntesis, todos estos diseños tienen limitaciones graves.

Un tipo distinto de multiprocesador trata de superar todos estos problemas utilizando la memoria principal de cada CPU como caché. En este diseño, llamado **acceso sólo a memoria caché** (COMA, *Cache Only Memory Access*), las páginas no tienen máquinas base fijas, como sucede en las máquinas NUMA y CC-NUMA. De hecho, las páginas no son importantes.

En vez de ello, el espacio de direcciones físico se divide en líneas de caché, que migran dentro del sistema según se les necesita. Los bloques no tienen máquinas base. Al igual que los nómadas de algunos países del Tercer Mundo, el hogar es el lugar donde están en ese momento. Una memoria que simplemente atrae líneas conforme las necesita se denomina **memoria de atracción**. Utilizar la RAM principal como caché de gran capacidad eleva considerablemente la tasa de aciertos, y por ende el desempeño.

Claro que, como ya sabemos, nada es gratis. Los sistemas COMA introducen dos problemas nuevos:

1. ¿Cómo se localizan las líneas de caché?
2. Cuando una línea se elimina de la memoria, ¿qué sucede si es la última copia?

El primer problema tiene que ver con el hecho de que, una vez que la MMU ha traducido una dirección virtual a una dirección física, si la línea no está en la verdadera caché de hardware, no es fácil determinar si está o no en la memoria principal. El hardware de paginación no

ayuda aquí porque cada página se compone de muchas líneas de caché individuales que divagan por la máquina individualmente. Además, aunque se sepa que una línea no está en la memoria principal, ¿dónde está entonces? No es posible preguntarle a la máquina base, porque no hay una máquina base.

Se han propuesto algunas soluciones al problema de la localización. Para ver si una línea de caché está o no en la memoria principal, podría agregarse hardware nuevo que siga la pista a la etiqueta de cada línea que está en caché. Luego, la MMU podría comparar la etiqueta de la línea requerida con las etiquetas de todas las líneas de caché en la memoria en busca de un acierto. Esta solución requiere hardware adicional.

Una solución un poco distinta sería mapear páginas enteras pero sin exigir que estén presentes todas las líneas de caché. En esta solución, el hardware necesitaría un mapa de bits por cada página, con un bit por cada línea de caché para indicar la presencia o ausencia de esa línea. En este diseño, llamado **COMA simple**, si una línea de caché está presente, deberá estar en la posición correcta de su página, pero si no está presente, cualquier intento de usarla causará una trampa que permitirá al software ir a buscarla y traerla a la memoria.

Esto nos lleva a encontrar líneas que son realmente remotas. Una solución es asignar a cada página una máquina base en términos de dónde está su entrada de directorio, pero no dónde están los datos. Así, puede enviarse un mensaje a la máquina base para al menos localizar la línea de caché. Otros esquemas implican organizar la memoria en forma de árbol y buscar hacia arriba hasta encontrar la línea.

El segundo problema de la lista anterior tiene que ver con no eliminar la última copia. Como en CC-NUMA, una línea de caché podría estar en varios nodos al mismo tiempo. Cuando ocurre un fallo de caché, se debe traer una línea, lo que normalmente implica que hay que desalojar otra línea. ¿Qué sucede si la línea escogida para el desalojo es la última copia? En tal caso, no podemos eliminarla.

Una solución es regresar al directorio y verificar si hay otras copias. Si las hay, la línea puede desalojarse sin peligro; si no, hay que pasarla a otro lugar. Otra solución consiste en marcar una copia de cada línea de caché como copia maestra y nunca desalojarla. Esta solución evita tener que consultar el directorio. Tomando todo en cuenta, COMA promete alcanzar un mejor desempeño que CC-NUMA, pero no se han construido muchas máquinas COMA, así que se necesita más experiencia. Las dos máquinas COMA que se han construido hasta ahora son la KSR-1 (Burkhardt *et al.*, 1992) y la Data Diffusion Machine (Hagersten *et al.*, 1992). Se puede encontrar más información acerca de máquinas COMA en (Falsafi y Wood, 1997; Joe y Hennessy, 1994; Morin *et al.*, 1996; y Saulsbury *et al.*, 1995).

8.4 MULTICOMPUTADORAS DE TRANSFERENCIA DE MENSAJES

Como vimos en la figura 8-14, los dos tipos de procesadores paralelos MIMD son los multiprocesadores y las multicomputadoras. En la sección anterior estudiamos los multiprocesadores. Vimos que, desde la perspectiva del sistema operativo, los multiprocesadores parecen tener una memoria compartida a la que se accede con instrucciones **LOAD** y **STORE**

ordinarias. Como vimos, esta memoria compartida se puede implementar de varias maneras, que incluyen espiar buses, conmutadores de barras cruzadas, redes de conmutación multietapas, y diversos esquemas basados en directorios. No obstante, los programas escritos para un multiprocesador pueden acceder a cualquier localidad de memoria sin saber nada acerca de la topología interna ni del esquema de implementación. Esta ilusión es lo que hace a los multiprocesadores tan atractivos.

Por otra parte, los multiprocesadores también tienen sus limitaciones, y ésta es la razón por la que las multicomputadoras también son importantes. En primer lugar, los multiprocesadores no pueden escalar a tamaños muy grandes. Ya vimos la enorme cantidad de hardware que Sun tuvo que usar para escalar el Enterprise 10000 a 64 CPU. La Sequent NUMA-Q llega a 256 CPU, pero el precio que se paga es un tiempo de acceso a memoria no uniforme. En contraste, ahora estudiaremos dos multicomputadoras que tienen 2048 y 9152 CPU, respectivamente. Pasarán años antes de que alguien construya un multiprocesador comercial con 9000 nodos, y para entonces ya se estarán usando multicomputadoras de 100,000 nodos.

Además, la contención por la memoria en un multiprocesador puede afectar mucho el desempeño. Si 100 CPU están tratando de leer y escribir las mismas variables constantemente, la lucha por las distintas memorias, buses y directorios puede perjudicar enormemente el desempeño.

Como consecuencia de éstos y otros factores, ha surgido un gran interés por construir y usar computadoras paralelas en las que cada CPU tiene su propia memoria privada, no accesible directamente para ninguna otra CPU. Éstas son las multicomputadoras. Los programas en CPU de multicomputadoras interactúan empleando primitivas como **send** y **receive** para transferir explícitamente mensajes, porque no pueden acceder directamente a la memoria de otra CPU con instrucciones **LOAD** y **STORE**. Esta diferencia cambia totalmente el modelo de programación.

Cada nodo de una multicomputadora consiste en una o unas cuantas CPU, algo de RAM, (tal vez compartida entre las CPU de ese nodo únicamente), un disco u otros dispositivos de E/S, y un procesador de comunicaciones. Los procesadores de comunicaciones están conectados por una red de interconexión de alta velocidad de los tipos que vimos en la sección 8.1.2. Se usan muchas topologías, esquemas de conmutación y algoritmos de enrutamiento distintos. Lo que todas las multicomputadoras tienen en común es que cuando un programa de aplicación ejecuta la primitiva **send**, se avisa al procesador de comunicaciones y éste transmite un bloque de datos de usuario a la máquina de destino (tal vez después de pedir y obtener permiso). En la figura 8-34 se muestra una multicomputadora genérica.

Hay multicomputadoras de todas las formas y tamaños, por lo que es difícil dar una taxonomía nítida. No obstante, destacan dos "estilos" generales: los MPP y los COW. A continuación estudiaremos cada uno de estos tipos.

8.4.1 Procesadores masivamente paralelos (MPP)

La primera categoría consiste en los **procesadores masivamente paralelos** (MPP, *Massively Parallel Processors*), que son enormes supercomputadoras que cuestan muchos millones de dólares. Éstos se usan en la ciencia, ingeniería y la industria para cálculos de gran envergadu-

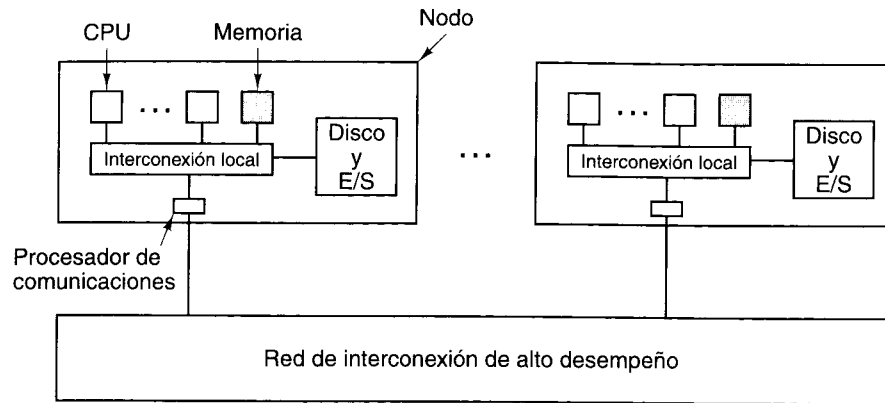


Figura 8-34. Multicomputadora genérica.

ra, para manejar números enormes de transacciones por segundo, o como bodega de datos (almacenamiento y gestión de bases de datos inmensas). En un principio los MPP se usaron primordialmente como supercomputadoras científicas, pero ahora casi todos se usan en entornos comerciales. En cierto sentido, estas máquinas son las sucesoras de las poderosas *mainframes* de los años sesenta (pero la conexión es tenue, algo así como que un paleontólogo diga que un gorrión es el sucesor de *Tyrannosaurus Rex*). En gran medida, los MPP han desplazado a las máquinas SIMD, las supercomputadoras vectoriales y los procesadores de arreglos en la cima de la cadena alimenticia digital.

Casi todas estas máquinas usan CPU estándar como procesadores. Las preferidas son la línea Pentium de Intel, el UltraSPARC de Sun, el IBM RS/6000 y el DEC Alpha. Lo que distingue a los MPP es su uso de una red de interconexión propia de alto desempeño diseñada para transferir mensajes con baja latencia y gran ancho de banda. Ambas cosas son importantes porque casi todos los mensajes son pequeños (mucho menos de 256 bytes) pero casi todo el tráfico total se debe a mensajes grandes (más de 8 KB). Los MPP también usan una gran cantidad de software y bibliotecas propios.

Otro aspecto que caracteriza a los MPP es su enorme capacidad de E/S. Los problemas lo bastante grandes como para justificar el uso de MPP casi siempre tienen cantidades enormes de datos que procesar, a veces del orden de terabytes. Estos datos deben distribuirse entre muchos discos y tienen que desplazarse dentro de la máquina a gran velocidad.

Por último, otro aspecto característico de los MPP es su atención a la tolerancia ante fallos. Con miles de CPU, son inevitables varios fallos por semana. Abortar una serie de procesamiento de 18 horas porque una CPU se cayó es inaceptable, sobre todo cuando cabe esperar un fallo de esos cada semana. Por ello, los MPP grandes siempre cuentan con hardware y software especial para monitorear el sistema, detectar fallos y recuperarse de ellos sin tener que parar.

Aunque sería agradable pasar ahora al estudio de los principios generales del diseño de los MPP, la verdad es que no hay muchos principios. En lo esencial, un MPP es una colección

de nodos de computación más o menos estándar unidos por una interconexión muy rápida de los tipos que ya estudiamos. Lo que haremos, entonces, es examinar dos ejemplos de MPP: el Cray T3E y el Intel/Sandia Option Red.

El Cray T3E

La familia Cray T3E, sucesora de la T3D, incluye las más modernas supercomputadoras de una línea que se remonta a la innovadora 6600 de Seymour Cray a mediados de los años sesenta. Los diversos modelos, el T3E, T3E-900 y T3E-1200, tienen idéntica arquitectura, y sólo difieren en su precio y su desempeño (por ejemplo, 600, 900 o 1200 megaFLOPs por CPU). Un megaFLOP es un millón de operaciones de punto flotante por segundo. A diferencia de la 6600 y la Cray-1, que tenían poco paralelismo, estas máquinas son masivamente paralelas, con hasta 2048 CPU por sistema. Usaremos el término "T3E" para referirnos a toda la familia en un sentido genérico, pero las cifras de desempeño que citemos serán para la T3E-1200. Estas máquinas son vendidas por Cray Research, una división de Silicon Graphics, y se usan para modelado del clima, diseño de fármacos, exploración petrolera y muchas otras aplicaciones.

La CPU que utiliza el T3E es la DEC Alpha 21164, que es un procesador RISC super-escalar capaz de emitir cuatro instrucciones por ciclo de reloj. Esta CPU trabaja a 300, 450 o 600 MHz, dependiendo del modelo. De hecho, la velocidad del reloj es la diferencia primordial entre los distintos modelos de T3E. El Alpha es una máquina de 64 bits, con registros de 64 bits. No obstante, las direcciones virtuales están limitadas a 43 bits, y las físicas, a 40 bits, lo que permite acceder hasta 1 TB de memoria física.

Cada Alpha tiene dos niveles de cachés en el chip. El de primer nivel tiene 8 KB para instrucciones y 8 KB para datos. El de segundo nivel es una caché asociativa por conjuntos de tres vías, unificado, de 96 KB, tanto para instrucciones como para datos. No existe una caché en el nivel de tarjeta. Las cachés sólo contienen instrucciones y datos de la RAM local, que puede ser de hasta 2 GB por CPU. Con un máximo de 2048 CPU, la memoria total del sistema puede alcanzar los 4 TB.

Cada Alpha está encapsulado por circuitos especiales que reciben el nombre de **concha**, como se muestra en la figura 8-35. La concha contiene la memoria, el procesador de comunicaciones y 512 registros especiales llamados **registros E**. Estos registros se pueden cargar con direcciones de memoria remota para leer o escribir palabras (o bloques de 8 palabras) de memoria remota. Por tanto, la T3E sí tiene acceso a la memoria remota, pero no a través de las instrucciones **LOAD** y **STORE** normales. Esto hace que sea una especie de híbrido entre una máquina NC-NUMA y un MPP, pero tiene más rasgos de MPP, ya que el sistema operativo ciertamente sabe que no puede leer o escribir en la memoria remota como si fuera local. La coherencia de la memoria está garantizada porque las palabras que se leen de una memoria remota no se colocan en caché; la memoria remota es la única copia.

Los nodos T3E se conectan de dos formas distintas, como se muestra en la figura 8-35. La interconexión principal es un toroide 3D dúplex. Por ejemplo, un sistema con 512 nodos podría configurarse como un cubo de $8 \times 8 \times 8$. Cada nodo del toroide 3D tiene seis enlaces

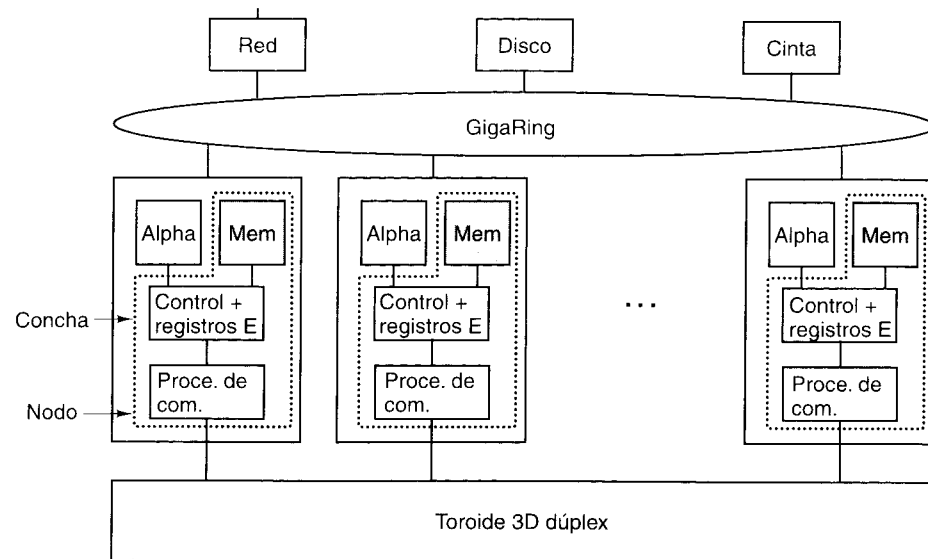


Figura 8-35. El T3E de Cray Research.

con otros nodos: sus vecinos al norte, oriente, sur, poniente, arriba y abajo. La tasa de transferencia por estos enlaces es de 480 MB/s en cada dirección.

Los nodos también están conectados por uno o más **GigaRings**, el subsistema de E/S de gran ancho de banda conmutado por paquetes. Los nodos lo usan para comunicarse entre sí y con las redes, discos y otros periféricos enviando paquetes de hasta 256 bytes por él. Cada GigaRing consiste en un par de anillos contrarrotatorios con una anchura de 32 bits que conectan los nodos de CPU con nodos de E/S especiales. Estos últimos contienen ranuras para insertar tarjetas de red (por ejemplo, HIPPI, Ethernet, ATM, FDDI o canal de fibra), discos, cintas y otros dispositivos.

Como puede haber hasta 2048 nodos en un sistema T3E, ocurren fallos con cierta regularidad. Para resolver este problema, por cada 128 nodos de usuario, el sistema contiene un nodo de repuesto. Los nodos muertos pueden sustituirse por nodos de repuesto mientras está funcionando el sistema, sin necesidad de un rearranque. Además de los nodos de usuario y los nodos de repuesto, algunos nodos se dedican a ejecutar servidores de sistema operativo porque los nodos de usuario no ejecutan el sistema operativo completo, sino sólo un núcleo básico. El sistema operativo que se usa es una versión de UNIX.

El Intel/Sandia Option Red

Las computadoras del extremo superior y las fuerzas armadas han estado entrelazadas en Estados Unidos desde 1943, cuando el ejército contrató a John Mauchly para construir ENIAC, la primera computadora electrónica moderna. La relación entre las fuerzas armadas estado-

unidenses y la computación de alta velocidad continúa. A mediados de los años noventa, los departamentos de la Defensa y de Energía de Estados Unidos lanzaron un ambicioso programa para efectuar avances en supercomputación diseñando cinco MPP que operarían a velocidades de 1, 3, 10, 30 y 100 teraflops/s, respectivamente. Para hacer una comparación con una computadora más conocida, 100 teraflops (10^{14} operaciones de punto flotante/s) es 500,000 veces la potencia de un Pentium Pro de 200 MHz.

A diferencia de la Cray T3E, que es un producto comercial normal que se compra en la tienda (aunque hay que llevar mucho dinero), las máquinas de teraflops son sistemas únicos otorgados mediante licitación competitiva por el Departamento de Energía, que opera los laboratorios nacionales de Estados Unidos. Intel ganó el primer contrato; IBM ganó los dos siguientes. Si planea licitar por contratos futuros, 80 millones de dólares sería una buena cifra para tenerla en mente. Dado que estas máquinas son para uso militar (primordialmente para simular explosiones nucleares), alguna persona imaginativa del Pentágono pensó en nombres patrióticos para las tres primeras: rojo, blanco y azul (los colores de la bandera estadounidense); lo malo es que se equivocó en el orden: la máquina de 1 TFLOPS/s (instalada en el Sandia National Laboratory en diciembre de 1996) se llama **Option Red** (opción roja), y las dos siguientes se llaman **Option Blue** (1999) y **Option White** (1900), respectivamente. En el resto de esta sección nos concentraremos en la primera de las máquinas de teraflops, la Option Red.

La máquina Option Red consiste en 4608 nodos dispuestos en una malla tridimensional. Las CPU están instaladas en dos tipos de tarjetas. Las tarjetas **kestrel** se usan como nodos de cómputo, y las tarjetas **eagle** (águila) se usan como nodos de servicio, disco, red y arranque. Hay 4536 nodos de cómputo, 32 nodos de servicio, 32 nodos de disco, 6 nodos de red y dos nodos de arranque.

La tarjeta kestrel, que se ilustra en la figura 8-36(a), contiene dos nodos lógicos, cada uno con dos CPU Pentium Pro de 200 MHz y 64 MB de RAM compartida. Cada nodo kestrel tiene su propio bus local de 64 bits y su propio **chip de interfaz con red** (NIC, *Network Interface Chip*). Los dos NIC están interconectados, por lo que sólo uno de ellos está vinculado realmente con la red de interconexión, a fin de hacer más compacto el paquete (aun así, la máquina ocupa un área de piso de 160 m²). Las tarjetas eagle también tienen CPU Pentium Pro, pero sólo dos por tarjeta. Estas tarjetas tienen también capacidad de E/S adicional.

Las tarjetas están interconectadas por una estructura de cuadrícula de $32 \times 38 \times 2$ dispuesta en forma de dos planos 32×38 con todos los nodos correspondientes interconectados (el tamaño de la cuadrícula lo determinaron consideraciones de empaque, por lo que no todos los puntos de la cuadrícula tienen tarjetas). En cada punto de cuadrícula hay un chip enrutador especial con seis enlaces: norte, oriente, sur, poniente, al otro plano y a la tarjeta kestrel o eagle conectada a él. Cada enlace puede transferir 400 MB/s en cada dirección simultáneamente. Los chips enrutadores utilizan enrutamiento por túnel a fin de reducir la latencia.

Se usa enrutamiento dimensional; los paquetes pueden pasar primero al otro plano, luego en dirección oriente-poniente, luego norte-sur y por último al plano correcto, si no están ya ahí. La razón para efectuar dos traslados entre los planos es la tolerancia ante fallos. Supongamos que un paquete simplemente tiene que llegar a su vecino que está un brinco al norte, pero el enlace entre ellos está roto. En vez de ello, el mensaje puede pasar al otro plano, dar un brinco al norte y regresar al mismo plano, pasando por alto el enlace defectuoso.

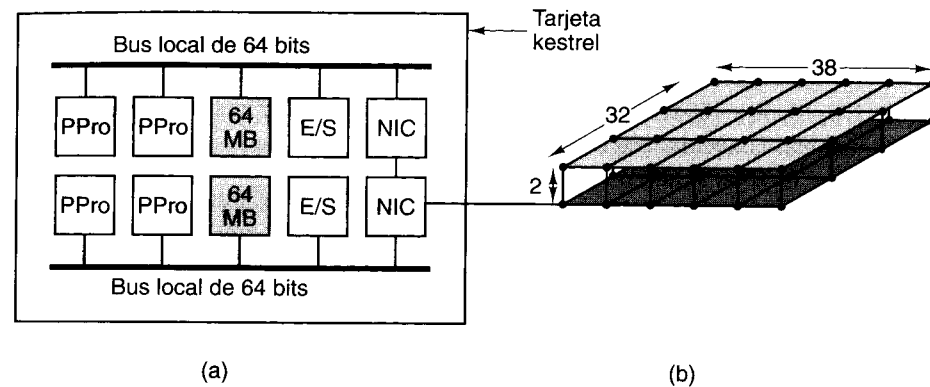


Figura 8-36. El sistema Intel/Sandia Option Red. (a) La tarjeta kestrel. (b) La red de interconexión.

El sistema se divide lógicamente en cuatro particiones: servicio, cómputo, E/S y sistema. Los nodos de servicio son máquinas UNIX de tiempo compartido, de propósito general, que permiten a los programadores escribir y depurar sus programas. Los nodos de cómputo realizan el trabajo pesado (por ejemplo, ejecutan las grandes aplicaciones numéricas para las que se ordenó la máquina). Éstos no ejecutan un sistema UNIX completo, sino un micronúcleo llamado **cougar**. Los nodos de E/S administran los 640 discos con más de 1 TB de datos. Hay dos conjuntos independientes de nodos de E/S, uno para trabajos militares confidenciales y otro para trabajos no confidenciales. Los dos conjuntos se instalan y desinstalan manualmente, de modo que sólo un conjunto está conectado en un momento dado, a fin de evitar la fuga de información de los discos confidenciales a los no confidenciales. Por último, los nodos de sistema sirven para arrancar el sistema.

8.4.2 Cúmulos de estaciones de trabajo (COW)

El otro estilo de multicomputadora es el **cúmulo de estaciones de trabajo** (COW, *Cluster Of Workstations*) o **red de estaciones de trabajo** (NOW, *Network Of Workstations*) (Anderson et al., 1995; Martin et al., 1997). Normalmente, un COW consiste en unos cuantos cientos de PC o estaciones de trabajo conectadas por una tarjeta de red comercial. La diferencia entre un MPP y un COW es análoga a la diferencia entre una *mainframe* y una PC. Ambas tienen una CPU, ambas tienen RAM, ambas tienen discos, ambas tienen un sistema operativo, etc. La *mainframe* simplemente tiene cosas más rápidas (con la posible excepción del sistema operativo). Sin embargo, en el aspecto cualitativo los dos sistemas se sienten diferentes y se usan y controlan de forma distinta. Esta misma diferencia existe entre los MPP y los COW.

La fuerza impulsora de los COW es la tecnología. Las CPU que se usan en los MPP no son más que procesadores comerciales que cualquiera puede comprar. La T3E usa Alphas; la Option Red usa Pentium Pros. No hay nada de especial ahí. Esos sistemas también usan DRAMs ordinarias y ejecutan UNIX. Estas cosas tampoco tienen nada fuera de lo común.

Históricamente, el elemento clave que hizo especiales a los MPP fue su interconexión de alta velocidad, pero la reciente aparición en el mercado de interconexiones de alta velocidad que se venden en tiendas ha comenzado a cerrar la brecha. Por ejemplo, el grupo de investigación del autor ha ensamblado un COW llamado **DAS** que consiste en 128 nodos, cada uno de los cuales contiene un Pentium Pro de 200 MHz y 128 MB de RAM (vea <http://www.cs.vu.nl/~bal/das.html>). Los nodos están conectados por un toroide bidimensional con enlaces que pueden transferir 160 MB/s en ambas direcciones a la vez. Estas especificaciones no son muy distintas de las de Option Red: los enlaces tienen la mitad de la velocidad, pero la RAM por nodo es dos veces mayor. La única diferencia real es que Sandia tiene un presupuesto mayor; técnicamente, los dos sistemas son prácticamente idénticos.

La ventaja de construir un COW en lugar de un MPP es que el primero se construye en su totalidad con componentes comerciales que se pueden adquirir y ensamblar localmente. Estas piezas tienen series de producción grandes y se benefician de las economías de escala; además, existen en un mercado competitivo, lo que tiende a elevar el desempeño y bajar el precio. En síntesis, es probable que los COW releguen a los MPP a nichos cada vez más diminutos, así como las PC han convertido a las *mainframes* en artículos de especialidad esotéricos.

Aunque existen muchos tipos de COW, dos especies dominan: los centralizados y los descentralizados. Un COW centralizado es un cúmulo de estaciones de trabajo o PC montadas en un anaquel enorme en un solo recinto. A veces se empaquetan de forma mucho más compacta que lo normal a fin de reducir el tamaño físico y la longitud de los cables. Por lo regular, las máquinas son homogéneas y no tienen más periféricos que tarjetas de red y posiblemente discos. Gordon Bell, el diseñador de la PDP-11 y la VAX, ha llamado a tales máquinas **estaciones de trabajo sin cabeza** (porque no tienen dueño). Estuvimos tentados a llamarlas COW sin cabeza, pero temimos que tal término ofendería a demasiadas vacas sagradas, por lo que nos abstuvimos.

Los COW descentralizados consisten en estaciones de trabajo dispersas dentro de un edificio o campus universitario. Casi todas ellas están ociosas muchas horas al día, sobre todo de noche, y por lo regular están conectadas a una LAN. Es común que tales estaciones de trabajo sean heterogéneas y cuenten con un surtido completo de periféricos, aunque tener un COW con 1024 ratones no es realmente mucho mejor que un COW con cero ratones. Lo más importante es que muchas de las estaciones de trabajo tienen dueños con vínculos emocionales con sus máquinas y no les hace mucha gracia que algún astrónomo trate de simular el “big bang” con la suya. Usar estaciones de trabajo ociosas para formar un COW implica tener alguna forma de trasladar trabajos a otra máquina cuando el dueño llega y reclama la suya. Tal migración de trabajos es posible pero hace más complejo el software.

8.4.3 Planificación

Una pregunta obvia es: “¿Qué diferencia hay entre un COW descentralizado y una LAN llena de máquinas de usuario?” La respuesta tiene que ver con el software y la utilización, no con el hardware. En una LAN, los usuarios tienen máquinas personales y las usan para todo su trabajo. En contraste, un COW descentralizado es un recurso compartido al cual los usuarios

pueden presentar trabajos que requieren múltiples CPU. Para atender solicitudes de múltiples usuarios, cada uno de los cuales necesita múltiples CPU, un COW (centralizado o descentralizado) necesita un planificador de trabajos.

En el modelo más sencillo, el planificador requiere que cada trabajo especifique cuántas CPU necesita. Luego los trabajos se ejecutan en orden FIFO estricto, como se muestra en la figura 8-37(a). En este modelo, una vez que se inicia un trabajo, se determina si hay suficientes CPU disponibles para iniciar el siguiente trabajo de la cola de entrada. Si las hay, el trabajo se inicia, etc.; si no, el sistema espera hasta que se desocupan suficientes CPU. Por cierto, aunque hemos sugerido que este COW tiene ocho CPU, bien podría tener 128 CPU que se asignan en unidades de 16 (lo que da ocho grupos de CPU), o alguna otra combinación.

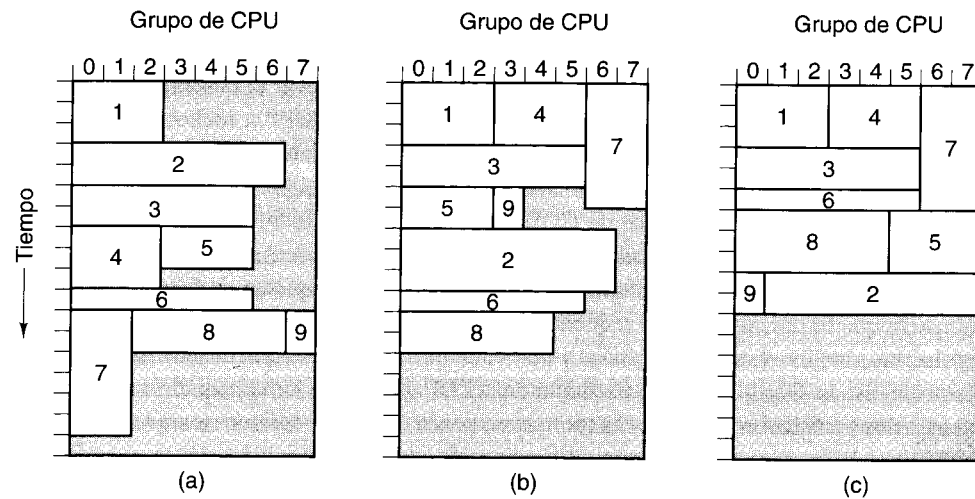


Figura 8-37. Planificación de un COW. (a) FIFO. (b) Sin bloqueo de cabeza de línea. (c) Mosaico. Las áreas sombreadas indican CPU ociosas.

Un mejor algoritmo de planificación evita el bloqueo de cabeza de línea pasando por alto trabajos que no encajan y escogiendo el primero que sí quepa. Cada vez que un trabajo termina, la cola de trabajos restantes se examina en orden FIFO. Este algoritmo da el resultado de la figura 8-37(b).

Un algoritmo de planificación todavía más sofisticado requiere que cada trabajo presentado especifique sus dimensiones, es decir, cuántas CPU durante cuántos minutos. Con esa información, el planificador de trabajos puede tratar de cubrir el rectángulo CPU-tiempo con "mosaicos". Esta técnica es eficaz sobre todo cuando los trabajos se presentan durante el día para ejecutarse durante la noche, de modo que el planificador cuenta con toda la información acerca de todos los trabajos por adelantado y puede ejecutarlos en el orden óptimo, como se ilustra en la figura 8-37(c).

Aunque hemos bosquejado dos extremos en el mundo de las multicomputadoras, enormes MPP patentados y sencillos COW "hechos en casa", existe mucho terreno entre ellos. En

particular, muchos fabricantes de computadoras también entienden que pueden construir COW con componentes que se venden comercialmente y venderlos como sistemas completos. El resultado ha sido una rápida expansión del mercado.

Redes de interconexión comerciales

Puesto que los COW tienen éxito o fracasan dependiendo de la cantidad de las tarjetas de red insertables que pueden obtenerse comercialmente, vale la pena examinar algunas de las tecnologías que están disponibles. Nuestro primer ejemplo es Ethernet, que viene en tres versiones, lenta, de media velocidad y rápida (o, más correctamente, Ethernet clásica, Ethernet rápida y Ethernet de gigabits). Estos tres tipos operan a 10, 100 y 1000 Mbps (1.25, 12.5 y 125 MB/s), respectivamente. Los tres son compatibles en términos de medios, formato de paquetes y protocolos. Sólo el desempeño es diferente.

Cada computadora de una Ethernet tiene un chip de Ethernet, por lo regular en una tarjeta insertable. En su forma más antigua y sencilla, un cable de la tarjeta se insertaba a la fuerza en medio de un cable de cobre grueso, en una disposición conocida como **derivación vampiro** (¿quién, si no un vampiro, mordería un cable coaxial?). Posteriormente se comenzaron a usar cables más delgados y conectores con forma de T. En todos los casos, las tarjetas Ethernet de todas las máquinas están conectadas eléctricamente, como si se hubieran soldado juntas. En la figura 8-38(a) se ilustra la situación de tres máquinas conectadas a una Ethernet.

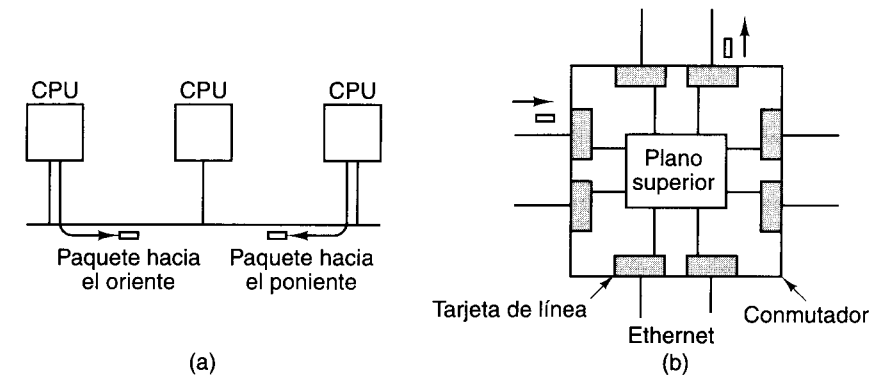


Figura 8-38. (a) Tres computadoras en una Ethernet. (b) Conmutador Ethernet.

En el protocolo Ethernet básico, cuando una máquina quiere enviar un paquete primero sondea el cable para ver si alguien más está transmitiendo actualmente. Si el cable está ocioso, la máquina envía su paquete. Si el cable está ocupado, la máquina espera hasta que la transmisión termina antes de transmitir. Si dos transmisores comienzan a transmitir en el mismo instante, ocurre una colisión. Los dos detectan la colisión, se detienen, esperan cada uno un tiempo aleatorio, y lo vuelven a intentar. Si ocurre otra colisión, se paran y lo vuelven a intentar otra vez, duplicando el tiempo de espera medio después de cada colisión sucesiva.

Un problema con la Ethernet básica de la figura 8.38(a) es que las derivaciones vampiro se rompen fácilmente y detectar un fallo de cable no es tarea fácil. Por esta razón comenzó a

usarse un diseño modificado, en el que un cable de cada máquina llega a un **centro**. Eléctricamente, este diseño es igual al primero, pero el mantenimiento es más fácil porque los cables pueden desconectarse del centro uno por uno hasta que se aísla el cable defectuoso.

Una vez que todas las máquinas tenían un cable tendido hacia el cuarto de máquinas, se hizo posible un tercer diseño: **Ethernet conmutada**, el cual se ilustra en la figura 8-38(b). Aquí el centro es sustituido por un dispositivo que contiene un plano posterior en el que pueden insertarse **tarjetas de línea**. Cada tarjeta de línea acepta una o más Ethernets, y diferentes tarjetas pueden manejar diferentes velocidades, lo que permite conectar entre sí Ethernets clásica, rápida y de Gigabits.

Cuando un paquete llega a una tarjeta de línea, se almacena temporalmente ahí hasta que la tarjeta solicita y le es concedido acceso al plano posterior, el cual funciona de forma parecida a un bus. Una vez que el paquete se ha transferido a la tarjeta de línea a la que está conectada la máquina de destino, se le puede enviar ahí. Si cada tarjeta de línea tiene sólo una Ethernet, y ésta sólo tiene una máquina, ya no ocurren colisiones, aunque sigue siendo posible perder un paquete por desbordamiento de un buffer en una tarjeta de línea. Si se usa como interconexión de multicomputadora, la Ethernet de gigabits conmutada, con una máquina por Ethernet y un plano posterior muy rápido, tiene un desempeño potencial (al menos en términos de ancho de banda) de la cuarta parte del desempeño de los enlaces de una T3E, pero cuesta menos de una cuarta parte del precio.

Si esto suena demasiado bueno para ser verdad, lo es. Si el número de tarjetas de línea es grande, un plano posterior con un precio razonable no podrá manejar la carga, y será necesario colocar varias máquinas en cada Ethernet, con lo que se reintroducen colisiones. Con todo, en términos de potencia por dólar (es decir, precio/desempeño), la Ethernet de gigabits conmutada es un competidor de respeto.

Una segunda tecnología de interconexión es **modo de transferencia asincrónico**, (ATM, *Asynchronous Transfer Mode*). ATM fue inventado por un consorcio de alcance mundial formado por compañías telefónicas y otros organismos interesados, como sustituto totalmente digital del sistema telefónico actual. La idea era que cada teléfono y cada computadora del mundo estuvieran conectados por un tubo de bits digital libre de errores con una velocidad de por lo menos 155 Mbps, y posteriormente 622 Mbps. Para no hacer largo el cuento, fue más fácil proponer esto que hacerlo realidad. No obstante, muchas compañías ahora fabrican tarjetas insertables para PC que operan a 155 Mbps o 622 Mbps. La segunda velocidad, llamada **OC-12**, es probablemente lo bastante buena como para ser la interconexión de una multicomputadora.

El alambre o fibra que sale de una tarjeta ATM llega a un conmutador ATM, un dispositivo con cierto parecido a un conmutador Ethernet en cuanto a que los paquetes llegan y se almacenan temporalmente en tarjetas de línea hasta que pueden pasar a la tarjeta de línea de salida para ser transmitidos a su destino. No obstante, Ethernet y ATM tienen algunas diferencias importantes.

En primer lugar, dado que ATM se diseñó para reemplazar al sistema telefónico, está orientado hacia las conexiones. Antes de enviar un paquete a un destino, la máquina de origen tiene que establecer un circuito virtual desde el origen, a través de uno o más conmutadores ATM, hasta el destino. En la figura 8-39 se muestran dos circuitos virtuales. En contraste, Ethernet no usa circuitos virtuales. Puesto que establecer un circuito virtual toma un tiempo

apreciable, para poder usarse en una interconexión de multicomputadora cada máquina establecería normalmente un circuito virtual con todas las demás máquinas desde el principio y los usaría durante toda la ejecución. Los paquetes que se envían en sucesión por un circuito virtual nunca se entregan en desorden, pero los buffers de las tarjetas de línea pueden desbordarse, como en el caso de la Ethernet conmutada, de modo que la entrega no está garantizada con ninguna de las dos tecnologías.

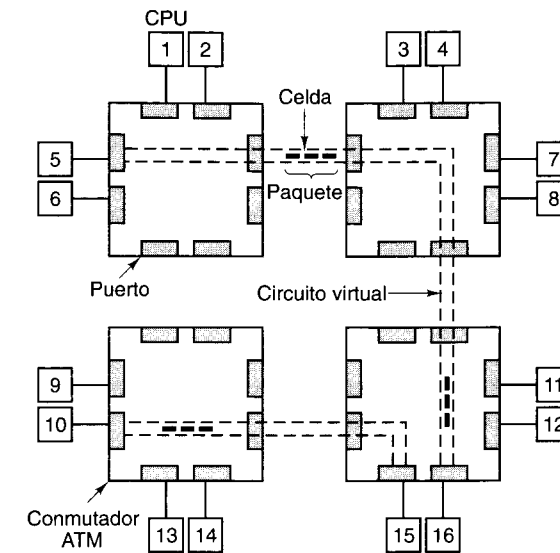


Figura 8-39. Dieciséis CPU conectadas por cuatro conmutadores ATM. Se muestran dos circuitos virtuales.

Otra diferencia entre ATM y Ethernet es que Ethernet puede transmitir paquetes enteros (hasta 1500 bytes de datos) como una sola unidad. En ATM todos los paquetes se dividen en celdas de 53 bytes (ATM fue diseñado por un comité; ¿qué esperaba usted?). Cinco de estos bytes son campos de cabecera que indican a cuál circuito virtual pertenece la celda, qué tipo de celda es, qué prioridad tiene, y varias otras cosas. La porción de carga útil es de 48 bytes. La división de paquetes en celdas y el reensamblado en el otro extremo corre por cuenta de hardware.

Nuestro tercer ejemplo de red de interconexión comercial es Myrinet, una tarjeta insertable producida por una compañía de California que se ha popularizado entre muchos diseñadores de COW (Boden *et al.*, 1995). Myrinet usa el mismo modelo que Ethernet y ATM en cuanto a que cada tarjeta insertable se conecta con un conmutador, y los conmutadores pueden interconectarse en una topología arbitraria. Los enlaces Myrinet son dúplex, de 1.28 Gbps en cada sentido. El tamaño de los paquetes no está limitado por el hardware, y los conmutadores son de barras totalmente cruzadas, lo que les confiere baja latencia y también alto ancho de banda.

Lo que hace a Myrinet interesante para los diseñadores de COW es que las tarjetas insertables contienen una CPU programable y una buena cantidad de RAM. Aunque Myrinet viene con un sistema operativo estándar, una docena de grupos universitarios han escrito el

suyo propio. Muchos de éstos añaden características diseñadas para ampliar la funcionalidad y mejorar el desempeño (por ejemplo, Blumrich *et al.*, 1995; Pakin *et al.*, 1997; Verstoep *et al.*, 1996). Entre las características típicas están protección, control de flujo, difusión y multidifusión confiables, y la capacidad para ejecutar una parte del código de la aplicación en la tarjeta.

8.4.4 Software de comunicación para multicomputadoras

Para programar una multicomputadora se requiere software especial, por lo regular bibliotecas, que se encargue de la comunicación entre procesos y la sincronización. En esta sección hablaremos un poco acerca de ese software. En su mayor parte, los mismos paquetes de software se ejecutan en MPP y COW, así que no es difícil trasladar las aplicaciones de una plataforma a otra.

Los sistemas de transferencia de mensajes tienen dos o más procesos que se ejecutan de forma independiente. Por ejemplo, un proceso podría estar produciendo ciertos datos y otro u otros procesos podrían estarlos consumiendo. No existe garantía de que cuando el transmisor tenga más datos los receptores estarán listos para recibirlos, ya que cada uno ejecuta su propio programa.

Casi todos los sistemas de transferencia de mensajes cuentan con dos primitivas (por lo regular llamadas de biblioteca), **send** y **receive**, pero puede haber varios tipos de semánticas distintas. Las tres principales variantes son:

1. Transferencia de mensajes sincrónica.
2. Transferencia de mensajes con buffers.
3. Transferencia de mensajes no bloqueadora.

En la **transferencia de mensajes sincrónica**, si el transmisor ejecuta un **send** y el receptor todavía no ha ejecutado un **receive**, el transmisor se bloquea (suspende) hasta que el receptor ejecuta un **receive**, y en ese momento se copia el mensaje. Cuando el transmisor recupera el control después de la llamada, sabe que el mensaje se envió y se recibió correctamente. Este método tiene la semántica más sencilla y no requiere buffers. Una desventaja importante es que el transmisor permanece bloqueado hasta que el receptor ha obtenido el mensaje y ha confirmado tal recepción.

En la **transferencia de mensajes con buffers**, cuando se envía un mensaje antes de que el receptor esté listo, el mensaje se coloca en un buffer en algún lado, por ejemplo en un buzón, hasta que el receptor lo saca. Así, en la transferencia de mensajes con buffers un transmisor puede continuar después de un **send**, aunque el receptor esté ocupado con otra cosa. Puesto que el mensaje realmente se envió, el transmisor está en libertad de reutilizar el buffer de mensajes de inmediato. Este esquema reduce el tiempo que el transmisor tiene que esperar. Básicamente, tan pronto como el sistema ha enviado el mensaje el transmisor puede continuar. Sin embargo, el transmisor no tiene ninguna garantía de que el mensaje se recibió correctamente. Incluso si la comunicación es confiable, el receptor podría haberse caído antes de recibir el mensaje.

En la **transferencia de mensajes no bloqueadora**, se permite al transmisor continuar de inmediato después de efectuar la llamada. Lo único que la biblioteca hace es pedir al sistema

operativo que lleve a cabo la llamada después, cuando tenga tiempo. El resultado es que el transmisor prácticamente no se bloquea. La desventaja de este método es que cuando el transmisor continúa después del **send** no puede reutilizar el buffer de mensajes, porque es posible que el mensaje todavía no se haya enviado. El transmisor necesita una forma de averiguar si ya puede reutilizar el buffer. Una idea es hacer que le pregunte repetidamente al sistema. La otra es recibir una interrupción cuando el buffer esté disponible. Ninguna de estas cosas simplifica el software.

En las dos secciones que siguen examinaremos brevemente dos sistemas de transferencia de mensajes que se usan en muchas multicomputadoras: PVM y MPI. Éstos no son los únicos, pero son los más ampliamente utilizados. PVM es más antiguo y fue el estándar *de facto* durante años, pero ahora MPI es el muchacho nuevo en el barrio y está retando al campeón reinante.

PVM — Máquina Virtual Paralela

PVM (*Parallel Virtual Machine*) es un sistema de transferencia de mensajes del dominio público diseñado inicialmente para ejecutarse en COW basadas en UNIX (Geist *et al.*, 1994; Sunderram, 1990). Posteriormente se le ha trasladado a muchas otras plataformas, incluidos casi todos los MPP comerciales. Se trata de un sistema autosuficiente, que incluye gestión de procesos y apoyo de E/S.

PVM consta de dos partes: una biblioteca que el usuario puede invocar y un proceso demonio que se ejecuta todo el tiempo en cada máquina de la multicomputadora. Cuando se inicia PVM, éste determina cuáles máquinas van a formar parte de su multicomputadora virtual leyendo un archivo de configuración. En cada una de esas máquinas se inicia un demonio PVM. Es posible añadir y eliminar máquinas emitiendo comandos en la consola PVM.

PVM permite iniciar n procesos paralelos con el comando de consola

```
spawn -count n prog
```

Cada proceso ejecutará *prog*. PVM decide dónde colocará los procesos, pero un usuario puede supeditar esa decisión con argumentos del comando **spawn**. También se pueden iniciar procesos desde un proceso en ejecución invocando *Pvm_spawn*. Los procesos pueden organizarse en grupos cuya composición cambia dinámicamente durante la ejecución.

La comunicación en PVM se maneja con primitivas de transferencia de mensajes explícitas y de tal manera que se puedan comunicar máquinas que usen diferentes representaciones de los números. Cada proceso PVM tiene un buffer de envío activo y un buffer de recepción activo en cualquier instante. Para enviar un mensaje, un proceso invoca procedimientos de biblioteca para empacar valores en el buffer de envío activo en un formato que se describe a sí mismo, de modo que el receptor pueda reconocerlos y convertirlos a su formato nativo.

Una vez que se ha armado el mensaje, el transmisor invoca el procedimiento de biblioteca *pvm_send*, que es una transmisión bloqueadora. El receptor tiene varias opciones. La más sencilla es *pvm_recv*, que bloquea al receptor hasta que llega un mensaje apropiado. Cuando la llamada regresa el mensaje está en el buffer de recepción activo, y puede desempacarse y convertirse en valores apropiados para su máquina utilizando una serie de procedimientos de

desempacado. Si un receptor está preocupado por la posibilidad de bloquearse indefinidamente, puede invocar *pvm_trecv*, que bloquea al receptor durante cierto tiempo, pero si en ese tiempo no llega un mensaje apropiado se desbloquea e informa del fracaso. Una opción más es *pvm_nrecv*, que regresa inmediatamente, sea con un mensaje o con una indicación de que no ha llegado ningún mensaje. Esta llamada puede efectuarse una y otra vez para ver si han llegado mensajes.

Además de estas primitivas para mensajes de punto a punto, PVM también maneja difusión (*pvm_bcast*) y multidifusión (*pvm_mcast*). La primera transmite a todos los procesos de un grupo; la segunda transmite a una lista específica de procesos.

La sincronización entre procesos se efectúa usando *pvm_barrier*. Cuando un proceso invoca este procedimiento, se bloquea hasta que cierto número de otros procesos han llegado también a la barrera y han efectuado la misma llamada. Hay otras llamadas PVM para gestión de anfitriones, gestión de grupos, gestión de buffers, señalización, verificación de estado y funciones diversas. En síntesis, PVM es un paquete sencillo, directo, fácil de usar, disponible en casi todas las computadoras paralelas, todo lo cual explica su popularidad.

MPI—Interfaz de Transferencia de Mensajes

Un segundo paquete de comunicaciones para programar multicomputadoras es **MPI** (*Message-Passing Interface*). MPI es mucho más rico y complejo que PVM, con muchas más llamadas de biblioteca, muchas más opciones y muchos más parámetros por llamada. La versión original de MPI, ahora llamada MPI-1, fue aumentada por MPI-2 en 1997. A continuación presentaremos una introducción muy superficial a MPI-1, y luego hablaremos un poco acerca de lo que se añadió en MPI-2. Si desea más información acerca de MPI, vea (Gropp *et al.*, 1994; y Snir *et al.*, 1996).

MPI-1 no se ocupa de la creación o gestión de procesos, como hace PVM. Corresponde al usuario crear procesos utilizando llamadas al sistema locales. Una vez creados, los procesos se acomodan en grupos estáticos, inmutables. Es con estos grupos que MPI trabaja.

MPI se basa en cuatro conceptos ortogonales: comunicadores, tipos de datos de mensajes, operaciones de comunicación y topologías virtuales. Un **comunicador** es un grupo de procesos más un contexto. Un contexto es un rótulo que identifica algo, como una fase de ejecución. Cuando se envían y reciben mensajes, el contexto puede servir para evitar que mensajes no relacionados entre sí se interfieran.

Los mensajes se tipifican, y se reconocen muchos tipos de datos, que incluyen caracteres, enteros, cortos, normales y largos, números de punto flotante de precisión sencilla y doble, etc. También es posible construir otros tipos derivados de éstos.

MPI maneja un amplio conjunto de operaciones de comunicación. La más básica sirve para enviar mensajes como sigue:

```
MPI_Send(buffer, cuenta, tipo_datos, destino, etiqueta, comunicador)
```

Esta llamada envía al destino un buffer con *cuenta* de elementos del tipo de datos especificados. El campo *etiqueta* rotula el mensaje para que el receptor pueda decir que sólo quiere recibir un mensaje con esa etiqueta. El último campo indica en cuál grupo de procesos está el destino

(el campo *destino* sólo es un índice dentro de la lista de procesos para el grupo especificado). La llamada correspondiente para recibir un mensaje es

```
MPI_Recv(&buffer, cuenta, tipo_datos, origen, etiqueta, comunicador, &estado)
```

que anuncia que el receptor está buscando un mensaje de cierto tipo proveniente de cierto origen, con cierta etiqueta.

MPI reconoce cuatro modos de comunicación básicos. El modo 1 es sincrónico, en el que el transmisor no puede comenzar a transmitir hasta que el receptor haya invocado a *MPI_Recv*. El modo 2 usa buffers, por lo que no aplica la restricción anterior. El modo 3 es estándar, que depende de la implementación y puede ser sincrónico o usar buffers. El modo 4 es "listo", en el que el transmisor asegura que el receptor está disponible (como en el sincrónico), pero no se comprueba que sea cierto. Cada una de estas primitivas tiene una versión bloqueadora y una no bloqueadora, así que hay ocho primitivas en total. La recepción sólo tiene dos variantes: bloqueadora y no bloqueadora.

MPI maneja la comunicación colectiva, incluidas la difusión, dispersión/reunión, intercambio total, agregado y barrera. En todas las formas de comunicación colectiva, todos los procesos de un grupo deben emitir la llamada con argumentos compatibles. Si no se hace esto ocurre un error. Una forma típica de comunicación colectiva aplica a procesos organizados en forma de árbol, en el que los valores se propagan hacia arriba, desde las hojas hasta la raíz, sufriendo cierto procesamiento en cada paso; por ejemplo obteniendo la sumatoria o el máximo de los valores.

El cuarto concepto básico en MPI es la **topología virtual**, que permite acomodar los procesos en un árbol, anillo, cuadrícula, toroide u otras topologías. Un acomodo así permite nombrar caminos de comunicación y facilita la comunicación.

MPI-2 añade procesos dinámicos, acceso a memorias remotas, comunicación colectiva no bloqueadora, apoyo para E/S escalable, procesamiento en tiempo real y muchas características nuevas que rebasan el alcance de este libro. En la comunidad científica prevalece actualmente una lucha entre los partidarios de MPI y de PVM. El lado de PVM dice que PVM es más fácil de aprender y más sencillo de usar. El lado de MPI dice que MPI hace más y también señala que es un estándar formal con un comité de estandarización y un documento de definición oficial. El lado de PVM dice que es cierto, y asegura que la falta de una burocracia de estandarización con todas las de la ley no necesariamente es una desventaja.

8.4.5 Memoria compartida en el nivel de aplicaciones

Por nuestros ejemplos, debe ser obvio que es más fácil aumentar la escala de una multicomputadora que de un multiprocesador. El Sun Enterprise 10000, con un máximo de 64 CPU, y el NUMA-Q, con un máximo de 256 CPU, son representativos de los multiprocesadores UMA y NUMA grandes, respectivamente. En contraste, las máquinas T3E y Option Red pueden tener 2048 y 9416 CPU, respectivamente. Podemos debatir eternamente la importancia de estas cifras, pero no podemos escapar la conclusión de que las multicomputadoras pueden ser mucho más grandes que los multiprocesadores. Esto siempre ha sido cierto y seguirá siendo cierto durante muchos años más.

Sin embargo, las multicomputadoras no tienen memoria compartida en el nivel de arquitectura, lo que dio pie al desarrollo de sistemas de transferencia de mensajes como PVM y MPI. A muchos programadores no les gusta este modelo y quisieran tener la ilusión de memoria compartida, aunque no esté realmente ahí. Si se puede lograr este objetivo se tendría lo mejor de dos mundos: hardware grande de bajo costo (al menos por nodo) y facilidad de programación. Éste es el Santo Grial de la computación en paralelo.

Muchos investigadores han llegado a la conclusión de que si bien no es fácil aumentar la escala de la memoria compartida en el nivel de arquitectura, puede haber otras formas de alcanzar la misma meta. En la figura 8-3 vemos que hay otros niveles en el que puede introducirse una memoria compartida. En las secciones que siguen veremos algunas formas de introducir memoria compartida en el modelo de programación de una multicomputadora sin que esté presente en el nivel de hardware.

Memoria compartida distribuida

Una clase de sistema de memoria compartida en el nivel de aplicación es el sistema basado en páginas que se conoce como **memoria compartida distribuida** (DSM, *Distributed Shared Memory*). La idea es sencilla: una colección de CPU en una multicomputadora comparte un espacio de direcciones paginado común. En la versión más simple, cada página se tiene en la RAM de una y sólo una CPU. En la figura 8-40(a) vemos un espacio de direcciones virtual compartido que consiste en 16 páginas, distribuidas entre cuatro CPU.

Cuando una CPU hace referencia a una página en su propia RAM local, la lectura o escritura se efectúa sin retraso. En cambio, cuando una CPU hace referencia a una página en una memoria remota obtiene un fallo de página. La cuestión es que en lugar de traer la página faltante del disco, el sistema de tiempo de ejecución o sistema operativo envía un mensaje al nodo que tiene la página ordenándole que anule la correspondencia de esa página con su memoria y la envíe. Una vez que la página llega, se establece una correspondencia con la memoria local y la instrucción que causó la falla se reinicia, igual que con un fallo de página normal. En la figura 8-40(b) vemos la situación después de que la CPU 0 ha causado un fallo por la página 10: esa página se traslada de la CPU 1 a la CPU 0.

Esta idea básica se implementó por primera vez en IVY (Li y Hudak, 1986, 1989). Se obtiene una memoria totalmente compartida, secuencialmente consistente, en una multicomputadora. Sin embargo, pueden efectuarse muchas optimaciones para mejorar el desempeño. La primera optimación, presente en IVY, es permitir que páginas que están marcadas como sólo de lectura estén presentes en varios nodos al mismo tiempo. Así, cuando ocurre un fallo de página, se envía una copia de la página a la máquina que causó el fallo, pero el original se queda donde estaba porque no hay peligro de conflictos. En la figura 8-40(c) se ilustra la situación de dos CPU que comparten una página sólo de lectura (página 10).

Aun con esta optimación, el desempeño muchas veces es inaceptable, sobre todo cuando un proceso está escribiendo activamente unas cuantas palabras en la parte superior de la página y otro proceso en una CPU diferente está activamente escribiendo más palabras en la parte inferior de la página. Puesto que sólo existe una copia de la página, ésta tiene que rebotarse de un lado al otro constantemente, situación que se conoce como **compartimiento falso**.

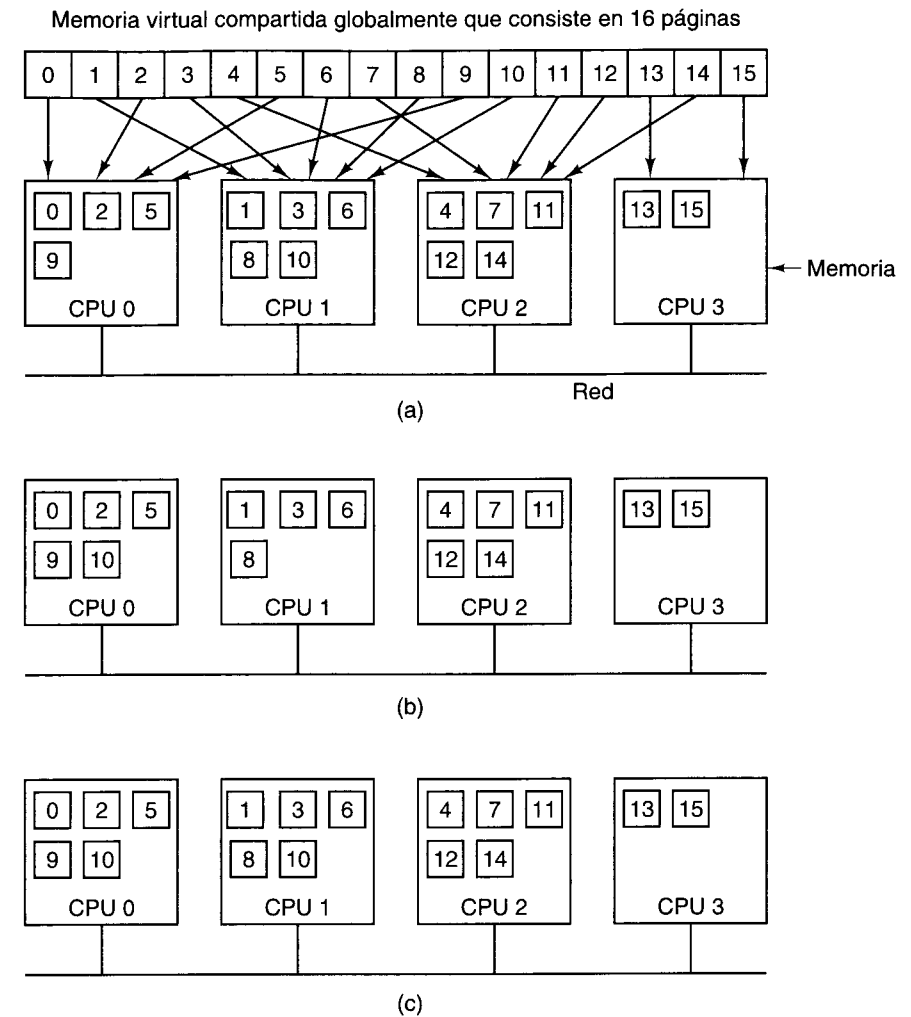


Figura 8-40. Espacio de direcciones virtual que consiste en 16 páginas distribuidas en cuatro nodos de una multicomputadora. (a) La situación inicial. (b) Después de que la CPU 0 hace referencia a la página 10. (c) Después de que la CPU 1 hace referencia a la página 10, que aquí se supone es sólo de lectura.

El problema del compartimiento falso se puede atacar de varias maneras. En el sistema Treadmarks, por ejemplo, se abandona una memoria secuencialmente consistente en favor de la consistencia de liberación (Amza, 1996). Páginas que podrían ser escribibles pueden estar presentes en varios nodos al mismo tiempo, pero antes de efectuar una escritura un proceso debe efectuar una operación *acquire* para indicar su intención. En ese punto, todas las copias con excepción de la más reciente se invalidan. No puede crearse ninguna otra copia hasta que se ejecute el *release* correspondiente, y entonces la página puede volverse a compartir.

Una segunda optimación que se efectúa en Treadmarks es establecer inicialmente la correspondencia de cada página escribible en modo de sólo lectura. La primera vez que se escribe en la página, ocurre una falla de protección y el sistema crea una copia de la página, llamada **gemela**. Entonces se establece la correspondencia de la página como de lectura-escritura y las escrituras subsecuentes pueden efectuarse a toda velocidad. Si después ocurre un fallo de página remota y la página tiene que enviarse allá, se efectúa una comparación palabra por palabra entre la página actual y la gemela. Sólo se envían las palabras que se modificaron, lo que reduce el tamaño de los mensajes.

Cuando ocurre un fallo de página, es preciso localizar la página faltante. Puede haber varias soluciones, incluidas las que se usan en máquinas NUMA y COMA, como los directorios (en la base). De hecho, muchas de las soluciones que se usan en DSM también son aplicables a NUMA y COMA porque DSM en realidad sólo es una implementación en software de NUMA o COMA en la que cada página se trata como una línea de caché.

DSM es un área de investigación muy activa. Entre los sistemas interesantes están CASHMERE (Kontothanassis *et al.*, 1997; Stets *et al.*, 1997), CRL (Johnson *et al.*, 1995), Shasta (Scales *et al.*, 1996) y Treadmarks (Amza, 1996; Lu *et al.*, 1997).

Linda

Los sistemas DSM basados en páginas como IVY y Treadmarks usan el hardware de MMU para atrapar los accesos a páginas faltantes. Si bien es útil hacer y enviar diferencias en lugar de páginas enteras, persiste el hecho de que las páginas son una unidad no natural para compartir. Por ello, se han intentado otros enfoques.

Uno de ellos es Linda, que proporciona a procesos de múltiples páginas una memoria compartida distribuida altamente estructurada (Carriero y Gelernter, 1986, 1989). El acceso a esta memoria es a través de un conjunto pequeño de operaciones primitivas que se pueden añadir a lenguajes existentes, como C y FORTRAN, para formar lenguajes paralelos, en este caso C-Linda y FORTRAN-Linda.

El concepto unificador en que se basa Linda es el de un **espacio de tuplas** abstracto, que es global para todo el sistema y accesible para todos sus procesos. El espacio de tuplas es como una memoria compartida global, sólo que con cierta estructura integrada. El espacio de tuplas contiene cierto número de **tuplas**, cada una de las cuales consiste en uno o más campos. En el caso de C-Linda, los tipos de campos incluyen enteros, enteros largos y números de punto flotante, además de tipos compuestos como arreglos (que incluyen cadenas) y estructuras (pero no otras tuplas). En la figura 8-41 se muestran tres tuplas como ejemplos.

Se proporcionan cuatro operaciones con tuplas. La primera, **out**, coloca una tupla en el espacio de tuplas. Por ejemplo,

```
out("abc", 2, 5);
```

coloca la tupla ("abc", 2, 5) en el espacio de tuplas. Los campos de **out** normalmente son constantes, variables o expresiones, como en

```
out("matrix-1", i, j, 3.14);
```

```
("abc", 2, 5)
("matrix-1", 1, 6, 3.14)
("familia", "es hermana", Carolina, Leonor)
```

Figura 8-41. Tres tuplas Linda.

que produce una tupla con cuatro campos, de los cuales el segundo y el tercero están determinados por los valores actuales de las variables *i* y *j*.

Las tuplas se recuperan del espacio de tuplas con la primitiva **in**. El direccionamiento es por contenido, más que por nombre o dirección. Los campos de **in** pueden ser expresiones o parámetros formales. Considere, por ejemplo,

```
in("abc", 2, ? i);
```

Esta operación examina el espacio de tuplas en busca de una tupla que consista en la cadena "abc", el entero 2 y un tercer campo que contenga cualquier entero (suponiendo que *i* es un entero). Si se encuentra, la tupla se saca del espacio de tuplas y se asigna a la variable *i* el valor del tercer campo. La comparación y la eliminación son atómicas, así que si dos procesos ejecutan la misma operación **in** simultáneamente, sólo uno de ellos tendrá éxito, a menos que estén presentes dos o más tuplas que coincidan. El espacio de tuplas podría incluso contener varias copias de la misma tupla.

El algoritmo de comparación que **in** usa es sencillo. Los campos de la primitiva **in**, llamados **plantilla**, se comparan (conceptualmente) con los campos correspondientes de cada tupla del espacio de tuplas. Hay coincidencia si se cumplen las tres condiciones siguientes:

1. La plantilla y la tupla tienen el mismo número de campos.
2. Los tipos de los campos correspondientes son iguales.
3. Cada constante o variable de la plantilla coincide con su campo de tupla.

Los parámetros formales, que se indican con un signo de interrogación seguido de un nombre o tipo de variable, no participan en la comparación (excepto para verificar el tipo), aunque a los que contienen un nombre de variable se les asigna un valor si la comparación tiene éxito.

Si ninguna de las tuplas presentes coincide, el proceso invocador se suspende hasta que otro proceso inserta la tupla requerida; en ese momento el invocador se revive automáticamente y recibe la nueva tupla. El hecho de que los procesos se bloquean y desbloquean automáticamente implica que si un proceso está a punto de producir una tupla y otro está a punto de capturarla, no importa cuál vaya primero.

Además de **out** e **in**, Linda tiene una primitiva **read**, que es igual a **in** sólo que no elimina la tupla del espacio de tuplas. También hay una primitiva **eval** que hace que sus parámetros se evalúen en paralelo y la tupla resultante se deposite en el espacio de tuplas. Este mecanismo

puede servir para realizar un cálculo arbitrario. Es así como se crean procesos paralelos en Linda.

Un paradigma de programación común en Linda es el **modelo de trabajador repetido**. Este modelo se basa en la idea de un **almacén de tareas** (*task bag*) llena de trabajos que hay que realizar. El proceso principal inicia ejecutando un ciclo que contiene

```
out("task-bag", job);
```

en el que se introduce en el espacio de tuplas una descripción de trabajo distinta en cada iteración. Cada trabajador inicia obteniendo una tupla de descripción de trabajo mediante

```
in("task-bag", ?job);
```

y luego lleva a cabo el trabajo; cuando termina, obtiene otro trabajo. También es posible colocar trabajos nuevos en el almacén de tareas durante la ejecución. De esta forma tan sencilla, el trabajo se reparte dinámicamente entre los trabajadores y todos los trabajadores se mantienen ocupados continuamente, con muy poco gasto extra.

Existen diversas implementaciones de Linda en sistemas de multicomputadora. En todos ellos, un aspecto clave es cómo se distribuyen las tuplas entre las máquinas y cómo se les puede localizar si es necesario. Diversas posibilidades incluyen difusión y directorios. El copiado es otro aspecto importante. Estos puntos se tratan en (Bjornson, 1993).

Orca

Una estrategia un tanto distinta para tener memoria compartida en el nivel de aplicaciones en una multicomputadora es usar como unidades de compartimiento objetos con todas las de la ley, en lugar de sólo tuplas. Los objetos consisten en un estado interno (oculto) más métodos para operar sobre ese estado. Como no se permite al programador acceder al estado directamente, se abren muchas posibilidades para poder compartir entre máquinas que no tienen una memoria física compartida.

Un sistema basado en objetos que da la ilusión de una memoria compartida en sistemas de multicomputadora es Orca (Bal, 1991; Bal *et al.*, 1992; Bal y Tanenbaum, 1988). Orca es un lenguaje de programación tradicional (basado en Modula 2) al cual se han añadido dos características nuevas: objetos y la capacidad para crear procesos nuevos. Un objeto Orca es un tipo de datos abstracto, análogo a un objeto en Java o un paquete en Ada. El objeto encapsula estructuras de datos internas y métodos escritos por el usuario, llamados **operaciones**. Los objetos son pasivos, es decir, no contienen enlaces a los que puedan enviarse mensajes. En vez de ello, los procesos acceden a los datos internos de un objeto invocando sus métodos.

Cada método Orca consiste en una lista de pares (guardia, bloque de enunciados). Un guardia (guard) es una expresión booleana que no contiene efectos secundarios, o el guardia vacío, que equivale al valor *true* (verdadero). Cuando se invoca una operación, todos sus guardias se evalúan en un orden no especificado. Si todos ellos son *false*, el proceso invocador espera hasta que uno de ellos se vuelve *true*. Cuando la evaluación de un guardia da *true*,

se ejecuta el bloque de enunciados que le siguen. En la figura 8-42 se muestra un objeto *pila* con dos operaciones, *meter* y *sacar*.

```
Object implementation pila;
  tope: integer;
  pila: array [integer 0..N-1] of integer;

  operation meter(elem: integer);
  begin
    pila[tope] := elem;
    tope := tope + 1;
  end;

  operation sacar(): integer;
  begin
    guard tope > 0 do
      tope := tope - 1;
      return pila[tope];
    od;
  end;

begin
  tope := 0;
end;
```

memoria para la pila
función que no devuelve nada
meter elem(ento) en la pila
incrementar apuntador de pila
función que devuelve un entero
suspender si la pila está vacía
decrementar apuntador de pila
devolver elemento del tope
inicialización

Figura 8-42. Objeto de pila ORCA simplificado, con datos internos y dos operaciones.

Una vez definida una *pila*, es posible declarar variables de este tipo, como en

```
s, t: pila;
```

que crea dos objetos *pila* e inicializa la variable *tope* de cada uno con 0. Podemos meter la variable entera *k* en la pila *s* con el enunciado

```
s$push(k);
```

etcétera. La operación *sacar* tiene un guardia, por lo que un intento por sacar una variable de una pila vacía suspenderá al invocador hasta que otro proceso haya metido algo en la pila.

Orca cuenta con un enunciado **fork** para crear un proceso nuevo en un procesador especificado por el usuario. El nuevo proceso ejecuta el procedimiento nombrado en el enunciado **fork**. Es posible pasar parámetros, que podrían ser objetos, al nuevo proceso, y es así como se distribuyen los objetos entre las máquinas. Por ejemplo, el enunciado

```
for i in 1 .. n do fork foobar(s) on i; od;
```

genera un proceso nuevo en cada una de las máquinas de la 1 hasta la *n*, ejecutando el programa *foobar* en cada una de ellas. Puesto que estos *n* procesos nuevos (y el padre) se ejecutan en paralelo, todos pueden meter elementos en la pila compartida *s*, y sacarlos de ella, como si todos se estuvieran ejecutando en un multiprocesador con memoria compartida. Correspon-

de al sistema de tiempo de ejecución mantener la ilusión de una memoria compartida que en realidad no existe.

Las operaciones con objetos compartidos son atómicas y secuencialmente consistentes. El sistema garantiza que si varios procesos realizan operaciones con el mismo objeto compartido casi al mismo tiempo, el sistema escogerá algún orden y todos los procesos verán el mismo orden.

Orca integra datos compartidos y sincronización en una forma que no está presente en los sistemas DSM basados en páginas. En los programas paralelos se requieren dos tipos de sincronización. El primero es la sincronización por exclusión mutua, para evitar que dos procesos ejecuten la misma región crítica al mismo tiempo. En Orca, cada operación con un objeto compartido es de hecho como una región crítica porque el sistema garantiza que el resultado final será el mismo que se obtendría si todas las regiones críticas se ejecutaran una por una (es decir, secuencialmente). En este sentido, un objeto Orca es como una forma distribuida de monitor (Hoare, 1975).

El otro tipo de sincronización es la sincronización por condición, en la que un proceso se bloquea hasta que se cumple una condición. En Orca, la sincronización por condición se efectúa con guardias. En el ejemplo de la figura 8-42, un proceso que trata de sacar un elemento de una pila vacía se suspende hasta que la pila deja de estar vacía.

El sistema de tiempo de ejecución Orca se encarga del copiado de objetos, migración, consistencia e invocación de operaciones. Cada objeto puede estar en uno de dos estados: copia única o repetido. Un objeto en estado de copia única existe en una sola máquina, por lo que todas las solicitudes que lo requieren se envían ahí. Un objeto repetido está presente en todas las máquinas que contienen un proceso que lo usa, lo cual facilita las operaciones de lectura (porque se pueden efectuar localmente) a expensas de hacer las actualizaciones más costosas. Cuando se ejecuta una operación que modifica un objeto repetido, primero debe obtener un número de secuencia de un proceso centralizado que los emite. Luego se envía un mensaje a cada máquina que tiene una copia del objeto, pidiéndole que ejecute la operación. Puesto que todas estas actualizaciones llevan números de secuencia, todas las máquinas realizan las operaciones en orden secuencial, lo que garantiza la consistencia secuencial.

Globe

Casi todos los sistemas DSM, Linda y Orca se ejecutan en sistemas locales, es decir, dentro de un solo edificio o campus universitario. Sin embargo, también es posible construir un sistema de memoria compartida en el nivel de aplicaciones en una multicomputadora distribuida en todo el mundo. En el sistema Globe, un objeto puede estar en el espacio de direcciones de varios procesos al mismo tiempo, posiblemente en continentes distintos (Kermarrec *et al.*, 1998; van Steen *et al.*, 1999). Para acceder a los datos de un objeto compartido, los procesos de usuario deben usar sus métodos, lo que permite a diferentes objetos tener diferentes estrategias de implementación. Por ejemplo, una opción es tener una sola copia de los datos que se solicita dinámicamente cuando se necesita (apropiado para datos que un solo dueño actualiza con frecuencia). Otra opción es tener todos los datos situados dentro de cada copia del objeto y enviar actualizaciones a cada copia con un protocolo de multidifusión confiable.

Lo que hace a Globe un tanto ambicioso es su meta de abarcar mil millones de usuarios y un billón de objetos (posiblemente móviles). Localizar objetos, administrarlos y controlar el cambio de escala son aspectos cruciales. Globe logra esto con la ayuda de un marco general en el que cada objeto puede tener su propia estrategia de copiado, estrategia de seguridad, etc. Esto evita el problema “unitalla” que se presenta en otros sistemas, sin perder la facilidad de programación que ofrece la memoria compartida.

Otros sistemas distribuidos de área extensa son Globus (Foster y Kesselman, 1998a; Foster y Kesselman, 1998b) y Legion (Grimshaw y Wulf, 1996; Grimshaw y Wulf, 1997), pero éstos no ofrecen la ilusión de memoria compartida como Globe.

8.5 RESUMEN

Las computadoras paralelas pueden dividirse en dos categorías principales: SIMD y MIMD. Las máquinas SIMD ejecutan una instrucción a la vez en paralelo con muchos conjuntos de datos. Estas máquinas incluyen los procesadores de arreglos y las computadoras vectoriales. Las máquinas MIMD, que dominan la computación en paralelo, ejecutan diferentes programas en diferentes máquinas. Las máquinas MIMD se pueden dividir en multiprocesadores, que comparten la memoria primaria, y multicomputadoras, que no lo hacen. Ambos tipos de sistemas consisten en CPU y módulos de memoria conectados por diversas redes de alta velocidad que transfieren paquetes de solicitud y respuesta entre las CPU y la memoria y entre las CPU. Se usan muchas topologías, que incluyen cuadrículas, toroides, anillos e hipercubos.

Para todos los multiprocesadores, uno de los aspectos clave es el modelo de consistencia de la memoria que se maneja. Entre los modelos más comunes están la consistencia secuencial, la consistencia de procesador, la consistencia débil y la consistencia de liberación. Se pueden construir multiprocesadores en los que se espía el bus (usando el protocolo MESI, por ejemplo). También son posibles diseños de barras cruzadas y redes de conmutación. Otras posibilidades son las máquinas NUMA y COMA basadas en directorios.

Las multicomputadoras se pueden dividir a grandes rasgos en MPP y COW, aunque la frontera es un tanto arbitraria. Los MPP son enormes sistemas comerciales, como el Cray T3E y el Intel/Sandia Option Red, que usan interconexiones de alta velocidad patentadas. En contraste, los COW se construyen con piezas comerciales, como Ethernet, ATM o Myrinet.

Las multicomputadoras a menudo se programan empleando un paquete de transferencia de mensajes como PVM o MPI. Ambos cuentan con llamadas de biblioteca para enviar y recibir mensajes, con muchas opciones, y se ejecutan encima de sistemas operativos existentes.

Una estrategia alternativa es usar memoria compartida en el nivel de aplicaciones, como un sistema DSM basado en páginas, el espacio de tuplas Linda o los objetos Orca o Globe. DSM simula una memoria compartida en el nivel de páginas, lo que lo hace similar a una máquina NUMA, excepto que las referencias remotas son mucho más lentas. Linda, Orca y Globe proporcionan al usuario la ilusión de un modelo de memoria compartida (restringido) empleando tuplas, objetos locales y objetos globales, respectivamente.

PROBLEMAS

1. Una mañana, la abeja reina de cierta colmena llama a todas sus abejas obreras y les dice que la tarea del día es recolectar néctar de caléndula. Entonces, las obreras vuelan en diferentes direcciones en busca de caléndulas. ¿Se trata de un sistema SIMD o MIMD?
2. Para cada una de las topologías de la figura 8-4, calcule el diámetro de la red.
3. Para cada una de las topologías de la figura 8-4, determine el grado de tolerancia ante fallos, que se define como el número máximo de enlaces que pueden perderse sin partir la red en dos.
4. Considere la topología de toroide doble de la figura 8-4(f) pero expandida a un tamaño de $k \times k$. ¿Qué diámetro tiene la red? Sugerencia: considere k par y k impar por separado.
5. Una red de interconexión tiene la forma de un cubo de $8 \times 8 \times 8$. Cada enlace tiene un ancho de banda dúplex de 1 GB/s. ¿Qué ancho de banda bisectriz tiene esta red?
6. En una red de interconexión con forma de cuadrícula rectangular de cuatro conmutadores de anchura por tres conmutadores de altura, los paquetes que parten de la esquina superior izquierda y se dirigen hacia la esquina inferior derecha pueden seguir cualquiera de varias rutas distintas. Numere la fila superior de conmutadores del 1 al 4, la siguiente del 5 al 8, y la de abajo del 9 al 12. Enumere todas las rutas en las que los paquetes sólo se muevan hacia la derecha o hacia abajo, partiendo del 1 y terminando en el 12.
7. La ley de Amdahl limita la aceleración que puede lograrse en una computadora paralela. Calcule, en función de f , la aceleración máxima posible a medida que el número de CPU se acerca al infinito. ¿Qué implicaciones tiene este límite para $f = 0.1$?
8. La figura 8-10 muestra cómo falla el aumento de escala con un bus pero tiene éxito con una cuadrícula. Suponiendo que cada bus o enlace tiene un ancho de banda b , calcule el ancho de banda medio por CPU para cada uno de los cuatro casos. Luego aumente la escala de cada sistema a 64 CPU y repita los cálculos. ¿Cuál es el límite a medida que el número de CPU se acerca a infinito?
9. La compañía de computadoras 2D tiene un producto que consiste en n computadoras con memoria disjunta dispuestas en una cuadrícula cuadrada. Cierta noche, al vicepresidente de topologías se le ocurre una idea para un producto nuevo: una cuadrícula tridimensional con los n procesadores dispuestos en un cubo perfecto (por ejemplo, esto es posible con $n = 4096$).
 - a. ¿Qué efecto tiene este cambio sobre el retraso en el peor de los casos?
 - b. ¿Qué efecto tiene este cambio sobre el ancho de banda total?
10. Cuando hablamos de los modelos de consistencia de la memoria, dijimos que un modelo de consistencia es una especie de contrato entre el software y la memoria. ¿Por qué se necesita semejante contrato?
11. Un procesador de vectores como el Cray-1 tiene unidades aritméticas con filas de procesamiento de cuatro etapas. Cada etapa tarda 1 ns. ¿Qué tiempo toma sumar dos vectores de 1024 elementos cada uno?
12. Considere un multiprocesador que usa un bus compartido. ¿Qué sucede si dos procesadores tratan de acceder a la memoria global en el mismo instante exactamente?
13. Suponga que por razones técnicas sólo es posible que una caché espía vigile líneas de direcciones, no líneas de datos. ¿Esto afectaría el protocolo de *escritura a través*?

14. Como modelo sencillo de sistema multiprocesador basado en bus sin cachés, suponga que una instrucción de cada cuatro hace una referencia a la memoria, y que una referencia a la memoria ocupa el bus durante todo un tiempo de instrucción. Si el bus está ocupado, la CPU solicitante se coloca en una cola FIFO. ¿Qué tanto más veloz será un sistema de 64 CPU que uno de una sola CPU?
15. El protocolo de coherencia de cachés MESI tiene cuatro estados. Otros protocolos de coherencia de cachés de escritura de vuelta sólo tienen tres estados. ¿Cuál de los cuatro estados MESI podría sacrificarse, y cuáles serían las consecuencias de cada opción? Si tuviera que escoger sólo tres estados, ¿cuáles escogería?
16. ¿Existen situaciones con el protocolo de coherencia de cachés MESI en las que una línea de caché está presente en la caché local pero para la cual de todos modos se necesita una transacción de bus? Explique.
17. Suponga que hay n CPU conectadas a un bus común. La probabilidad de que cualquier CPU trate de usar el bus en un ciclo dado es p . ¿Qué posibilidades hay de que
 - a. El bus esté ocioso (0 solicitudes)?
 - b. Se haya hecho exactamente una solicitud?
 - c. Se haya hecho más de una solicitud?
18. Las CPU del Sun Enterprise 10000 operan a 333 MHz, pero los buses de espionaje operan a sólo 83.3 MHz. Con 64 CPU, es evidente que los buses nunca podrán manejar la carga. Sin embargo, la máquina funciona. Explique por qué.
19. En el texto calculamos que la capacidad del conmutador de barras cruzadas era suficiente para manejar 167 millones de espionajes por segundo cuando hay 16 tarjetas conectadas a él, aun tomando en cuenta el hecho de que la contención reduce el ancho de banda práctico al 60% del valor teórico. Un Enterprise 10000 pequeño podría tener sólo cuatro tarjetas (16 CPU). ¿Este sistema podría operar a toda velocidad?
20. Suponga que el alambre entre el conmutador 2A y el 3B de la red omega se rompe. ¿Quién queda aislado de quién?
21. Los puntos álgidos (localidades de memoria a las que se hace referencia muy a menudo) son obviamente un problema importante en las redes de conmutación multietapas. ¿También son un problema en los sistemas basados en bus?
22. Una red de conmutación omega conecta 4096 CPU RISC que tienen un tiempo de ciclo de 60 ns, con 4096 módulos de memoria infinitamente rápidos. Cada elemento de conmutación tiene un retraso de 5 ns. ¿Cuántas ranuras de retraso necesita una instrucción LOAD?
23. Considere una máquina que usa una red de conmutación omega como la que se muestra en la figura 8-28. Suponga que el programa y la pila para el procesador i se guardan en el módulo de memoria i . Proponga un cambio pequeño a la topología que mejore mucho el desempeño (la IBM RP3 y la BBN Butterfly usan esa topología modificada). ¿Qué desventajas tiene la nueva topología en comparación con el original?
24. En un multiprocesador NUMA, las referencias a la memoria local tardan 20 ns y las referencias remotas tardan 120 ns. Cierta programa efectúa un total de N referencias a la memoria durante su ejecución, de las cuales 1% son a una página P . Esa página inicialmente es remota, y se requieren C ns para copiarla localmente. ¿En qué condiciones deberá copiarse localmente la página si otros procesadores no hacen mucho uso de ella?

25. Un sistema DASH tiene b bytes de memoria divididos entre n cúmulos. Cada cúmulo tiene p procesadores. El tamaño de línea de caché es de c bytes. Dé una fórmula para la cantidad total de memoria dedicada a directorios (excluidos los dos bits de estado por entrada de directorio).
26. Considere un multiprocesador CC-NUMA como el de la figura 8-30 excepto que con 512 nodos de 8 MB cada uno. Si las líneas de caché son de 64 bytes, determine el porcentaje de gasto extra que representan los directorios. ¿Aumentar el número de nodos aumenta el gasto extra, reduce el gasto extra o no tiene ningún efecto?
27. ¿Cuál es la operación de peor caso (la que más tiempo consume) en SCI?
28. Un multiprocesador basado en SCI tiene 63 nodos, una línea de caché de 32 bytes y un espacio de direcciones total de 2^{32} bytes. La caché en cada nodo es de 1 MB. ¿Cuántos bytes se necesitan en cada directorio de ésta?
29. Proponga un cambio a la implementación NUMA-Q 2000 que permita tener 64 nodos en lugar de 63. ¿Por qué cree que Sequent haya escogido 63 en lugar de 64 como máximo?
30. En el texto se habló de tres variaciones de **send**: sincrónica, bloqueadora y no bloqueadora. Proponga un cuarto método que sea similar a la versión bloqueadora, pero que tenga propiedades un poco distintas. Cite una ventaja y una desventaja de su método en comparación con un **send** bloqueador.
31. Considere una multicomputadora que opera en una red con difusión por hardware, como Ethernet. ¿Por qué importa la proporción de operaciones de lectura (que no actualizan variables de estado internas) por cada operación de escritura (que sí actualiza variables de estado internas)?
32. Muchas de las cuestiones que se presentan en el diseño de multiprocesadores también se presentan en la memoria compartida en el nivel de aplicaciones. Entre ellas están la política de invalidar o actualizar. ¿Cuál política usa Orca?

9

LISTA DE LECTURAS Y BIBLIOGRAFÍA

En los ocho capítulos anteriores se trató un gran número de temas con distintos grados de detalle. El propósito de este capítulo es ayudar a los lectores interesados a profundizar en el estudio de la organización de las computadoras. La sección 9.1 contiene una lista de lecturas sugeridas organizadas por capítulo. La sección 9.2 es una bibliografía alfabética de todos los libros y artículos citados en este libro.

9.1 SUGERENCIAS PARA LECTURAS ADICIONALES

9.1.1 Introducción y obras generales

Hamacher *et al.*, *Computer Organization*, 4a. ed.

Libro de texto tradicional sobre organización de computadoras, CPU, memoria, E/S, aritmética y periféricos. Los ejemplos principales son el 68000 y la PowerPC.

Hayes, *Computer Architecture and Organization*, 3a. ed.

Otro texto tradicional sobre organización de computadoras, como Hamacher *et al.*, con orientación hacia el hardware. Los temas que se cubren incluyen la CPU, el camino de datos, microprogramación, filas de procesamiento, organización de la memoria y E/S.

Patterson y Hennessy, *Computer Organization and Design*

Con casi 1000 páginas, este libro monumental tiene mucho material sobre arquitectura de computadoras, sobre todo el diseño de CPU RISC. Se hace hincapié en cómo obtener un alto desempeño usando filas de procesamiento y otras técnicas.

Price, "A History of Calculating Machines"

Aunque las computadoras modernas iniciaron con Babbage en el siglo XIX, los seres humanos han estado calculando desde los albores de la civilización. Este fascinante artículo ilustrado sigue la historia del conteo, las matemáticas, los calendarios y el cómputo desde 3000 años A.C. hasta principios del siglo XX.

Slater, *Portraits in Silicon*

¿Por qué Dennis Ritchie no entregó su tesis de doctorado en Harvard? ¿Por qué Steve Jobs se convirtió en vegetariano? Las respuestas están en este fascinante libro que contiene biografías cortas de 34 personas que moldearon la industria de las computadoras, desde Charles Babbage hasta Donald Knuth.

Stallings, *Computer Organization and Architecture, 4a. ed.*

Texto general sobre arquitectura de computadoras. Algunos de los temas que se tratan en este libro también se cubren en el de Stallings.

Wilkes, "Computers Then and Now"

Historia personal de las computadoras desde 1946 hasta 1968 por un diseñador de computadoras pionero e inventor de la microprogramación, Maurice Wilkes. Él habla de las primeras batallas entre los "cadetes espaciales" que creían en la programación automática (antes de los compiladores FORTRAN) y los tradicionalistas, que preferían programar en octal.

9.1.2 Organización de sistemas de cómputo

Ng, "Advances in Disk Technology: Performance Issues"

La gente ha estado pronosticando el fin del disco magnético desde hace por lo menos 20 años. Hasta ahora, los discos siguen vivos. Y según este artículo, su tecnología está avanzando rápidamente, por lo que es probable que los tengamos entre nosotros durante varios años más.

Messmer, *The Indispensable PC Hardware Book, 3a. ed.*

Con 1384 páginas (divididas en 36 capítulos y 13 apéndices), este libro podría ser indispensable o no, pero ciertamente es grueso. Aquí encontrará prácticamente todas las intimidades de los procesadores 80x86, memorias, buses, chips de apoyo y periféricos. Si ha leído y digerido el libro de Norton y Goodman (vea en seguida) y quiere pasar al siguiente nivel de detalle técnico, comience aquí.

Norton y Goodman, *Inside the PC, 7a. ed.*

Casi todos los libros sobre hardware de PC están escritos para gente que estudió ingeniería eléctrica y son difíciles de leer por quienes están orientados hacia el software. Éste es distinto: explica el hardware de PC de una forma técnica pero muy accesible. Los temas incluyen la CPU, memoria, buses, discos, pantallas, dispositivos de E/S, PC portátiles, trabajo con redes y más. Un singular y valioso libro.

Pilgrim, *Build Your Own Pentium II PC*

Si es miembro fundador del club del destornillador y el cautín, y quiere construir su propia PC con trocitos de metal, este libro le dice cómo hacerlo. Sin embargo, aunque se conforme con comprar su PC en una tienda, este libro contiene abundante información acerca de los componentes de una PC, cómo funcionan y qué opciones hay.

9.1.3 El nivel de lógica digital

Floyd, *Digital Fundamentals, 6a. ed.*

Para lectores orientados hacia el hardware que quieren profundizar en su estudio del nivel de lógica digital, este enorme libro a cuatro colores, generosamente ilustrado, es una verdadera joya. Los capítulos cubren lógica combinatoria, dispositivos lógicos programables, flip-flops, registros de desplazamiento, memorias, interfaces y mucho más.

Katayama, "Trends in Semiconductor Memory"

Si bien las memorias no son tan rápidas como las CPU, la tecnología de memoria no está estancada. Están ocurriendo todo tipo de avances en tecnología DRAM. Este artículo reseña varios de ellos.

Mano y Kime, *Logic and Computer Design Fundamentals*

Aunque este libro no tiene el diseño elegante del de Floyd, también es una buena referencia para el nivel de lógica digital. La cobertura incluye circuitos combinatorios y secuenciales, registros, memorias, diseño de CPU y E/S.

Mazidi y Mazidi, *The 80x86 IBM PC and Compatible Computers, 2a. ed.*

Para lectores interesados en entender todos los chips que hay dentro de una PC, este libro tiene capítulos enteros sobre los principales chips, así como abundante información acerca del hardware de la IBM PC y la programación en lenguaje ensamblador.

McKee *et al.*, "Smarter Memory: Improving Bandwidth for Streamed References"

En comparación con las CPU, las memorias se han estado volviendo más lentas desde hace décadas. En este artículo se examinan varias cuestiones relacionadas con el desempeño de las memorias, y se analiza una posible solución.

Nelson *et al.*, *Digital Logic and Circuit Analysis and Design*

Otro libro muy completo sobre lógica digital; cubre circuitos tanto combinatorios como secuenciales con gran detalle.

Triebel, *The 80386, 80486, and Pentium Processor*

Es un poco difícil clasificar este libro porque se ocupa de hardware, software e interfaces. Puesto que el autor trabaja para Intel, llamémoslo un libro de hardware. Aquí se dice todo sobre los procesadores, memorias, dispositivos de E/S e interfaces de los chips 80x86, pero también se explica cómo programarlos en lenguaje ensamblador. Aunque sólo tiene 915 páginas, contiene casi tanto material como el libro de Messmer porque las páginas son más grandes.

9.1.4 El nivel de microarquitectura

Handy, *The Cache Memory Book*, 2a. ed.

El diseño de cachés es lo bastante importante como para que ya haya libros enteros sobre el tema. En éste se tratan las cachés lógicas *versus* las físicas, el tamaño de línea, las políticas de escritura a través y las de escritura de vuelta, las cachés unificadas *versus* las divididas, y también cuestiones de software. Además, hay un capítulo sobre coherencia de cachés en multiprocesadores.

Johnson, *Superscalar Microprocessor Design*

Para lectores interesados en los detalles del diseño de CPU superescalares, este libro es el mejor punto de partida: cubre la obtención y decodificación de instrucciones, la emisión de instrucciones en desorden, el cambio de nombre de registros, las estaciones de reservación, la predicción de ramas, y más.

Normoyle *et al.*, “UltraSPARC III: Expanding the Boundaries of a System on a Chip”

El UltraSPARC III es una versión para el bus PCI del UltraSPARC II. En este artículo sus diseñadores explican su funcionamiento interno.

McGhan y O'Connor, “picoJava: A Direct Execution Engine for Java Bytecode”

Si desea una breve introducción a la microarquitectura del diseño picoJava en hardware de Sun (y por tanto el chip microJava 701), este artículo es un buen punto de partida. Se presenta un diagrama de bloques, se analiza el conducto y se explica el funcionamiento de varias optimizaciones.

Shriver y Smith, “The Anatomy of a High-Performance Microprocessor”

Si desea un estudio detallado de un chip de CPU moderno en el nivel de microarquitectura, le recomendamos este libro, que examina el chip AMD K6, un clon del Pentium, con gran detalle, haciendo hincapié en la fila de procesamiento, la planificación de instrucciones y optimización del desempeño.

Sima, “Superscalar Instruction Issue”

La emisión de instrucciones superescalares es cada vez más importante en las CPU modernas. Mencionamos algunos de los aspectos en este capítulo, como el cambio de nombres y la ejecución especulativa. En este artículo se examinan éstas y varias otras cuestiones.

Wilson, “Challenges and Trends in Processor Design”

¿El diseño de procesadores se ha estancado? De ninguna manera. Seis importantes arquitectos de CPU de Sun, Cyrix, Motorola, Mips, Intel y Digital nos dicen hacia dónde se dirige el diseño de CPU en los próximos años. Va a ser divertido leer esto en 2008 (pero también vale la pena leerlo ahora).

9.1.5 El nivel de arquitectura del conjunto de instrucciones

Antonakos, *The Pentium Microprocessor*

Los primeros nueve capítulos de este libro explican cómo programar el Pentium en lenguaje ensamblador. Los dos últimos se ocupan del hardware del Pentium. Se proporcionan muchos fragmentos de código y se trata exhaustivamente el BIOS.

Paul, *SPARC Architecture, Assembly Language, Programming, and C*

Maravilla de maravillas: un libro acerca de la programación en lenguaje ensamblador que no se refiere a la línea Intel 80x86. En vez de ello, se ocupa del SPARC y de cómo programarlo.

Weaver y Germond, *The SPARC Architecture Manual*

Con la creciente internacionalización de la industria de las computadoras, los estándares se vuelven cada vez más importantes, y es importante familiarizarse con ellos. Ésta es la definición de la versión 9 de SPARC y da una buena idea de cómo es un estándar, además de proporcionar abundante información sobre el funcionamiento de los SPARC de 64 bits.

9.1.6 El nivel de máquina del sistema operativo

Hart, “Win32 System Programming”

A diferencia de casi todos los demás libros sobre Windows, éste no se concentra en la interfaz gráfica con el usuario (de hecho, ni siquiera la trata); en vez de ello, se concentra en las llamadas al sistema que Windows ofrece y en la forma de utilizarlas para acceder a archivos, administrar la memoria, administrar procesos, comunicación entre procesos, líneas, E/S y otros temas.

Jacob y Mudge, “Virtual Memory: Issues of Implementation”

Si busca una buena introducción moderna a la memoria virtual, aquí está. Se explican diversas estructuras de tabla de páginas y TLB y se ilustran las ideas utilizando los procesadores MIPS, PowerPC y Pentium.

Korn, “Porting UNIX to Windows NT”

Aunque a primera vista podría pensarse que dado el gran número de llamadas al sistema que tiene NT sería fácil trasladar a él programas en UNIX, los intentos por hacerlo muestran que es mucho más difícil de lo que parece. En este artículo, alguien que lo ha intentado explica por qué.

McKusick *et al.*, *Design and Implementation of the 4.4 BSD Operating System*

A diferencia de la mayor parte de los libros sobre UNIX, éste inicia con una fotografía de los cuatro autores en una Conferencia USENIX; tres de ellos escribieron una buena parte del 4.4 BSD y están eminentemente calificados para explicar su funcionamiento interno. El libro cubre las llamadas al sistema, procesos, E/S, y tiene una sección excelente sobre trabajo con redes.

Ritchie y Thompson, “The UNIX Time-Sharing System”

Éste es el artículo original publicado acerca de UNIX. Todavía es muy provechoso leerlo. De esta pequeña semilla creció un gran sistema operativo.

Solomon, *Inside Windows NT*, 2a. ed.

Si quiere saber cómo NT funciona internamente, le recomendamos este libro. Se explica la arquitectura del sistema, sus mecanismos, procesos, líneas, gestión de memoria, seguridad, E/S, cachés y sistema de archivos, entre otros temas.

Tanenbaum y Woodhull, *Operating Systems: Design and Implementation*, 2a. ed.

A diferencia de la mayor parte de los libros sobre sistemas operativos, que sólo se ocupan de la teoría, éste cubre toda la teoría pertinente y la ilustra analizando el código real de un sistema operativo tipo UNIX, MINIX, que se ejecuta en la IBM PC y otras computadoras. El código fuente, con abundantes comentarios, se presenta en un apéndice.

9.1.7 El nivel de lenguaje ensamblador

Irvine, *Assembly Language for Intel-Based Computers*, 3a. ed.

La programación de las CPU Intel en lenguaje ensamblador es el tema de este libro. También se cubren la programación de E/S, macros, archivos, enlace, interrupciones y muchos otros temas relacionados.

Saloman, *Assemblers and Loaders*

Todo lo que quiera saber acerca del funcionamiento de los ensambladores de una y dos pasadas, así como el de los enlazadores y cargadores. También se cubren macros y ensamblado condicional.

9.1.8 Arquitecturas de computadoras paralelas

Adve y Gharachorloo, “Shared Memory Consistency Models: A Tutorial”

Muchas computadoras modernas, sobre todo multiprocesadores, manejan un modelo de memoria más débil que la consistencia secuencial. Este tutorial analiza varios modelos y

explica cómo funcionan; también plantea y refuta numerosos mitos acerca de la memoria con consistencia débil.

Almasi y Gottlieb, *Highly Parallel Computing*, 2a. ed.

Si desea un panorama general de la computación paralela, incluidas redes de interconexión, arquitectura, compiladores, modelos y aplicaciones, este libro es muy recomendable. Su contenido está balanceado entre temas de hardware, software y aplicaciones.

Hill, “Multiprocessors Should Support Simple Memory-Consistency Models”

La semántica de memoria relajada es un tema álgido del diseño de memorias para multiprocesadores, y causa grandes controversias. Los modelos más débiles permiten ciertas optimaciones en hardware, como hacer referencias a la memoria en desorden, pero hacen más difícil la programación. En este artículo el autor trata muchas cuestiones importantes para la consistencia de la memoria y luego concluye que la memoria relajada no vale la pena.

Hwang y Xu, *Scalable Parallel Computing*

Al tratar tanto el hardware como el software, los autores logran presentar una reseña completa y amena de la computación en paralelo. Los temas incluyen multiprocesadores UMA y NUMA, MPP y COW, transferencia de mensajes y programación paralela.

Pfister, *In Search of Clusters*, 2a. ed.

Aunque la definición de cúmulo no aparece sino hasta la página 72 (un grupo de computadoras completas que trabajan juntas), al parecer incluye todos los sistemas de multiprocesador y multicomputadora usuales. Se estudian con detalle su hardware, software, desempeño y disponibilidad. Sin embargo, el lector queda advertido de que si bien el monísimo estilo de redacción del autor al principio resulta divertido, después de 500 páginas es más bien empalagoso.

Snir *et al.*, *MPI: The Complete Reference Manual*

El título lo dice todo. Si quiere aprender a programar en MPI, aquí está lo que busca. El libro cubre la comunicación punto a punto y colectiva, comunicadores, gestión de entornos, perfiles y más.

Stenstrom *et al.*, “Trends in Shared Memory Multiprocessing”

Aunque muchos creen que los multiprocesadores con memoria compartida son supercomputadoras para realizar cálculos científicos masivos, en realidad ésa es sólo una fracción diminuta de su mercado. En este artículo los autores explican cuál es el verdadero mercado de estas máquinas, y qué implicaciones tiene para su arquitectura.

9.1.9 Números binarios y de punto flotante

Cody, "Analysis of Proposals for the Floating-Point Standard"

Hace algunos años, IEEE diseñó una arquitectura de punto flotante que se ha convertido en el estándar *de facto* para todos los chips de CPU modernos. Cody explica las diversas cuestiones, propuestas y controversias que surgieron durante el proceso de estandarización.

Garner, "Number Systems and Arithmetic"

Tutorial sobre conceptos avanzados de aritmética binaria, incluida la propagación de acarreo, sistemas de numeración redundantes, sistemas de numeración de residuo, y multiplicación y división no estándar. Muy recomendable para quien cree que ya sabe todo lo habido y por haber sobre aritmética en la primaria.

IEEE, *Proc. of the n-th Symposium on Computer Arithmetic*

A pesar de lo que comúnmente se cree, la aritmética es un área de investigación activa en la que se escriben muchos trabajos por y para especialistas en aritmética. En esta serie de simposios se presentan avances sobre suma y multiplicación de alta velocidad, hardware VLSI para aritmética, coprocesadores, tolerancia ante fallos y redondeo, entre otros temas.

Knuth, *Seminumerical Algorithms*, 3a. ed.

Abundante material sobre sistemas de numeración posicionales, aritmética de punto flotante, aritmética de múltiple precisión y números aleatorios. Este material requiere y merece un estudio cuidadoso.

Wilson, "Floating-Point Survival Kit"

Una bonita introducción a los números de punto flotante y sus estándares para gente que cree que el mundo acaba en 65,535. También se tratan algunas pruebas estándar para desempeño de punto flotante, como *Linpack*.

9.2 BIBLIOGRAFÍA EN ORDEN ALFABÉTICO

ADAMS, G.B. III, AGRAWAL, D.P., and SIEGEL, H.J.: "A Survey and Comparison of Fault-Tolerant Multistage Interconnection Networks," *IEEE Computer Magazine*, vol. 20, pp. 14-27, June 1987.

ADVE, S.V., and CHARACHORLOO, K.: "Shared Memory Consistency Models: A Tutorial," *IEEE Computer Magazine*, vol. 29, pp. 66-76, Dec. 1996.

ADVE, S.V., and HILL, M.: "Weak Ordering: A New Definition," *Proc. 17th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 2-14, 1990.

AGERWALA, T., and COCKE, J.: "High Performance Reduced Instruction Set Processors," IBM T.J. Watson Research Center Technical Report RC12434, 1987.

ALMASI, G.S., and GOTTLIEB, A.: *Highly Parallel Computing*, 2nd ed. Redwood City, CA: Benjamin/Cummings, 1994.

AMZA, C., COX, A., DWARKADAS, S., KELEHER, P., LU, H., RAJAMONY, R., YU, W., ZWAENEPOEL, W.: "TreadMarks: Shared Memory Computing on a Network of Workstations," *IEEE Computer Magazine*, vol. 29, pp. 18-28, Feb. 1996.

ANDERSON, D.: *Universal Serial Bus System Architecture*, Reading, MA: Addison-Wesley, 1997.

ANDERSON, T.E., CULLER, D.E., PATTERSON, D.A., and the NOW team: "A Case for NOW (Networks of Workstations)," *IEEE Micro Magazine*, vol. 15, pp. 54-64, Feb. 1995.

ANTONAKOS, J.L.: *The Pentium Microprocessor*, Upper Saddle River, NJ: Prentice Hall, 1997.

AUGUST, D.I., CONNORS, D.A., MSHLKE, S.A., SIAS, J.W., CROZIER, K.M., CHENG, B.-C., EATON, P.R., OLANIRAN, Q.B., and HWU, W.-M.: "Integrated Predicated and Speculative Execution in the IMPACT EPIC Architecture," *Proc. 25th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 227-237, 1998.

BAL, H.E.: *Programming Distributed Systems*, Hemel Hempstead, England: Prentice Hall Int'l., 1991.

BAL, H.E., BHOEDJANG, R., HOFMAN, R., JACOBS, C., LANGENDOEN, K., RUHL, T., and KAASHOEK, M.F.: "Performance Evaluation of the Orca Shared Object System," *ACM Trans. on Computer Systems*, vol. 16, pp. 1-40, Feb. 1998.

BAL, H.E., KAASHOEK, M.F., and TANENBAUM, A.S.: "Orca: A Language for Parallel Programming of Distributed Systems," *IEEE Trans. on Software Engineering*, vol. 18, pp. 190-205, March 1992.

BAL, H.E., and TANENBAUM, A.S.: "Distributed Programming with Shared Data," *Proc. 1988 Int'l. Conf. on Computer Languages*, IEEE, pp. 82-91, 1988.

BHUYAN, L.N., YANG, Q., and AGRAWAL, D.P.: "Performance of Multiprocessor Interconnection Networks," *IEEE Computer Magazine*, vol. 22, pp. 25-37, Feb. 1989.

BJORNSON, R.D.: "Linda on Distributed Memory Multiprocessors," Ph.D. Thesis, Yale Univ., 1993.

BLUMRICH, M.A., DUBNICKI, C., FELTEN, E.W., LI, K., and MESARINA, M.R.: "Virtual-Memory Mapped Network Interfaces," *IEEE Micro Magazine*, vol. 15, pp. 21-28, Feb. 1995.

BODEN, N.J., COHEN, D., FELDERMAN, R.E., KULAWIK, A.E., SEITZ, C.L., SEIZOVIC, J.N., and SU, W.-K.: "Myrinet: A Gigabit per second Local Area Network," *IEEE Micro Magazine*, vol. 15, pp. 29-36, Feb. 1995.

BOUKNIGHT, W.J., DENENBERG, S.A., MCINTYRE, D.E., RANDALL, J.M., SAMEH, A.H., and SLOTNICK, D.L.: "The Illiac IV System," *Proc. IEEE*, pp. 369-388, April 1972.

- BURKHARDT, H., FRANK, S., KNOBE, B., and ROTHNIE, J.: "Overview of the KSR-1 Computer System," Technical Report KSR-TR-9202001, Kendall Square Research Corp, Cambridge, MA, 1992.
- CARRIERO, N., and GELERNTER, D.: "The S/Net's Linda Kernel," *ACM Trans. on Computer Systems*, vol. 4, pp. 110-129, May 1986.
- CARRIERO, N., and GELERNTER, D.: "Linda in Context," *Commun. of the ACM*, vol. 32, pp. 444-458, April 1989.
- CHARLESWORTH, A.: "Starfire: Extending the SMP Envelope," *IEEE Micro Magazine*, vol. 18, 39-49, Jan./Feb. 1998.
- CHARLESWORTH, A., PHELPS, A., WILLIAMS, R., and GILBERT, G.: "Gigaplane-XB: Extending the Ultra Enterprise Family," *Proc. Hot Interconnects V*, IEEE, 1998.
- CODY, W.J.: "Analysis of Proposals for the Floating-Point Standard," *IEEE Computer Magazine*, vol. 14, pp. 63-68, Mar. 1981.
- COHEN, D.: "On Holy Wars and a Plea for Peace," *IEEE Computer Magazine*, vol. 14, pp. 48-54, Oct. 1981.
- CORBATO, F.J.: "PL/1 as a Tool for System Programming," *Datamation*, vol. 15, pp. 68-76, May 1969.
- CORBATO, F.J., and VYSSOTSKY, V.A.: "Introduction and Overview of the MULTICS System," *Proc. FJCC*, pp. 185-196, 1965.
- DENNING, P.J.: "The Working Set Model for Program Behavior," *Commun. of the ACM*, vol. 11, pp. 323-333, May 1968.
- DIJKSTRA, E.W.: "GOTO Statement Considered Harmful," *Commun. of the ACM*, vol. 11, pp. 147-148, Mar. 1968a.
- DIJKSTRA, E.W.: "Co-operating Sequential Processes," in *Programming Languages*, F. Genuys (ed.), New York: Academic Press, 1968b.
- DRIESEN, K., and HOLZLE, URS: "Accurate Indirect Branch Prediction," *Proc. 25th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 167-177, 1998.
- DUBOIS, M., SCHEURICH, C., and BRIGGS, F.A.: "Memory Access Buffering in Multiprocessors," *Proc. 13th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 434-442, 1986.
- DULONG, C.: "The IA-64 Architecture at Work," *IEEE Computer Magazine*, vol. 31, pp. 24-32, July 1998.
- FAGGIN, F., HOFF, M.E., Jr., MAZOR, S., and SHIMA, M.: "The History of the 4004," *IEEE Micro Magazine*, vol. 16, pp. 10-20, Dec. 1996.
- FALSAFI, B., and WOOD, D.A.: "Reactive NUMA: A Design Unifying S-COMA and CC-NUMA," *Proc. 25th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 229-240, 1997.
- FISHER, J.A., and FREUDENBERGER, S.M.: "Predicting Conditional Branch Directions from Previous Runs of a Program," *Proc. 5th Int'l. Conf. on Arch. Support for Prog. Lang. and Operating Syst.*, ACM, pp. 85-95, 1992.

- FLOYD, T.L.: *Digital Fundamentals*, 6th ed., Upper Saddle River, NJ: Prentice Hall, 1997.
- FLYNN, M.J.: "Some Computer Organizations and Their Effectiveness," *IEEE Trans. on Computers*, vol. C-21, pp. 948-960, Sept. 1972.
- FOSTER, I., and KESSELMAN, C.: "Globus: A Metacomputing Infrastructure Toolkit," *Int'l. J. of Supercomputer Applications*, vol. 11, pp. 115-128, 1998a.
- FOSTER, I., and KESSELMAN, C.: "The Globus Project: A Status Report," *IPPS/SPDP '98 Heterogeneous Computing Workshop*, IEEE, pp. 4-18, 1998b.
- FOTHERINGHAM, J.: "Dynamic Storage Allocation in the Atlas Computer Including an Automatic Use of a Backing Store," *Commun. of the ACM*, vol. 4, pp. 435-436, Oct. 1961.
- GAJSKI, D.D., and PIER, K.-K.: "Essential Issues in Multiprocessor Systems," *IEEE Computer Magazine*, vol. 18, pp. 9-27, June 1985.
- GARNER, H.L.: "Number Systems and Arithmetic," in *Advances in Computers*, vol. 6, F. Alt and M. Rubinoff (eds.), New York: Academic Press, 1965, pp. 131-194.
- GEIST, A., BEGUELIN, A., DONGARRA, J., JIANG, W., MANCHECK, R., and SUNDERRAM, V.: *PVM: Parallel Virtual Machine — A User's Guide and Tutorial for Networked Parallel Computing*, Cambridge, MA: M.I.T. Press, 1994.
- GHARACHORLOO, K., LENOSKI, D., LAUDON, J., GIBBONS, P., GUPTA, A., and HENNESSY, J.: "Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors," *Proc. 17th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 15-26, 1990.
- GOODMAN, J.R.: "Using Cache Memory to Reduce Processor Memory Traffic," *Proc. 10th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 124-131, 1983.
- GOODMAN, J.R.: "Cache Consistency and Sequential Consistency," Tech. Rep. 61, IEEE Scalable Coherent Interface Working Group, IEEE, 1989.
- GRAHAM, R.: "Use of High Level Languages for System Programming," Project MAC Report TM-13, Project MAC, MIT, Sept. 1970.
- GRIMSHAW, A.S., and WULF, W.: "Legion: A View from 50,000 Feet," *Proc. Fifth Int'l. Symp. on High-Performance Distributed Computing*, IEEE, pp. 89-99, Aug. 1996.
- GRIMSHAW, A.S., and WULF, W.: "The Legion Vision of a Worldwide Virtual Computer," *Commun. of the ACM*, vol. 40, pp. 39-45, Jan. 1997.
- GROPP, W., LUSK, E., and SKJELLUM, A.: "Using MPI: Portable Parallel Programming with the Message Passing Interface," Cambridge, MA: M.I.T. Press, 1994.
- HAGERSTEN, E., LANDIN, A., HARIDI, S.: "DDM—A Cache-Only Memory Architecture," *IEEE Computer Magazine*, vol. 25, pp. 44-54, Sept. 1992.
- HAMACHER, V.V., VRANESIC, Z.G., and ZAKY, S.G.: *Computer Organization*, 4th ed., New York: McGraw-Hill, 1996.
- HAMMING, R.W.: "Error Detecting and Error Correcting Codes," *Bell Syst. Tech. J.*, vol. 29, pp. 147-160, April 1950.

- HANDY, J.: *The Cache Memory Book*, 2nd ed., Orlando, FL: Academic Press, 1998.
- HART, J.M.: *Win32 System Programming*, Reading, MA: Addison-Wesley, 1997.
- HAYES, J.P.: *Computer Architecture and Organization*, 3rd ed., New York: McGraw-Hill, 1998.
- HENNESSY, J.L.: "VLSI Processor Architecture," *IEEE Trans. on Computers*, vol. C-33, pp. 1221-1246, Dec. 1984.
- HILL, M.: "Multiprocessors Should Support Simple Memory-Consistency Models," *IEEE Computer Magazine*, vol. 31, pp. 28-34, Aug. 1998.
- HOARE, C.A.R.: "Monitors, An Operating System Structuring Concept," *Commun. of the ACM*, vol. 17, pp. 549-557, Oct. 1974; Erratum in *Commun. of the ACM*, vol. 18, p. 95, Feb. 1975.
- HWANG, K., and XU, Z.: *Scalable Parallel Computing*, New York: McGraw-Hill, 1998.
- HWU, W.-M.: "Introduction to Predicated Execution," *IEEE Computer Magazine*, vol. 31, pp. 49-50, Jan. 1998.
- IRVINE, K.: *Assembly Language for Intel-Based Computers*, 3rd ed., Englewood Cliffs, NJ: Prentice Hall, 1999.
- JACOB, B., and MUDGE, T.: "Virtual Memory: Issues of Implementation," *IEEE Computer Magazine*, vol. 31, pp. 33-43, June 1998a.
- JACOB, B., and MUDGE, T.: "Virtual Memory in Contemporary Microprocessors," *IEEE Micro Magazine*, vol. 18, pp. 60-75, July/Aug. 1998b.
- JOE, T., and HENNESSY, J.L.: "Evaluating the Memory Overhead Required for COMA Architectures," *Proc. 21th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 82-93, 1994.
- JOHNSON, K.L., KAASHOEK, M.F., and WALLACH, D.A.: "CRL: High-Performance All-Software Distributed Shared Memory," *Proc. 15th Symp. on Operating Systems Principles*, ACM, pp. 213-228, 1995.
- JOHNSON, M.: *Superscalar Microprocessor Design*, Englewood Cliffs, NJ: Prentice Hall, 1991..
- JUAN, T., SANJEEVAN, S., and NAVARRO, J.J.: "Dynamic History-Length Fitting: A Third Level of Adaptivity for Branch prediction," *Proc. 25th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 155-166, 1998.
- KATAYAMA, Y.: "Trends in Semiconductor Memories," *IEEE Micro Magazine*, pp. 10-17, Nov./Dec. 1997.
- KERMARREC, A.-M., KUZ, I., VAN STEEN, M., and TANENBAUM, A.S.: "A Framework for Consistent Replicated Web Objects," *Proc. 18th Int'l. Conf. on Distr. Computing Syst.*, IEEE, pp. 276-284, 1998.
- KNUTH, D.E.: "An Empirical Study of FORTRAN Programs," *Software—Practice & Experience*, vol. 1, pp. 105-133, 1971.

- KNUTH, D.E.: *The Art of Computer Programming: Fundamental Algorithms*, 3rd ed., Reading, MA: Addison-Wesley, 1997.
- KNUTH, D.E.: *The Art of Computer Programming: Seminumerical Algorithms*, 3rd ed., Reading, MA: Addison-Wesley, 1998.
- KONTOTHANASSIS, L., HUNT, G., STETS, R., HARDAVELLAS, N., CIERNIAD, M., PARTHASARATHY, S., MEIRA, W., DWARKADAS, S., and SCOTT, M.: *VM-Based Shared Memory on Low Latency Remote Memory Access Networks*, *Proc. 24th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 157-169, 1997.
- KORN, D.: "Porting UNIX to Windows NT," *Proc. Winter 1997 USENIX Conf.*, pp. 43-57, 1997.
- KUMAR, V.P., and REDDY, S.M.: "Augmented Shuffle-Exchange Multistage Interconnection Networks," *IEEE Computer Magazine*, vol. 20, pp. 30-40, June 1987.
- LAMPORT, L.: "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Trans. on Computers*, vol. C-28, pp. 690-691, Sept. 1979.
- LAROWE, R.P., and ELLIS, C.S.: "Experimental Comparison of Memory Management Policies for NUMA Multiprocessors," *ACM Trans. on Computer Systems*, vol. 9, pp. 319-363, Nov. 1991.
- LENOSKI, D., LAUDON, J., GHARACHORLOO, K., WEBER, W.-D., GUPTA, A., HENNESSY, J., HOROWITZ, M., and LAM, M.: "The Stanford Dash Multiprocessor," *IEEE Computer Magazine*, vol. 25, pp. 63-79, March 1992.
- LI, K.: "IVY: A Shared Virtual Memory System for Parallel Computing," *Proc. 1988 Int'l. Conf. on Parallel Proc. (Vol. II)*, IEEE, pp. 94-101, 1988.
- LI, K., and HUDAK, P.: "Memory Coherence in Shared Virtual Memory Systems," *ACM Trans. on Computer Systems*, vol. 7, pp. 321-359, Nov. 1989.
- LI, K., and HUDAK, P.: "Memory Coherence in Shared Virtual Memory Systems," *Proc. 5th Ann. ACM Symp. on Prin. of Distr. Computing*, ACM, pp. 229-239, 1986.
- LINDHOLM, T., and YELLIN, F.: *The Java Virtual Machine Specification*, Reading, MA: Addison-Wesley, 1997.
- LOSHIN, D.: *High Performance Computing Demystified*, Cambridge, MA: AP Prof., 1994.
- LU, H., COX, A.L., DWARKADAS, S., RAJAMONY, R., and ZWAENEPOEL, W.: "Software Distributed Shared Memory Support for Irregular Applications," *Proc. 6th Conf. on Prin. and Practice of Parallel Progr.*, pp. 48-56, June 1997.
- LUKASIEWICZ, J.: *Aristotle's Syllogistic*, 2nd ed., Oxford: Oxford University Press, 1958.
- MANO, M.M., and KIME, C.R.: *Logic and Computer Design Fundamentals*, Upper Saddle River, NJ: Prentice Hall, 1997.
- MARTIN, R.P., VAHDAT, A.M., CULLER, D.E., and ANDERSON, T.E.: "Effects of Communication Latency, Overhead, and Bandwidth in a Cluster Architecture," *Proc. 24th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 85-97, 1997.

- MAZIDI, M.A., and MAZIDI, J.G.: *The 80x86 IBM PC and Compatible Computers, 2nd ed.*, Upper Saddle River, NJ: Prentice Hall, 1998.
- McGHAN, H. and O'CONNOR, J.M.: "picoJava: A Direct Execution Engine for Java Bytecode," *IEEE Computer Magazine*, vol. 31., Oct. 1998.
- McKEE, S.A., KLENKE, R.H., WRIGHT, K.L., WULF, W.A., SALINAS, M.H., AYLOR, J.H., and BATSON, A.P.: "Smarter Memory: Improving Bandwidth for Streamed References," *IEEE Computer Magazine*, vol. 31, pp. 54-63, July 1998.
- McKUSICK, M.K., BOSTIC, K., KARELS, M., and QUARTERMAN, J.S.: "The Design and Implementation of the 4.4 BSD Operating System," Reading, MA: Addison-Wesley, 1996.
- McKUSICK, M.K., JOY, W.N., LEFFLER, S.J., AND FABRY, R.S.: "A Fast File System for UNIX," *ACM Trans. on Computer Systems*, vol. 2, pp. 181-197, Aug. 1984.
- MESSMER, H.-P.: *The Indispensible PC Hardware Book, 3rd ed.*, Reading, MA: Addison-Wesley, 1997.
- MORGAN, C.: *Portraits in Computing*, New York: ACM Press, 1997.
- MORIN, C., GEFFLAUT, A., BANATRE, M., and KERMARREC, A.-M.: "COMA: An Opportunity for Building a Fault-Tolerant Scalable Shared Memory Multiprocessor," *Proc. 24th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 65-65, 1996.
- MOUDGILL, M., and VASSILIADIS, S.: "Precise Interrupts," *IEEE Micro Magazine*, vol. 16, pp. 58-67, Feb. 1996.
- MULLENDER, S.J., and TANENBAUM, A.S.: "Immediate Files," *Software—Practice and Experience*, vol. 14, pp. 365-368, 1984.
- NELSON, V.P., NAGLE, H.T., CARROLL, B.D., and IRWIN, J.D.: *Digital Logic and Circuit Analysis and Design*, Englewood Cliffs, NJ: Prentice Hall, 1995.
- NG, S.W.: "Advances in Disk Technology: Performance Issues," *IEEE Computer Magazine*, vol. 31, pp. 75-81, May 1998.
- NORMOYLE, K.B., CSOPPENSZKY, M.A., TZENG, A., JOHNSON, T.P., FURMAN, C.D., and MOSTOUFI, J.: "UltraSPARC IIi: Expanding the Boundaries of a System on a Chip," *IEEE Micro Magazine*, vol. 18, pp. 14-24, March/April 1998.
- NORTON, P., and GOODMAN, J.: *Inside the PC, 7th ed.*, Indianapolis, IN: Sams, 1997.
- O'CONNOR, J.M., and TREMBLAY, M.: "PicoJava-I: The Java Virtual Machine in Hardware," *IEEE Micro Magazine*, vol. 17, pp. 45-53, March/April 1997.
- ORGANICK, E.: *The MULTICS System*, Cambridge, MA: M.I.T. Press, 1972.
- PAKIN, S., KARAMCHETI, V., and CHIEN, A.A.: "Fast Messages (FM): Efficient, Portable Communication for Workstation Cluster and Massively-Parallel Processors," *IEEE Concurrency*, vol. 5, pp. 60-73, April-June 1997.
- PAN, S.-T., SO, K., and RAHMEH, J.T.: "Improving the Accuracy of Dynamic Branch Prediction Using Branch Correlation," *Proc. 5th Int'l. Conf. on Arch. Support for Prog. Lang. and Operating Syst.*, ACM, pp. 76-84, Oct. 1992.

- PAPAMARCOS, M., and PATEL, J.: "A Low Overhead Coherence Solution for Multiprocessors with Private Cache Memories," *Proc. 11th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 348-354, 1984.
- PATTERSON, D.A.: "Reduced Instruction Set Computers," *Commun. of the ACM*, vol. 28, pp. 8-21, Jan. 1985.
- PATTERSON, D.A., GIBSON, G., and KATZ, R.: "A case for redundant arrays of inexpensive disks (RAID)," *Proc. ACM SIGMOD Int'l. Conf. on Management of Data*, ACM, pp. 109-166, 1988.
- PATTERSON, D.A., and HENNESSY, J.L.: *Computer Organization and Design, 2nd ed.*, San Francisco, CA: Morgan Kaufmann, 1998.
- PATTERSON, D.A., and SEQUIN, C.H.: "A VLSI RISC," *IEEE Computer Magazine*, vol. 15, pp. 8-22, Sept. 1982.
- PAUL, R.P.: *SPARC Architecture, Assembly Language, Programming, and C*, Englewood Cliffs, NJ: Prentice Hall, 1994.
- PFISTER, G.F.: *In Search of Clusters, 2nd ed.*, Upper Saddle River, NJ: Prentice Hall, 1998.
- PILGRIM, A.: *Build Your Own Pentium II PC*, New York: McGraw-Hill, 1998.
- POUNTAIN, D.: "Pentium: More RISC than CISC," *Byte*, vol. 18, pp. 195-204, Sept. 1993.
- PRICE, D.: "A History of Calculating Machines," *IEEE Micro Magazine*, vol. 4, pp. 22-52, Feb. 1984.
- RADIN, G.: "The 801 Minicomputer," *Computer Arch. News*, vol. 10, pp. 39-47, March 1982.
- RITCHIE, D.M., and THOMPSON, K.: "The UNIX Time-Sharing System," *Commun. of the ACM*, vol. 17, pp. 365-375, July 1974.
- ROSENBLUM, M., and OUSTERHOUT, J.K.: "The Design and Implementation of a Log-Structured File System," *Proc. Thirteenth Symp. on Operating System Principles*, ACM, pp. 1-15, 1991.
- SALOMAN, D.: *Assemblers and Loaders*, Englewood Cliffs, NJ: Prentice Hall, 1993.
- SAULSBURY, A., WILKINSON, T., CARGER, J., and LANDIN, A.: "An Argument for Simple COMA," *Proc. of First IEEE Symp. on High-Performance Comp. Arch.*, IEEE, pp. 276-285, 1995.
- SCALES, D.J., GHARACHORLOO, K., and THEKKATH, C.A.: "Shasta: A Low-Overhead Software-Only Approach for Supporting Fine-Grain Shared Memory," *Proc. 7th Int'l. Conf. on Arch. Support for Prog. Lang. and Oper. Syst.*, ACM, pp. 174-185, 1996.
- SECHREST, S., LEE, C.-C., and MUDGE, T.: "Correlation and Aliasing in Dynamic Branch Predictors," *Proc. 23th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 22-32, 1996.
- SELTZER, M., BOSTIC, K., McKUSICK, M.K., and STAELIN, C.: "An Implementation of a Log-Structured File System for UNIX," *Proc. Winter 1993 USENIX Technical Conf.*, pp. 307-326, 1993.

- SHANLEY, T., and ANDERSON, D.: *ISA System Architecture*, Reading, MA: Addison-Wesley, 1995a.
- SHANLEY, T., and ANDERSON, D.: *PCI System Architecture, 3rd ed.*, Reading, MA: Addison-Wesley, 1995b.
- SHRIVER, B., and SMITH, B.: *The Anatomy of a High-Performance Microprocessor: A Systems Perspective*, Los Alamitos, CA: IEEE Computer Society, 1998.
- SIMA, D.: "Superscalar Instruction Issue," *IEEE Micro Magazine*, vol. 17, pp. 28-39, Sept./Oct 1997.
- SIMA, D., FOUNTAIN, T., KACSUK, P.: *Advanced Computer Architectures: A Design Space Approach*, Reading, MA: Addison-Wesley, 1997.
- SLATER, R.: *Portraits in Silicon*, Cambridge, MA: M.I.T. Press, 1987.
- SMITH, A.J.: "Cache Memories," *Computing Surveys*, vol. 14, pp. 473-530, Sept. 1982.
- SNIR, M., OTTO, S.W., HUSS-LEDERMAN, S., WALKER, D.W., and DONGARRA, J.: *MPI: The Complete Reference Manual*, Cambridge, MA: M.I.T. Press, 1996.
- SOLARI, E.: *ISA & EISA Theory and Operation*, San Diego, CA: Annabooks, 1993.
- SOLARI, E., and WILLSE, G.: *PCI Hardware and Software Architecture and Design, 4th ed.*, San Diego, CA: Annabooks, 1998.
- SOLOMON, D.A.: *Inside Windows NT, 2nd ed.*, Redmond, WA: Microsoft Press, 1998.
- SPRANGLE, E., CHAPPELL, R.S., ALSUP, M., PATT, Y.N.: "The Agree Predictor: A Mechanism for Reducing Negative Branch History Interference," *Proc. 24th Ann. Int'l. Symp. on Computer Arch.*, ACM, pp. 284-291, 1997.
- STALLINGS, W.: *Computer Organization and Architecture, 4th ed.*, Upper Saddle River, NJ: Prentice Hall, 1996.
- STENSTROM, P., HAGERSTEN, E., LILJA, D.J., MARTONOSI, M., and VENUGOPAL, M.: "Trends in Shared Memory Multiprocessing," *IEEE Computer Magazine*, vol. 30, pp. 44-50, Dec. 1997.
- STETS, R., DWARKADAS, S., HARDAVELLAS, N., HUNT, G., KONTOTHANASSIS, L., PARTHASARATHY, S., and SCOTT, M.: "CASHMERE-2L: Software Coherent Shared Memory on Clustered Remote-Write Networks," *Proc. 16th Symp. on Operating Systems Principles*, ACM, pp. 170-183, 1997.
- SUNDERRAM, V.B.: "PVM: A Framework for Parallel Distributed Computing," *Concurrency: Practice and Experience*, vol. 2, pp. 315-339, Dec. 1990.
- SWAN, R.J., FULLER, S.H., and SIEWIOREK, D.P.: "Cm*—A Modular Multiprocessor," *Proc. NCC*, pp. 645-655, 1977.
- TAN, W.M.: *Developing USB PC Peripherals*, San Diego, CA: Annabooks, 1997.
- TANENBAUM, A.S.: "Implications of Structured Programming for Machine Architecture," *Commun. of the ACM*, vol. 21, pp. 237-246, Mar. 1978.

- TANENBAUM, A.S., and WOODHULL, A.W.: *Operating Systems: Design and Implementation, 2nd ed.*, Upper Saddle River, NJ: Prentice Hall, 1997.
- THOMPSON, K.: "UNIX Implementation," *Bell Syst. Tech. J.*, vol. 57, pp. 1931-1946, July-Aug. 1978.
- TRELEAVEN, P.: "Control-Driven, Data-Driven, and Demand-Driven Computer Architecture," *Parallel Computing*, vol. 2, 1985.
- TREMBLAY, M. and O'CONNOR, J.M.: "UltraSPARC I: A Four-Issue Processor Supporting Multimedia," *IEEE Micro Magazine*, vol. 16, pp. 42-50, April 1996.
- TRIEBEL, W.A.: *The 80386, 80486, and Pentium Processor*, Upper Saddle River, NJ: Prentice Hall, 1998.
- UNGER, S.H.: "A Computer Oriented Toward Spatial Problems," *Proc. IRE*, vol. 46, pp. 1744-1750, 1958.
- VAHALIA, U.: *UNIX Internals*, Upper Saddle River, NJ: Prentice Hall, 1996.
- VAN DER POEL, W.L.: "The Software Crisis, Some Thoughts and Outlooks," *Proc. IFIP Congr. 68*, pp. 334-339, 1968.
- VAN STEEN, M., HOMBURG, P.C., and TANENBAUM, A.S.: "The Architectural Design of Globe: A Wide-Area Distributed System," *IEEE Concurrency*, 1999.
- VERSTOEP, K., LANGEDOEN, K., and BAL, H.E.: "Efficient Reliable Multicast on Myrinet," *Proc. 1996 Int'l. Conf. on Parallel Processing*, IEEE, pp. 156-165, 1996.
- WEAVER, D.L., and GERMOND, T.: *The SPARC Architecture Manual, Version 9*, Englewood Cliffs, NJ: Prentice Hall, 1994.
- WILKES, M.V.: "Computers Then and Now," *J. ACM*, vol. 15, pp. 1-7, Jan. 1968.
- WILKES, M.V.: "The Best Way to Design and Automatic Calculating Machine," *Proc. Manchester Univ. Computer Inaugural Conf.*, 1951.
- WILKINSON, B.: *Computer Architecture: Design and Performance, 2nd ed.*, Englewood Cliffs, NJ: Prentice Hall, 1994.
- WILKINSON, B. and ALLEN, M.: *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*, Upper Saddle River, NJ: Prentice Hall, 1999.
- WILSON, J.: "Challenges and Trends in Processor Design," *IEEE Computer Magazine*, vol. 31, pp. 39-48, Jan. 1998.
- WILSON, P.: "Floating-Point Survival Kit," *Byte*, vol. 13, pp. 217-226, March 1988.
- YEH, T.-Y., and PATT, Y.-N.: "Two-Level Adaptive Training Branch Prediction," *Proc. 24th Int'l. Symp. on Microarchitecture*, ACM/IEEE, pp. 51-61, 1991.

A

NÚMEROS BINARIOS

La aritmética que las computadoras usan difiere en ciertos aspectos de la aritmética que aplica la gente. La diferencia más importante es que las computadoras realizan operaciones con números cuya precisión es finita y fija. Otra diferencia es que casi todas las computadoras usan el sistema binario en lugar del decimal para representar números. Estos temas son materia del presente apéndice.

A.1 NÚMEROS DE PRECISIÓN FINITA

Al efectuar aritmética, pocas veces pensamos en cuántos dígitos decimales se necesitan para representar un número. Los físicos pueden calcular que existen 10^{78} electrones en el Universo sin preocuparse por el hecho de que se requieren 79 dígitos decimales para escribir este número con todas sus cifras. Alguien que calcula el valor de una función con lápiz y papel y que necesita una respuesta con seis cifras significativas simplemente guarda resultados intermedios con siete, ocho, o las que se necesiten. El problema de que el papel no tenga la anchura suficiente para números de siete dígitos nunca se presenta.

Con las computadoras las cosas son muy distintas. En casi todas las computadoras, la cantidad de memoria con que se cuenta para almacenar un número se fija en el momento en que se diseña la máquina. Con algo de esfuerzo, el programador puede representar números dos, tres o incluso más veces más grandes que esta cantidad física, pero hacerlo no altera la naturaleza de este problema. La naturaleza finita de la computadora nos obliga a manejar

sólo números que se puedan representar con un número fijo de dígitos, tales números se conocen como **números de precisión finita**.

A fin de estudiar las propiedades de los números de precisión finita, examinemos el conjunto de enteros positivos que pueden representarse con tres dígitos decimales, sin punto decimal y sin signo. Este conjunto tiene exactamente 1000 miembros: 000, 001, 002, 003, ..., 999. Con esta restricción, es imposible expresar cierto tipo de números, como

1. Números mayores que 999.
2. Números negativos.
3. Fracciones.
4. Números irracionales.
5. Números complejos.

Una propiedad importante de la aritmética del conjunto de todos los enteros es la **cerradura** respecto a las operaciones de suma, resta y multiplicación. En otras palabras, para todo par de enteros i y j , $i + j$, $i - j$ e $i \times j$ son también enteros. El conjunto de enteros no está cerrado respecto a la división, porque existen valores de i y j para los cuales i/j no puede expresarse como un entero (p. ej., $7/2$ y $1/0$).

Los números de precisión finita no están cerrados respecto a ninguna de estas cuatro operaciones básicas, como se muestra en seguida usando números decimales de tres dígitos como ejemplo:

$$\begin{array}{ll} 600 + 600 = 1200 & (\text{demasiado grande}) \\ 003 - 005 = -2 & (\text{negativo}) \\ 050 \times 050 = 2500 & (\text{demasiado grande}) \\ 007 / 002 = 3.5 & (\text{no es entero}) \end{array}$$

Las violaciones pueden dividirse en dos clases mutuamente exclusivas: operaciones cuyo resultado es más grande que el número más grande del conjunto (error de desbordamiento) o más pequeño que el número más pequeño del conjunto (error de subdesbordamiento), y operaciones cuyo resultado no es ni demasiado grande ni demasiado pequeño; simplemente no es miembro del conjunto. De las cuatro violaciones anteriores, las primeras tres son ejemplos del primer caso, y la cuarta es un ejemplo del segundo.

Puesto que las computadoras tienen memorias finitas y por tanto forzosamente realizan aritmética con números de precisión finita, los resultados de ciertos cálculos serán, desde el punto de vista de las matemáticas clásicas, equivocados. Un dispositivo de cálculo que da una respuesta errónea aunque está funcionando perfectamente podría parecer extraño a primera vista, pero el error es una consecuencia lógica de su naturaleza finita. Algunas computadoras tienen hardware especial que detecta errores de desbordamiento.

El álgebra de los números de precisión finita es diferente del álgebra normal. Por ejemplo, consideremos la ley asociativa:

$$a + (b - c) = (a + b) - c$$

Evaluemos ambos miembros para $a = 700$, $b = 400$, $c = 300$. Para calcular el miembro izquierdo, calculemos primero $(b - c)$, que es 100, y sumemos luego esta cantidad a a , lo que

da 800. Para calcular el miembro derecho, calculemos primero $(a + b)$, que produce un desbordamiento en la aritmética finita de enteros de tres dígitos. El resultado depende de la máquina que se esté usando, pero no será 1100. Restar 300 a un número que no es 1100 no da 800. La ley asociativa no se cumple. El orden de las operaciones es importante.

Como ejemplo adicional, consideremos la ley distributiva:

$$a \times (b - c) = a \times b - a \times c$$

Evaluemos ambos miembros para $a = 5$, $b = 210$, $c = 195$. El miembro izquierdo es 5×15 , que da 75. El miembro derecho no es 75 porque $a \times b$ causa un desbordamiento.

A juzgar por estos ejemplos, podríamos concluir que si bien las computadoras son dispositivos de propósito general, su naturaleza finita los hace poco apropiados para la aritmética. Claro que esta conclusión no es correcta, pero sirve para ilustrar la importancia de entender cómo funcionan las computadoras y qué limitaciones tienen.

A.2 SISTEMAS NUMÉRICOS CON BASE

Un número decimal ordinario como los que todos conocemos consiste en una serie de dígitos decimales y, posiblemente, un punto decimal. La forma general y su interpretación usual se muestran en la figura A-1. Se tomó la decisión de usar 10 como **base** para la exponenciación porque estamos usando números decimales, es decir, base 10. Al tratar con computadoras, a menudo es mejor utilizar bases distintas de 10. Las bases más importantes son 2, 8 y 16. Los sistemas de numeración basados en estas bases se llaman **binario**, **octal** y **hexadecimal**, respectivamente.

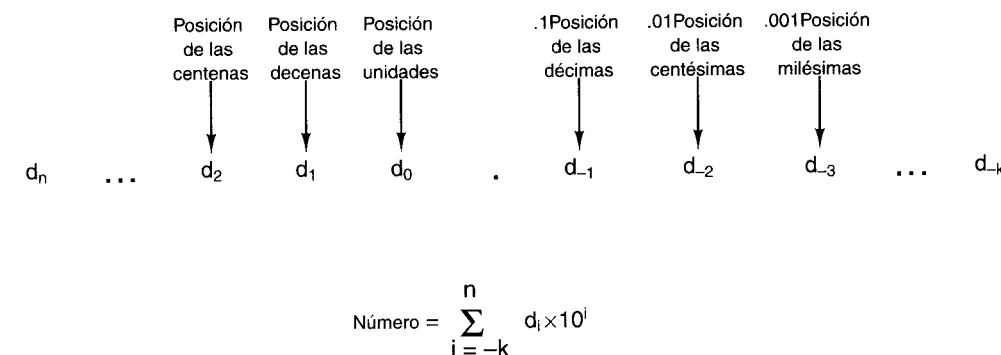


Figura A-1. Forma general de un número decimal.

Un sistema de numeración base k requiere k símbolos distintos para representar los dígitos del 0 a $k - 1$. Los números decimales se representan con los diez dígitos decimales

0 1 2 3 4 5 6 7 8 9

En contraste, los números binarios no usan estos diez dígitos; todos se representan exclusivamente con los dos dígitos binarios

0 1

Los números octales se representan con los ocho dígitos octales

0 1 2 3 4 5 6 7

En el caso de los números hexadecimales, se necesitan 16 dígitos. Por tanto requerimos seis símbolos nuevos. La convención es usar las letras mayúsculas de la A a la F para los seis dígitos que siguen al 9. Así, los números hexadecimales se representan con los dígitos

0 1 2 3 4 5 6 7 8 9 A B C D E F

La expresión “dígito binario”, que se refiere al 1 o al 0, dio origen al término **bit** (*Binary digit*). La figura A-2 muestra el número decimal 2001 expresado en forma binaria, octal y hexadecimal. El número 7B9 obviamente es hexadecimal, porque el número B sólo puede ocurrir en números hexadecimales. Sin embargo, el número 111 podría estar en cualquiera de los cuatro sistemas de numeración que hemos mencionado. A fin de evitar la ambigüedad, se usa un subíndice de 2, 8, 10 o 16 para indicar la base cuando no es obvio por el contexto.

Binario	1	1	1	1	1	0	1	0	0	0	1
	$1 \times 2^{10} + 1 \times 2^9 + 1 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$										
	1024	+ 512	+ 256	+ 128	+ 64	+ 0	+ 16	+ 0	+ 0	+ 0	+ 1
Octal	3	7	2	1							
	$3 \times 8^3 + 7 \times 8^2 + 2 \times 8^1 + 1 \times 8^0$										
	1536	+ 448	+ 16	+ 1							
Decimal	2	0	0	1							
	$2 \times 10^3 + 0 \times 10^2 + 0 \times 10^1 + 1 \times 10^0$										
	2000	+ 0	+ 0	+ 1							
Hexadecimal	7	D	1	.							
	$7 \times 16^2 + 13 \times 16^1 + 1 \times 16^0$										
	1792	+ 208	+ 1								

Figura A-2. El número 2001 en binario, octal y hexadecimal.

Como ejemplo de la notación binaria, octal, decimal y hexadecimal, considere la figura A-3, que muestra una serie de enteros no negativos expresados en cada uno de estos cuatro sistemas. Tal vez algún arqueólogo miles de años en el futuro descubrirá esta tabla y la usará como Piedra Roseta para descifrar los sistemas de numeración de fines del siglo xx y principios del siglo xxi.

Decimal	Binario	Octal	Hex
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
20	10100	24	14
30	11110	36	1E
40	101000	50	28
50	110010	62	32
60	111100	74	3C
70	1000110	106	46
80	1010000	120	50
90	1011010	132	5A
100	11001000	144	64
1000	1111101000	1750	3E8
2989	101110101101	5665	BA

Figura A-3. Números decimales y sus equivalentes binarios, octales y hexadecimales.

A.3 CONVERSIÓN DE UNA BASE A OTRA

La conversión entre números octales o hexadecimales y números binarios es fácil. Para convertir un número binario en octal, basta con dividirlo en grupos de tres bits, de modo que los tres bits que están inmediatamente a la izquierda (o derecha) del punto decimal (en este caso llamado punto binario) formen un grupo, los tres bits que siguen a la izquierda formen otro grupo y así. Cada grupo de 3 bits se puede convertir directamente en un solo dígito octal, del 0 al 7, según la conversión dada en las primeras ocho líneas de la figura A-3. Podría ser necesario añadir uno o dos ceros a la izquierda o a la derecha para completar un grupo de 3 bits. La conversión de octal a binario es igualmente trivial. Cada dígito octal simplemente se sustituye por el número binario de tres bits equivalente. La conversión de hexadecimal a

binario es en lo esencial igual a la de octal a binario, sólo que cada dígito hexadecimal corresponde a un grupo de 4 bits, en lugar de 3 bits. La figura A-4 da algunos ejemplos de conversiones.

Ejemplo 1

Hexadecimal	1	9	4	8	.	B	6
Binario	0001	1001	0100	1000	.	1011	0110
Octal	1	4	5	1	0	5	4

Ejemplo 2

Hexadecimal	7	B	A	3	.	B	C	4
Binario	0111	1011	1010	0011	.	1011	1100	0100
Octal	7	5	6	4	3	5	7	0

Figura A-4. Ejemplos de conversión de octal a binario y de hexadecimal a binario.

La conversión de números decimales a binarios puede efectuarse de dos formas distintas. El primer método es consecuencia directa de la definición de números binarios. Se resta al número la potencia de 2 menor que el número. Luego se repite el proceso con la diferencia. Una vez que el número se ha descompuesto en potencias de 2, el número binario puede armarse con unos en las posiciones de bit que corresponden a potencias de 2 usadas en la descomposición, y ceros en las demás posiciones.

El otro método (sólo para enteros) consiste en dividir el número entre 2. El cociente se escribe directamente debajo del número original y el residuo, 0 o 1, se escribe junto al cociente. Luego se considera el cociente y se repite el proceso hasta llegar a un cociente de 0. El resultado de este proceso es dos columnas de números, los cocientes y los residuos. Ahora el número binario puede leerse directamente de la columna de los residuos comenzando desde abajo. La figura A-5 muestra un ejemplo de conversión de decimal a binario.

Los enteros binarios también pueden convertirse a decimal de dos maneras. Un método consiste en obtener la sumatoria de las potencias de 2 que corresponden a los bits 1 del número. Por ejemplo,

$10110 \text{ es } 2^4 + 2^2 + 2^1 = 16 + 4 + 2 = 22$

En el otro método, el número binario se escribe verticalmente, con un bit en cada línea, con el bit de la extrema izquierda hasta abajo. La línea de hasta abajo se llama línea 1, la que está arriba, línea 2, etc. El número decimal se construye en una columna paralela junto al número binario. Comenzamos escribiendo un 1 en la línea 1. La entrada de la línea *n* corresponde a dos veces la entrada de la línea *n* - 1 más el bit que está en la línea *n* (0 o 1). La entrada de la línea de hasta arriba es la respuesta. En la figura A-6 se da un ejemplo de este método de conversión de binario a decimal.

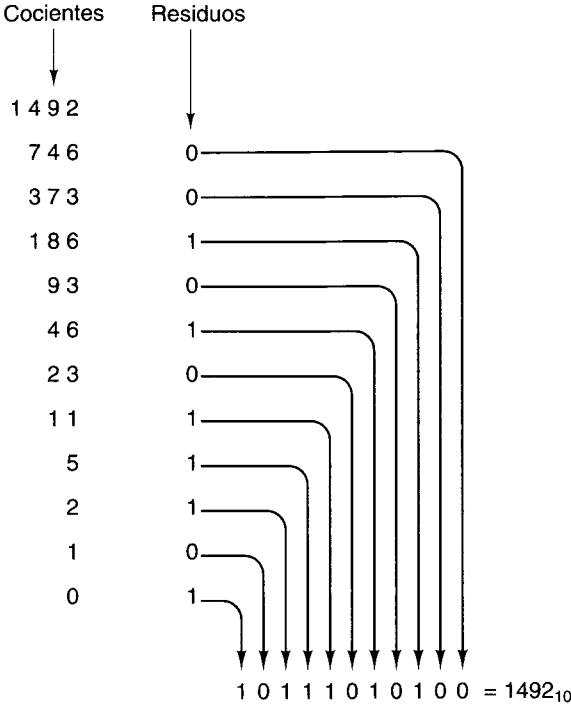


Figura A-5. Conversión del número decimal 1492 a binario por división sucesiva entre 2, comenzando desde arriba y trabajando hacia abajo. Por ejemplo, 93 dividido entre 2 da un cociente de 46 y un residuo de 1, que se escribe en la línea de abajo.

La conversión de decimal a octal y de decimal a hexadecimal puede efectuarse convirtiendo primero a binario y luego al sistema deseado, o restando potencias de 8 o de 16.

A.4 NÚMEROS BINARIOS NEGATIVOS

Se han usado cuatro sistemas distintos para representar números negativos en las computadoras digitales en un momento u otro de la historia. El primero se llama **magnitud con signo**. En este sistema el bit de la extrema izquierda es el bit de signo (0 es + y 1 es -) y los bits restantes contienen la magnitud absoluta del número.

El segundo sistema, llamado **complemento a uno**, también tiene un bit de signo, y se usa 0 para el signo positivo y 1 para el signo negativo. Para negar un número, sustituya cada 1 por 0 y cada 0 por 1. Esto también se hace con el bit de signo. El complemento a uno ya es obsoleto.

El tercer sistema, llamado **complemento a dos**, también tiene un bit de signo que es 0 para el signo positivo y 1 para el signo negativo. La negación de un número es un proceso de dos pasos. Primero, cada 1 se sustituye por un 0 y cada 0 por un 1, igual que en el complemento a 1. Luego se suma 1 al resultado. La suma binaria es igual a la suma decimal, excepto

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

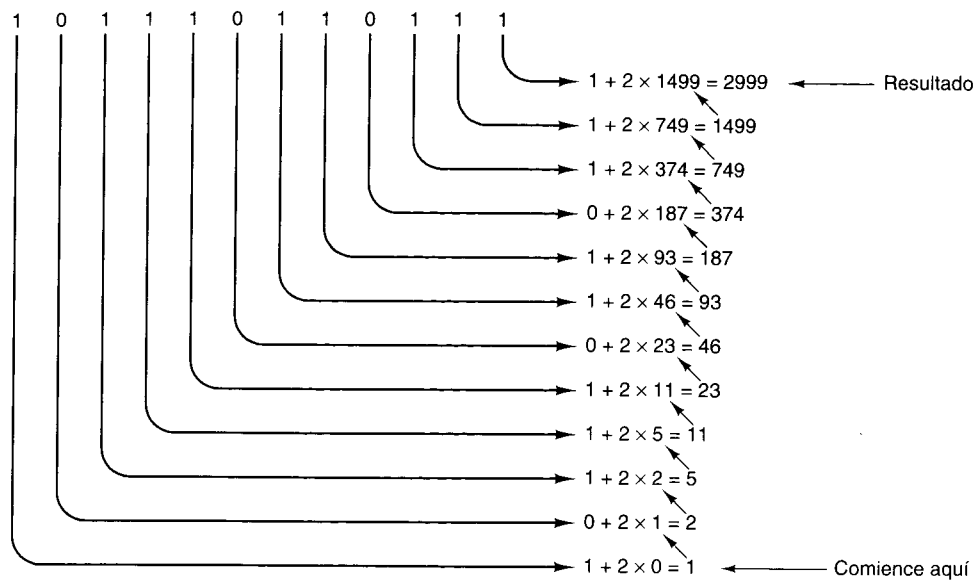


Figura A-6. Conversión del número binario 101110110111 a decimal por duplicación sucesiva, comenzando hasta abajo. Cada línea se forma duplicando la de abajo y sumándole el bit correspondiente. Por ejemplo, 749 es dos veces 374 más el bit 1 que está en la misma línea que 749.

que se genera un acarreo si la suma es mayor que 1, no si la suma es mayor que 9. Por ejemplo, la conversión de 6 a complemento a dos se efectúa en dos pasos:

00000110 (+6)
11111001 (−6 en complemento a uno)
11111010 (−6 en complemento a dos)

Si hay un acarreo hacia la izquierda del bit de la extrema izquierda, se desecha.

El cuarto sistema, que para números de m bits se llama **exceso de 2^{m-1}** , representa un número almacenándolo como la suma del mismo número y 2^{m-1} . Por ejemplo, en el caso de números de 8 bits, $m = 8$, el sistema se llama exceso de 128, y un número se almacena como su valor verdadero más 128. Así, -3 se convierte en $-3 + 128 = 125$, y -3 se representa con el número binario de 8 bits que corresponde a 125 (01111101). Los números de -128 a $+127$ se transforman en los números de 0 a 255, todos los cuales pueden expresarse como un entero positivo de 8 bits. Resulta interesante que este sistema es idéntico al de complemento a dos pero con el bit de signo invertido. La figura A-7 proporciona ejemplos de números negativos en los cuatro sistemas.

Los sistemas tanto de magnitud con signo como de complemento a uno tienen dos representaciones para el cero: un cero positivo y un cero negativo. Esta situación es indeseable. El sistema de complemento a dos no tiene este problema porque el complemento a dos de más cero también es más cero. Sin embargo, este sistema tiene una singularidad distinta. El patrón de bits que consiste en un 1 seguido de puros ceros es su propio complemento. El resultado es

N decimal	N binario	−N mag. c/signo	−N comp. a 1	−N comp. a 2	−N exceso en 128
1	00000001	10000001	11111110	11111111	01111111
2	00000010	10000010	11111101	11111110	01111110
3	00000011	10000011	11111100	11111101	01111101
4	00000100	10000100	11111011	11111100	01111100
5	00000101	10000101	11111010	11111011	01111011
6	00000110	10000110	11111001	11111010	01111010
7	00000111	10000111	11111000	11111001	01111001
8	00001000	10001000	11110111	11111000	01111000
9	00001001	10001001	11110110	11110111	01110111
10	00001010	10001010	11110101	11110110	01110110
20	00010100	10010100	11101011	11101100	01101100
30	00011110	10011110	11100001	11100010	01100010
40	00101000	10101000	11010111	11011000	01011000
50	00110010	10110010	11001101	11001110	01001110
60	00111100	10111100	11000011	11000100	01000100
70	01000110	11000110	10111001	10111010	00111010
80	01010000	11010000	10101111	10110000	00110000
90	01011010	11011010	10100101	10100110	00100110
100	01100100	11011010	10011011	10011100	00011100
127	01111111	11111111	10000000	10000001	00000001
128	No existe	No existe	No existe	10000000	00000000

Figura A-7. Números negativos de 8 bits en cuatro sistemas.

que la gama de números positivos y negativos es asimétrica; hay un número negativo que no tiene contraparte positiva.

La razón de estos problemas no es difícil de encontrar: queremos un sistema de codificación que tenga dos propiedades:

1. Sólo una representación para el cero.
2. Exactamente tantos números positivos como negativos.

El problema es que cualquier conjunto de números que tenga tantos números positivos como negativos y sólo un cero tendrá un número impar de miembros, mientras que m bits dan origen a un número par de patrones de bits. Siempre habrá un patrón de bits de más o de

menos, sea cual sea la representación que se escoja. Este patrón de bits extra se puede usar para -0 o para un número negativo grande, o para algo más, pero sea cual sea el uso que se le dé, siempre será un problema.

A.5 ARITMÉTICA BINARIA

La tabla de la suma para números binarios se da en la figura A-8.

Sumando	0	0	1	1
Sumando	<u>+0</u>	<u>+1</u>	<u>+0</u>	<u>+1</u>
Suma	0	1	1	0
Acarreo	0	0	0	0

Figura A-8. Tabla de suma en binario.

Dos números binarios pueden sumarse comenzando en el bit de la extrema derecha y sumando los bits correspondientes de los dos sumandos. Si se genera un acarreo, se lleva una posición a la izquierda, igual que en la aritmética decimal. En aritmética de complemento a uno, un acarreo generado por la suma de los bits de la extrema izquierda se suma al bit de la extrema derecha. Este proceso se llama acarreo al otro extremo. En aritmética de complemento a dos, un acarreo generado por la suma de los bits de la extrema izquierda simplemente se desecha. En la figura A-9 se muestran ejemplos de aritmética binaria.

Decimal	Complemento a 1	Complemento a 2
10 + (-3)	00001010 11111100	00001010 11111101
+7	1 00000110 ↘ acarreo 1 00000111	1 00000111 ↓ se desecha

Figura A-9. Suma en complemento a uno y en complemento a dos.

Si los sumandos tienen signos opuestos, no puede haber un error de desbordamiento. Si tienen el mismo signo y el resultado tiene el signo opuesto, ha ocurrido un error de desbordamiento y la respuesta no es correcta. En aritmética tanto de complemento a uno como de complemento a dos, ocurre un desbordamiento si y sólo si el acarreo hacia el bit de signo difiere del acarreo desde el bit de signo. Casi todas las computadoras conservan el acarreo desde el bit de signo, pero el acarreo hacia el bit de signo no es visible cuando se examina la respuesta. Es por esto que casi siempre se incluye un bit de desbordamiento especial.

PROBLEMAS

1. Convierta los siguientes números a binario: 1984, 4000, 8192.
2. ¿Qué es 1001101001 (binario) en decimal? ¿En octal? ¿En hexadecimal?
3. ¿Cuáles de los siguientes son números hexadecimales válidos? BED, CAB, DEAD, DECADE, ACCEDED, BAG, DAD.
4. Expresé el número decimal 100 en todas las bases de 2 a 9.
5. ¿Cuántos enteros positivos distintos se pueden expresar con k dígitos empleando números de base r ?
6. Casi todos nosotros sólo podemos contar hasta 10 con los dedos; sin embargo, los especialistas en computación pueden contar más allá del 10. Si consideramos cada dedo como un bit binario, y un dedo extendido representa 1 mientras que uno doblado representa 0, ¿hasta qué número podemos contar usando ambas manos? ¿Y usando también ambos pies? Ahora use ambas manos y ambos pies, pero el dedo gordo de su pie izquierdo hará las veces de bit de signo para números en complemento a dos. ¿Qué intervalo de números podemos expresar?
7. Realice los siguientes cálculos con números de 8 bits en complemento a dos:

$$\begin{array}{r} 00101101 \\ +01101111 \\ \hline \end{array} \quad \begin{array}{r} 11111111 \\ +11111111 \\ \hline \end{array} \quad \begin{array}{r} 00000000 \\ -11111111 \\ \hline \end{array} \quad \begin{array}{r} 11110111 \\ -11110111 \\ \hline \end{array}$$

8. Repita el cálculo del problema anterior pero ahora en complemento a uno.
9. Considere los siguientes problemas de suma para números binarios de tres bits en complemento a dos. Para cada suma, indique
 - a. Si el bit de signo del resultado es 1.
 - b. Si los tres bits de orden bajo son 0.
 - c. Si ocurrió un desbordamiento.

$$\begin{array}{r} 000 \\ +001 \\ \hline \end{array} \quad \begin{array}{r} 000 \\ +111 \\ \hline \end{array} \quad \begin{array}{r} 111 \\ +110 \\ \hline \end{array} \quad \begin{array}{r} 100 \\ +111 \\ \hline \end{array} \quad \begin{array}{r} 100 \\ +100 \\ \hline \end{array}$$

10. Los números decimales con signo que constan de n dígitos se pueden representar con $n + 1$ dígitos sin usar un signo. Los números positivos tienen 0 como dígito de la izquierda. Los números negativos se forman restando cada dígito de 9. Así, el negativo de 014725 es 985274. Tales números se llaman números en complemento a nueve y son análogos a los números binarios en complemento a uno. Expresé los siguientes como números de tres dígitos en complemento a nueve: 6, -2 , 100, -14 , -1 , 0.
11. Determine la regla para la suma de números en complemento a nueve y luego realice las siguientes sumas.

$$\begin{array}{r} 0001 \\ +9999 \\ \hline \end{array} \quad \begin{array}{r} 0001 \\ +9998 \\ \hline \end{array} \quad \begin{array}{r} 9997 \\ +9996 \\ \hline \end{array} \quad \begin{array}{r} 9241 \\ +0802 \\ \hline \end{array}$$

12. El complemento a 10 es análogo al complemento a dos. Un número negativo en complemento a 10 se forma sumando 1 al número en complemento a nueve correspondiente, desechando el acarreo final. ¿Cuál es la regla para la suma en complemento a 10?

B.1 PRINCIPIOS DE PUNTO FLOTANTE

Una forma de separar el intervalo y la precisión es expresar los números en la conocida notación científica

$$n = f \times 10^e$$

donde f es la **fracción** o **mantisa** y e es un entero positivo o negativo llamado **exponente**. La versión para computadora de esta notación se llama **punto flotante**. He aquí algunos ejemplos de números expresados en esta forma:

$$\begin{aligned} 3.14 &= 0.314 \times 10^1 = 3.14 \times 10^0 \\ 0.000001 &= 0.1 \times 10^{-5} = 1.0 \times 10^{-6} \\ 1941 &= 0.1941 \times 10^4 = 1.941 \times 10^3 \end{aligned}$$

El intervalo está determinado por el número de dígitos del exponente, y la precisión está determinada por el número de dígitos de la fracción. Puesto que hay más de una forma de representar un número dado, normalmente se escoge una forma como estándar. A fin de investigar las propiedades de este método de representación de los números, consideremos una representación, R , con una fracción de tres dígitos y signo dentro del intervalo $0.1 \leq |f| < 1$ o cero, y un exponente de dos dígitos con signo. La magnitud de estos números varía entre $+0.100 \times 10^{-99}$ y 0.999×10^{99} , un intervalo de casi 199 órdenes de magnitud, y sin embargo sólo se necesitan cinco dígitos y dos signos para almacenar un número.

Podemos usar números de punto flotante para modelar el sistema de números reales de las matemáticas, aunque hay varias diferencias importantes. La figura B-1 muestra un esquema muy exagerado de la línea de los números reales. La línea real se divide en siete regiones:

1. Números negativos grandes menores que -0.999×10^{99} .
2. Números negativos entre -0.999×10^{99} y -0.100×10^{-99} .
3. Números negativos pequeños con una magnitud menor que 0.100×10^{-99} .
4. Cero.
5. Números positivos pequeños con una magnitud menor que 0.100×10^{-99} .
6. Números positivos entre 0.100×10^{-99} y 0.999×10^{99} .
7. Números positivos grandes mayores que 0.999×10^{99} .

Una diferencia importante entre el conjunto de los números que pueden representarse con tres dígitos en la fracción y dos en el exponente, y los números reales, es que los primeros no se pueden usar para expresar números en las regiones 1, 3, 5 o 7. Si el resultado de una operación aritmética es un número en las regiones 1 o 7 —por ejemplo, $10^{60} \times 10^{60} = 10^{120}$ — ocurrirá un **error de desbordamiento** y la respuesta será incorrecta. La razón está en la naturaleza finita de la representación de los números y es inevitable. Así mismo, un resultado

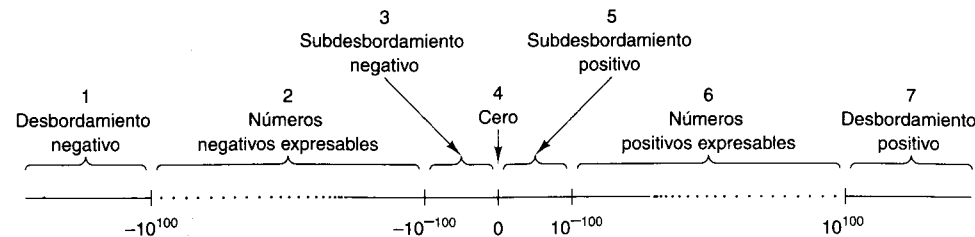


Figura B-1. La línea de los números reales se puede dividir en siete regiones.

en la región 3 o 5 tampoco puede expresarse. Esta situación se llama **error de subdesbordamiento**. Este tipo de error es menos grave que el de desbordamiento, porque en muchos casos 0 es una aproximación satisfactoria para números de las regiones 3 y 5. Un saldo en el banco de 10^{-102} dólares no es mucho mejor que un saldo de 0.

Otra diferencia importante entre los números de punto flotante y los números reales es su densidad. Entre cualesquier dos números reales x y y hay otro número real, por más cercano que esté x a y . Esta propiedad proviene del hecho de que para cualesquier números reales distintos, x y y , $z = (x + y)/2$ es un número real que está entre ellos. Los números reales forman un continuo.

Los números de punto flotante, en cambio, no forman un continuo. Se pueden expresar exactamente 179,100 números positivos en el sistema de cinco dígitos y dos signos que usamos antes, 179,100 números negativos y 0 (que se puede expresar de muchas formas), para un total de 358,201 números. Del número infinito de números reales que hay entre -10^{+100} y $+0.999 \times 10^{99}$, sólo 358,201 se pueden especificar con esta notación. Dichos números se simbolizan con los puntos de la figura B-1. Es muy posible que el resultado de un cálculo sea uno de los otros números, aunque esté en la región 2 o 6. Por ejemplo, $+0.100 \times 10^3$ dividido entre 3 no se puede expresar *con exactitud* en nuestro sistema. Si el resultado de un cálculo no se puede expresar en la representación numérica que se está usando, lo que obviamente debe hacerse es usar el número más cercano que sí pueda expresarse. Este proceso se llama **redondeo**.

La distancia entre números adyacentes que pueden expresarse no es constante a lo largo de las regiones 2 o 6. La separación entre $+0.998 \times 10^{99}$ y $+0.999 \times 10^{99}$ es inmensamente mayor que la distancia entre $+0.998 \times 10^0$ y $+0.999 \times 10^0$. Sin embargo, si expresamos la distancia entre un número y su sucesor como un porcentaje del número, no hay variación sistemática dentro de la región 2 o 6. En otras palabras, el **error relativo** introducido por el redondeo es aproximadamente el mismo para números pequeños que para números grandes.

Aunque la explicación anterior se refirió a un sistema de representación con una fracción de tres dígitos y un exponente de dos dígitos, las conclusiones que se sacan son válidas también para otros sistemas de representación. Modificar el número de dígitos de la fracción o exponente simplemente desplaza los límites de las regiones 2 y 6, y cambia el número de puntos expresables y que aquéllas contienen. Incrementar el número de dígitos en la fracción aumenta la densidad de puntos y por tanto mejora la precisión de las aproximaciones. In-

crementar el número de dígitos del exponente aumenta el tamaño de las regiones 2 y 6 al encoger las regiones 1, 3, 5 y 7. En la figura B-2 se muestran las fronteras aproximadas de la región 6 para números decimales de punto flotante con diversos tamaños de la fracción y el exponente.

Dígitos en la fracción	Dígitos en el exponente	Cota inferior	Cota superior
3	1	10^{-12}	10^9
3	2	10^{-102}	10^{99}
3	3	10^{-1002}	10^{999}
3	4	10^{-10002}	10^{9999}
4	1	10^{-13}	10^9
4	2	10^{-103}	10^{99}
4	3	10^{-1003}	10^{999}
4	4	10^{-10003}	10^{9999}
5	1	10^{-14}	10^9
5	2	10^{-104}	10^{99}
5	3	10^{-1004}	10^{999}
5	4	10^{-10004}	10^{9999}
10	3	10^{-1009}	10^{999}
20	3	10^{-1019}	10^{999}

Figura B-2. Límites superior e inferior aproximados de los números decimales de punto flotantes expresables (no normalizados).

En las computadoras se usa una variación de esta representación. Por eficiencia, la exponenciación es base 2, 4, 8 o 16, no 10, y entonces la fracción consiste en una cadena de dígitos binarios, base 4, octales o hexadecimales. Si el primero de esos dígitos a la izquierda es cero, todos los dígitos pueden desplazarse una posición a la izquierda y el exponente reducirse en 1, sin alterar el valor del número (siempre que no haya subdesbordamiento). Se dice que una fracción cuyo dígito de la izquierda no es cero está **normalizada**.

Los números normalizados generalmente son preferibles a los no normalizados porque sólo hay una representación normalizada, mientras que hay muchas representaciones no normalizadas. En la figura B-3 se dan ejemplos de números de punto flotante normalizados para dos bases de exponenciación. En estos ejemplos se muestra una fracción de 16 bits (incluido el bit de signo) y un exponente de siete bits empleando notación exceso de 64. El punto base está a la izquierda del bit de la extrema izquierda de la fracción, es decir, a la derecha del exponente.

B.2 ESTÁNDAR DE PUNTO FLOTANTE IEEE 754

Hasta cerca de 1980, cada fabricante de computadoras tenía su propio formato de punto flotante. Sobre decir que todos eran diferentes. Peor aún, algunos de ellos efectuaban aritmética incorrecta porque la de punto flotante tiene algunas sutilezas que no son obvias para el diseñador de hardware ordinario.

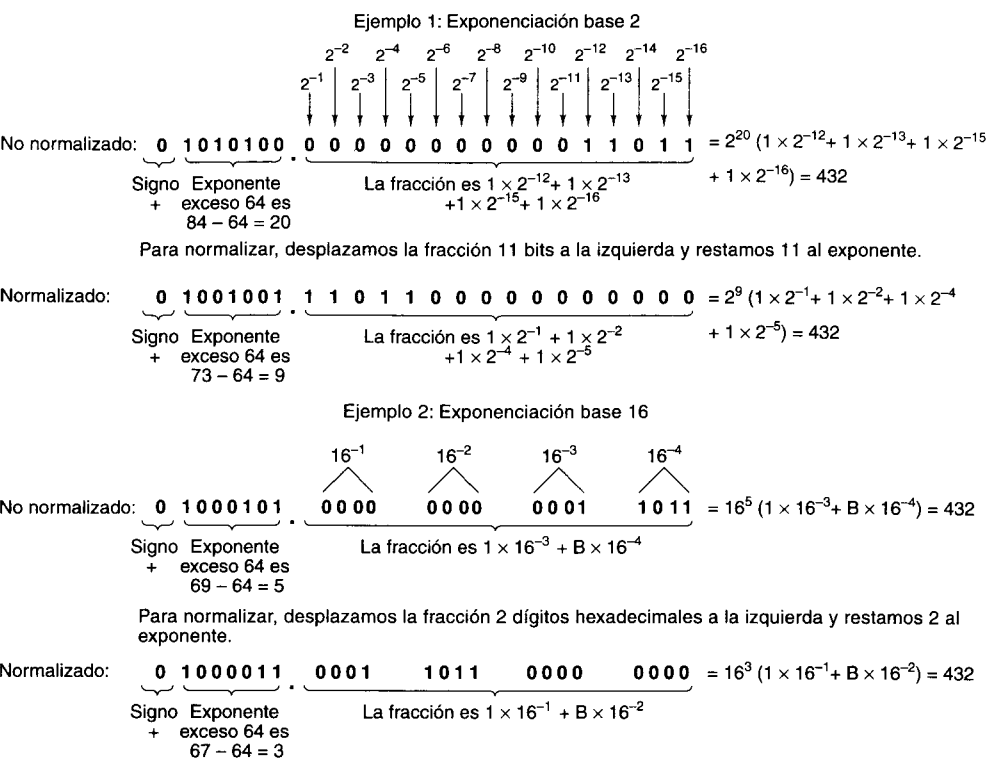


Figura B-3. Ejemplos de números de punto flotante normalizados.

A fin de rectificar esta situación, a fines de los años setenta el IEEE formó un comité para estandarizar la aritmética de punto flotante. La meta no sólo era poder intercambiar datos de punto flotante entre diferentes computadoras, sino también proporcionar a los diseñadores de hardware un diseño que se sabía era correcto. Los trabajos resultantes dieron pie al estándar IEEE 754 (IEEE, 1985). Casi todas las CPU actuales (incluidas las Intel, SPARC y JVM que estudiamos en este libro) tienen instrucciones de punto flotante que se ajustan al estándar de punto flotante IEEE. A diferencia de muchos estándares, que tienden a ser términos medios con los que nadie está contento, éste no está nada mal, en parte porque fue en gran medida el trabajo de una sola persona, el profesor de matemáticas de Berkeley William Kahan. En el resto de la sección describiremos el estándar.

El estándar define tres formatos: precisión sencilla (32 bits), doble precisión (64 bits) y precisión extendida (80 bits). El formato de precisión extendida pretende reducir los errores de redondeo; se usa primordialmente dentro de unidades aritméticas de punto flotante, por lo que no hablaremos más de él. Los formatos tanto de precisión sencilla como doble usan la base 2 para las fracciones y notación en exceso para los exponentes. Los formatos se muestran en la figura B-4.

Ambos formatos comienzan con un bit de signo para el número en su totalidad: 0 para positivo y 1 para negativo. Luego viene el exponente, que usa exceso en 127 en el caso de la

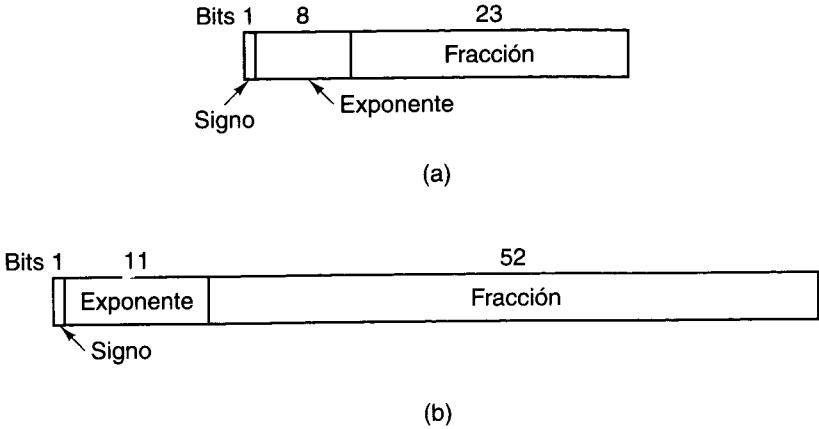


Figura B-4. Formatos de punto flotante IEEE. (a) Precisión sencilla. (b) Doble precisión.

precisión sencilla y exceso en 1023 para la doble precisión. Los exponentes mínimo (0) y máximo (255 y 2047) no se usan con números normalizados; tienen usos especiales que se describen más adelante. Por último, tenemos las fracciones, de 23 y 52 bits, respectivamente.

Una fracción normalizada comienza con un punto binario, seguido de un bit 1, y luego el resto de la fracción. Siguiendo una práctica iniciada en el PDP-11, los autores del estándar se dieron cuenta de que el bit inicial de la fracción no tiene que almacenarse, ya que puede simplemente darse por hecho que está presente. Por tanto, el estándar define la fracción de una forma un poco distinta a lo acostumbrado. La fracción consiste en un bit 1 implícito, un punto binario implícito, y 23 o 52 bits arbitrarios. Si todos los bits de la fracción son ceros, la fracción tiene el valor numérico 1.0; si todos son unos, la fracción es numéricamente un poco menos que 2.0. A fin de evitar confusiones con una fracción convencional, la combinación del 1 implícito, el punto binario implícito y los 23 o 52 bits explícitos se llama **significando** en vez de fracción o mantisa. Todos los números normalizados tienen un significando, s , dentro del intervalo $1 \leq s < 2$.

Las características numéricas de los números de punto flotante IEEE se dan en la figura B-5. Consideremos, por ejemplo, los números 0.5, 1 y 1.5 en formato de precisión sencilla normalizado. Éstos se representan en hexadecimal como 3F000000, 3F800000 y 3FC00000, respectivamente.

Uno de los problemas tradicionales con los números de punto flotante es cómo manejar el subdesbordamiento, el desbordamiento y los números no inicializados. El estándar IEEE aborda los problemas explícitamente, tomando su enfoque en parte de la CDC 6600. Además de los números normalizados, el estándar tiene otros cuatro tipos numéricos, que se describen a continuación y se muestran en la figura B-6.

Surge un problema cuando el resultado de un cálculo tiene una magnitud menor que el número de punto flotante normalizado más pequeño que puede representarse en el sistema. Antes, casi todo el hardware adoptaba uno de dos enfoques: igualar el resultado a cero y continuar, o causar una trampa de desbordamiento de punto flotante hacia cero. Ninguno de

Concepto	Precisión sencilla	Doble precisión
Bits del signo	1	1
Bits del exponente	8	11
Bits de la fracción	23	52
Total de bits	32	64
Sistema de exponente	Exceso en 127	Exceso en 1023
Intervalo del exponente	-126 a +127	-1022 a + 1023
Número normalizado más pequeño	2^{-126}	2^{-1022}
Número normalizado más grande	aprox. 2^{128}	aprox. 2^{1024}
Intervalo decimal	aprox. 10^{-38} a 10^{38}	aprox. 10^{-308} a 10^{308}
Número desnormalizado más pequeño	aprox. 10^{-45}	aprox. 10^{-324}

Figura B-5. Características de los números de punto flotante IEEE.

Normalizado	±	0 < Exp < Máx	Cualquier patrón de bits
Desnormalizado	±	0	Cualquier patrón de bits distinto de cero
Cero	±	0	0
Infinito	±	1 1 1...1	0
Ningún número	±	1 1 1...1	Cualquier patrón de bits distinto de cero

Bit de signo

Figura B-6. Representaciones numéricas IEEE.

éstos es realmente satisfactorio, por lo que IEEE inventó los **números desnormalizados**. Estos números tienen un exponente cero y una fracción dada por los siguientes 23 o 52 bits. El bit implícito a la izquierda del punto binario ahora se convierte en 0. Los números desnormalizados se pueden distinguir de los normalizados porque estos últimos no pueden tener un exponente cero.

El número de precisión sencilla normalizado más pequeño tiene 1 como exponente y 0 como fracción, y representa 1.0×2^{-126} . El número desnormalizado más grande tiene 0 como exponente y sólo unos en la fracción, y representa cerca de $0.9999999 \times 2^{-127}$, que es casi la misma cosa. Sin embargo, una cosa que debemos tener presente es que este número sólo tiene 23 bits de significancia, en vez de 24 como todos los números normalizados.

Si los cálculos reducen aún más este resultado, el exponente sigue siendo 0, pero los primeros bits de la fracción se van convirtiendo en ceros, lo que reduce tanto el valor como el número de bits significativos de la fracción. El número desnormalizado más pequeño distinto

de cero consiste en un 1 en el bit de la extrema derecha, con todos los demás 0. El exponente representa 2^{-127} y la fracción representa 2^{-23} , de modo que el valor es 2^{-150} . Este esquema hace posible un subdesbordamiento “gradual”, sacrificando significancia en lugar de saltar a 0 cuando el resultado no puede expresarse como número normalizado.

Este esquema cuenta con dos ceros, positivo y negativo, determinados por el bit de signo. Ambos tienen un exponente de 0 y una fracción de 0. Aquí, también, el bit a la izquierda del punto binario es implícitamente 0 en lugar de 1.

El desbordamiento no puede manejarse de forma gradual. No quedan más combinaciones de bits. En vez de ello, se proporciona una representación especial para infinito, que consiste en un exponente solamente de unos (lo cual no está permitido en los números normalizados) y una fracción de 0. Este número puede usarse como operando y se comporta según las reglas matemáticas usuales para el infinito. Por ejemplo, infinito más cualquier cosa es infinito, y cualquier número finito dividido entre infinito es cero. Así mismo, cualquier número finito dividido entre cero da infinito.

¿Y qué pasa con infinito dividido entre infinito? El resultado no está definido. Para manejar este caso se proporciona otro formato especial, llamado **ningún número** (NaN, *Not a Number*), que también puede usarse como operando con resultados predecibles.

PROBLEMAS

1. Convierta los números siguientes al formato IEEE de precisión sencilla. Represente los resultados como ocho dígitos hexadecimales.
 - a. 9
 - b. 5/32
 - c. -5/32
 - d. 6.125
2. Convierta los siguientes números de punto flotante IEEE de precisión sencilla de hexadecimal a decimal:
 - a. 42E48000H
 - b. 3F880000H
 - c. 00800000H
 - d. C7F00000H
3. El formato de los números de punto flotante de precisión sencilla en la 370 tiene un exponente de 7 bits en el sistema exceso de 64, y una fracción que contiene 24 bits más un bit de signo, con el punto binario en el extremo izquierdo de la fracción. La base de exponenciación es 16. El orden de los campos es bit de signo, exponente, fracción. Expresé el número 7/64 como número normalizado en este sistema, en hexadecimal.

4. Los siguientes números binarios de punto flotante consisten en un bit de signo, un exponente base 2 en exceso de 64 y una fracción de 16 bits. Normalícelos.
 - a. 0 1000000 0001010100000001
 - b. 0 0111111 0000001111111111
 - c. 0 1000011 1000000000000000
5. Para sumar dos números de punto flotante hay que ajustar los exponentes (desplazando la fracción) de modo que sean iguales. Luego se pueden sumar las fracciones y normalizar el resultado, si es necesario. Sume los números IEEE de precisión sencilla 3EE00000H y 3D800000H y exprese el resultado normalizado en hexadecimal.
6. La Compañía de Computadoras Económicas decidió sacar una máquina que maneje números de punto flotante de 16 bits. El Modelo 0.001 tiene un formato de punto flotante con un bit de signo, un exponente de 7 bits en exceso de 64, y una fracción de 8 bits. El Modelo 0.002 tiene un bit de signo, un exponente de 5 bits en exceso de 16, y una fracción de 10 bits. Ambas usan exponenciación base 2. ¿Cuáles son los números normalizados más grande y más pequeño en cada modelo? ¿Cuántos dígitos decimales de precisión tiene aproximadamente cada uno? ¿Compraría alguno de ellos?
7. Hay una situación en la que una operación con dos números de punto flotante puede causar una reducción drástica del número de bits significativos en el resultado. ¿Cuál es?
8. Algunos chips de punto flotante tienen incorporada una instrucción de raíz cuadrada. Un posible algoritmo es iterativo (por ejemplo, el de Newton-Raphson). Los algoritmos iterativos necesitan una aproximación inicial y luego la mejoran continuamente. ¿Cómo se puede obtener una raíz cuadrada aproximada rápida de un número de punto flotante?
9. Escriba un procedimiento que sume dos números de punto flotante IEEE de precisión sencilla. Cada número se representa con un arreglo booleano de 32 elementos.
10. Escriba un procedimiento para sumar dos números de punto flotante de precisión sencilla que usan base 16 para el exponente y base 2 para la fracción pero no tienen un bit 1 implícito a la izquierda del punto binario. Un número normalizado tiene 0001, 0010, ..., 1111 como los cuatro bits de extrema izquierda de la fracción, pero no 0000. Un número se normaliza desplazando la fracción a la izquierda 4 bits y sumando 1 al exponente.

ÍNDICE

A

Acceso

- a memoria sólo por caché, 553, 585-586
- directo a la memoria, 90, 358
- no uniforme a memoria, 553, 573-585
- uniforme a memoria, 552

Acierto en caché, 268

Aciertos, tasa de, 66

ACL (*véase* Lista de control de acceso)

Acontecimientos importantes en arquitectura
de computadoras, 13-24

Actualización, estrategia de, 566

Acumulador, 18, 42, 333

Administración

- de directorios, 435-436
- de procesos
- en UNIX, 470-473
- en Windows NT, 473-476

Administrador

- de caché, en Windows NT, 452
- de E/S, Windows NT, 452
- de memoria virtual, Windows NT, 452
- de objetos, Windows NT, 452

- de procesos, enlaces y transacciones,
Windows NT, 452

de seguridad, Windows NT, 453

Aiken, Howard, 16

Álgebra

- booleana, 120-127
- de conmutación, 120

Algoritmo, 8

de alimentación por demanda, 411

Almacén (*véase también* Memoria)

- de control, 45, 213
- de tareas, 606

Almacenamiento, 56

en línea, 435

fuera de línea, 435

temporal de entrada, 537

ALU (*véase* Unidad aritmética lógica)

Ancho

- de banda agregado, 540
- de banda del procesador, 51
- de bus, 159-160

Applet, 34

Apuntador, 335

a reserva constante, 205, 221, 232

a variable local, 205, 221, 224, 232
 de cuadro, 312
 de pila, 205, 218-221, 224
 Arbitraje de bus, 165-167
 PCI, 186
 Árbitro de bus, 90
 Archivo, 430-431
 exe, 506
 inmediato, 469
 Área de métodos, 221
 Aritmética binaria, 640
 Arquitectura, 7, 44
 acontecimientos importantes, 13-24
 de carga/almacenamiento, 48, 316
 de computadoras, 7
 de conjunto de instrucciones, 6
 de enlace escalable, 32
 del directorio para memoria compartida,
 Harvard, 67
 industrial estándar, 91
 superescalar, 52
 Arreglo
 de procesadores, 53, 554-555
 lógico programable, 132
 Asa, 454
 ASCII, código, 109-110
 Atanasoff, John, 15
 ATM (*véase* Modo de transferencia
 asincrónico)
 Atributos, byte de, 96

B

Babbage, Charles, 13-14
Ball Grid Array, 181
 Banderas, registro de, 310
 Barrera, 551, 600
 Basado-indizado, direccionamiento, 338
 Base
 (*base*), 118
 del sistema numérico, 633-634
 octal, 633
 (*radix*), 633
 Baud, 107
 BCD (*véase* Decimal codificado en binario)
 Bechtolsheim, Andy, 32
 BGA (*véase* Ball Grid Array)
 Biblioteca
 anfitriona, 519

compartida, 519
 de enlace dinámico, 517-518
 de importación, 518
 objetivo, 519
Big endian, 59
 BIOS (*véase* Sistema básico de entrada/salida)
 BIPUSH IJVM, instrucción, 222, 237
 Bisección del ancho de banda, 532
 Bit, 56-57, 634
 de paridad, 61
 de presente/ausente, 409
 venenoso, 283
 Bloque
 básico, 281
 con doble indirección, 464
 de caché, 448
 de triple indirección, 464
 indirecto, 464
 Bloqueo
 de cabeza de línea, 537
 mutuo (Deadlock), 538
 Boole, George, 120
 Bóveda de datos, 582
 Buffer(s)
 circular, 438
 de almacenamiento para traducción, 427-428
 de consulta para traducción, 427
 fallo de TLB, 427
 de prebúsqueda, 49
 de resurtido, 284
 de salida, 537
 inversor, 149
 no inversor, 149
 Burbuja de inversión, 119
 Burroughs B5000, 21
 Bus, 19, 39, 89-90, 156-170
 asincrónico, 160, 163-165
 de interconexión de componentes
 periféricos, 91, 183-189
 señales del, 187-189
 transacciones, 189
 de saludo (*handshaking*), 164
 de transferencia de bloques, 168
 del sistema, 157
 EISA, 183
 esclavo, 158
 IBM PC, 183
 ISA, 181-183
 ISA extendido, 91, 183
 maestro, 158

multiplexado, 160
 PCI, 183-189 (*véase también* Bus de
 interconexión de componentes
 periféricos)
 receptor, 158
 Serie Universal, 189-193
 síncrono, 160-163
 tradicional, 170
 USB, 189-193
 Búsqueda, 71
 binaria, 505
 Byron, Lord, 15
 Byte, 58, 307
 prefijo, 239, 327, 360

C

Caché
 asociativa por conjunto, 269-270
 con mapeo directo, 267-269
 de asignación de escritura, 270, 566
 de escritura diferida, 270
 de escritura inmediata, 270, 565
 de escritura una vez, 568
 de espionaje, 564-569
 de *n* vías, 269
 de reescritura, 270, 568
 dividida, 67, 265
 espía, 564-569
 nivel 2, 265
 unificada, 67
 Cambio de nombre de registros, 280-281
 Capa, 3
 de abstracción del hardware, 451
 Carga especulativa, 395-396
 Cargador de enlace, 506
 Cartucho de una sola arista, 171
 CC-NUMA (*véase* NUMA con caché
 coherente)
 CD
 grabable, 84-86
 reescrible, 86
 -ROM multisesión, 85
 CDC 6600, 14, 20, 21, 52, 277, 545
 CD-ROM, 80-83
 multisesión, 85
 pista de, 84
 sector de, 82
 XA, 84
 Celda de memoria, 57
 Celeron, 30
 Centro, 596
 cómputo, 23
 Cerradura, 632
 Certificado de acceso, 468
 Ciclo
 de bus, 160
 de búsqueda-decodificación-ejecución, 42,
 204
 de la trayectoria de datos, 41
 Cilindro, 71
 Circuito
 combinacional, 129-134
 aritmético, 134-139
 comparador, 131-132
 decodificador, 130-131
 multiplexor, 130-131
 integrado, 128-129
 uso en computadoras, 21-23
 lógico, 128-141
 virtuoso, 25
 CISC (*véase* Computadora con conjunto de
 instrucciones complejo)
 Clave de índice de archivos, 432
 Clon, 24
 Codificación por dispersión (hash coding), 506
 Código(s)
 de caracteres, 109-112
 de condición, 310
 de corrección de errores, 61-64
 de escape, 327
 de operación, 204
 de redundancia cíclica, 193
 independiente de la posición, 515
 Coherencia de caché, 565
 Cola de
 mensajes, 471
 procesamiento U, 51
 de procesamiento V, 51
 Colector abierto, 158
 Color indexado, 97
 COLOSSUS, 16
 COMA
 simple, 586
 (*véase* Acceso a memoria sólo por caché)
 Comparación
 de arquitecturas, 296-298
 y ramificación, instrucciones de, 352-353
 Comparador, 131

- Compatibilidad con modelos anteriores, 304
- Competencia, condición de, 438-442
- Compilador, 7, 484
 - JIT (*véase* Compilador justo a tiempo)
 - justo a tiempo, 35
- Complemento
 - a dos, 637
 - a unos, 637
- Compuerta, 5, 117-127
 - de llamada, 425
- Computadora
 - con conjunto de instrucciones complejo, 46-47
 - con conjunto de instrucciones reducido, 24, 46-53
 - principios de diseño, 47-49
 - versus* CISC, 46-47
- Computadoras paralelas
 - aspectos de diseño, 524-553
 - desempeño de, 539-545
 - software de, 545-546, 598-609
 - taxonomía de, 551-553
- Comunicador, 600
- Conjunto
 - de instrucciones, comparación de, 369-370
 - de instrucciones visuales, 33
 - de tareas, 548
- Conmutación
 - de circuitos, 536
 - de paquetes de almacenar y reenviar, 536
- Conmutador transversal, 569
- Consistencia (*véase también* Semántica de la memoria)
 - de caché, 565
 - débil, 562
 - del procesador, 561
 - en la liberación, 563
 - estricta, 560
 - secuencial, 560
- Constante de reubicación, 509
- Contador
 - de microprograma, 215
 - de programa, 40, 205, 224
 - de ubicación de instrucciones, 499
- Contexto, 427
- Control
 - Data Corporation, 20-21
 - de ciclo, instrucción de, 354-355
- Controlador, 89
 - de disco, 73
- Conversión
 - de base, 635-637
 - entre bases, 635-637
- Copiar al escribir, 456
- Corte virtual por enrutamiento, 537
- Corrutina, 376-379
- Cougar, 592
- COW (*véase* Grupo de estaciones de trabajo)
- CPP (*véase* Apuntador a reserva constante)
- CPU (*véase* Unidad central de procesamiento)
 - chip de, 154-156
- Cray, Seymour, 20-21
- Cray-1, 557-559
- Cray T3E, 589-590
- CRC (*véase* Código de redundancia cíclica)
- Creación de procesos, 437
- CRT (*véase* Tubo de rayos catódicos)
- Cuadrulado, 419
- Cuadro
 - barrido por, 93
 - de CD-ROM, 82
- Cuarta generación de computadoras, 23-24
- Cúmulo, 468
- Chip, 128
 - de CPU, 154-156
 - de E/S en paralelo, 194-195
 - de interfaz de red, 591
 - de memoria, 150-152

D

- DAS (*véase* Supercomputadora ASCI distribuida)
- DASH, multiprocesador, 577-580
- Datos obsoletos, 565
- DEC PDP-1, 19
- DEC PDP-8, 19
- DEC VAX, 45, 46
- Decimal codificado en binario, 57
- Decodificación
 - de direcciones, 195-197
 - parcial de direcciones, 197
- Decodificador, 130
- Demultiplexor, 130
- Dependencia
 - de escritura después de escritura, 278
 - de escritura después de lectura, 278
 - verdadera, 258
- WAR (*véase* Dependencia de escritura después de lectura)

- WAW (*véase* Dependencia de escritura después de escritura)
- Derivación vampiro, 595
- Desbordamiento, error de, 644, 645
- Descriptor
 - de archivo, 459
 - de seguridad, 468
- Desempeño de computadoras en paralelo
 - cómo lograr, 543-545
 - cómo mejorar, 264-283
 - escalabilidad, 543-544
 - ley de Amdahl, 541-542
 - métricas de hardware, 539-541
 - métricas de software, 541-543
 - ocultamiento de latencia, 544-545
- Diámetro de la red, 532
- Difusión (Broadcasting), 550
- Digital Equipment Corporation, 14, 19, 45
- Dimensionalidad, 533
- DIMM
 - de contorno pequeño, 68
 - (*véase* Módulo de memoria dual en línea)
- Diodo emisor de luz, 100
- DIP (*véase* Paquete dual en línea)
- Dirección
 - de memoria, 57-58
 - lineal, 423
- Direccionamiento, 322
 - basado-indizado, 338
 - de nivel ISA, 342-347
 - de pila, 338-341
 - de registro, 334-335
 - de registro indirecto, 335-336
 - directo, 334
 - indirecto por registro, 335-336
 - indizado, 336-338
 - inmediato, 334
 - instrucciones de ramificación, 341-342
 - por bloque lógico, 74
- Directorio, 435
 - de páginas, 423
 - de trabajo, 461
 - raíz, 461
- Directriz de ensamblador, 491
- Disco, 68-88
 - audiovisual, 72
 - CD-ROM, 80-83
 - con circuitos integrados a la unidad, 73-75
 - digital versátil, 86-88
 - unidad de, 72
- Diseño a nivel de microarquitectura, 243-264
- Disparado
 - por flanco *versus* disparado por nivel, 144
 - por nivel *versus* disparado por flanco, 144
- Distancia de Hamming, 61
- Divide y vencerás, algoritmo, 548
- DLL (*véase* Biblioteca de enlazado dinámico)
- DMA (*véase* Acceso directo a memoria)
- DPI (*véase* Puntos por pulgada)
- DRAM (*véase* RAM dinámica)
 - sincrónica, 153
- DSM (*véase también* Memoria compartida distribuida)
 - por hardware, 574
- DUP IJVM, instrucción, 222, 223, 237
- DVD, 86-88
 - extendido, 74
 - flexible, 73
 - IDE, 73-75
 - magnético, 70-80
 - óptico, 80-88
 - RAID, 76-80
 - SCSI, 75-76
 - único, grande y caro, 76
 - (*véase* Disco versátil digital versátil)
 - Winchester, 71

E

- E/S (*véase también* Entrada/salida)
 - en UNIX, 459-465
 - en Windows NT, 465-470
 - implementación, 431-435
 - instrucciones de, 356-359
 - por mapeo de memoria, 195-197
 - programada, 356
 - virtual, 429-436
- Eagle, 591
- Eckert, J. Presper, 17
- ECL (*véase* Lógica de emisor acoplado)
- Editor de enlace, 506
- EDVAC, 17
- EEPROM (*véase* PROM borrrable eléctricamente)
- EIDE (*véase* IDE extendido)
- EISA, bus (*véase* Bus ISA extendido)
- Eje raíz, 191
- Ejecución
 - condicional, 393

de instrucciones, 42-45
 especulativa, 281-283
 fuera de orden, 276-281

Emisor, 118

Empaquetamiento de memoria, 67-68

Encadenamiento
 circular, 165-167
 de operaciones, 559

Endian, 59

ENIAC, 17

ENIGMA, 16

Enlace (*thread*), 547
 dinámico, 515
 en Java, 439
 en UNIX, 471-473
 llamada al sistema, 461

Enlaces (*linking*), 506-519
 en MULTICS, 515-516
 en UNIX, 519
 en Windows, 517-518

Enlazado
 explícito, 518
 implícito, 518

Enlazador, 506
 tareas del, 508-511

Enrutamiento
 adaptativo, 538
 algoritmos de, 538-539
 adaptativos, 538
 dimensionales, 538
 dinámicos, 538
 estáticos, 538
 de origen, 538
 dimensional, 538
 estático, 538
 por túnel, 537

Ensamblado, proceso de, 498-506

Ensamblador, 7, 484, 498-506
 de dos pasadas, 498-499
 primera pasada, 499-502
 tabla de símbolos, 505-507

Enterprise, Sun, 570-571

Entrada estándar, 461

Entrada/salida, 89-112 (*véase también* E/S)

EPIC (*véase* Computación con instrucciones explícitamente paralelas)

Epílogo de procedimiento, 375

EPROM (*véase* PROM borrrable)

Equivalencia de circuitos, 123-127

Error
 estándar, 461
 relativo, 645

Escala, índice, base, 328, 345-346

Espacio
 de direcciones, 405
 de tuplas, 604
 físico de direcciones, 406
 virtual de direcciones, 406

Espera activa, 357

Estación de trabajo esclava, 593

Estado de espera, 161

Estados finitos, máquina de, 250

Estandarización, 306

Estrategia de invalidación, 566

Estridge, Philip, 23-24

Etapas de fila de procesamiento, 49

Ethernet, 595-596
 conmutado, 596

Evolución de las computadoras, 8-13

Exclusión mutua, 550

Executive, Windows NT, 452

Expansión de código de operación, 325-327

Explorador de páginas, 575

Exponente, 644

Extensiones para multimedia, 30

F

Fallo
 de caché, 268
 de página, 409

Fanout, 532

FAT (*véase* Tabla de asignación de archivos)

Fibra, 474

FIFO (*véase* Primero en entrar, primero en salir, algoritmo)

Fila, 470
 de procesamiento, 49, 548
 Mic-3, 253-260
 Mic-4, 260-264
 Pentium, 51-52
 picoJava II, 293-294
 UltraSPARC II, 290-291

Filtro, 461

Flash, memoria, 153-154

Flip-flop, 143-145
 octal, 147

I

Flujo de control
 de procedimientos, 372-376
 nivel ISA, 370-383
 secuencial, 371

Flynn, taxonomía de, 551-553

Formatos de instrucción
 criterio de diseño, 322-324
 de Pentium II, 327-328
 de UltraSPARC II, 328-330
 en el nivel ISA, 322-332

FORTTRAN, 9-10
 sistema monitor de, 10-11

Foso, 80

Fracción, 644

Fragmentación
 externa, 419
 interna, 414-415

Frecuencia de pantalla de medio tono, 104

FSM (*véase* Máquina de estados finitos)

G

Gama, 105
 de colores, 105

GDT (*véase* Tabla de descriptores globales)

Gigaplane-XB, 570

GigaRing, 590

Globe, 608-609

Goteo, 293

Goto, MAL, 230-232

GOTO IJVM, instrucción, 222, 223, 240-241

Grado, 532

Granularidad del paralelismo, 547-548

Grupo de estaciones de trabajo, 28, 553, 592-593

GUI (*véase* Interfaz gráfica para el usuario)

H

Habilitar, 143

Hamming, código de, 62-64

Hardware, 8

Haz, 392

Hipercubo, red de, 534

Hiperpaginación, 414

IA-32, 285, 311

IA-64, 388-397
 carga especulativa, 395-396
 haz, 392
 modelo EPIC, 391-393
 técnica de predicación, 393-395

IADD IJVM, instrucción, 222, 223, 233, 236-237

IAS, máquina, 17

IBM 360, 21-22, 23

IBM 701, 19

IBM 704, 19

IBM 709, 10

IBM 801, 46

IBM 1401, 21

IBM 7094, 14, 20, 21, 23

IBM Corporation, 18-19, 20, 21

IBM PC
 bus de la, 183
 origen de la, 24

IBM PS/2, 182-183

IC (*véase* Circuito integrado)

IDE, disco, 73-75

Identificador de seguridad, 468

IF_ICMPEQ IJVM, instrucción, 222, 223, 242

IFEQ IJVM, instrucción, 222, 223, 242

IFLT IJVM, instrucción, 222, 223, 242

IFU (*véase* Unidad de búsqueda de instrucciones)

IINC IJVM, instrucción, 222, 240

IJVM, 203-213, 218, 227
 área de método, 221
 código Java, 226-227
 conjunto de instrucciones, 222-226
 implementación Mic-1, 232-243
 implementación Mic-2, 253-255
 implementación Mic-3, 253-260
 implementación Mic-4, 260-264
 marco de variable local, 218
 modelo de memoria, 220-222
 operación de memoria, 209-210
 pila, 218-220
 pila de operandos, 221
 reserva constante, 220
 señal de control, 211-213
 temporización, 207-209
 trayectoria de datos, 204-210

ILC (*véase* Contador de ubicación de instrucciones)

ILOAD IJVM, instrucción, 222, 237

ILLIAC, 17

ILLIAC IV, 53-54, 554

Impresora, 101-106

- a color, 104-106
- CYMK, 104
- de cera, 106
- de inyección de tinta, 102
- de matriz, 101-102
- de sublimación de colorante, 106
- de tinta sólida, 105
- láser, 102-104
- monocromática, 101-104
- por sublimación de colorante, 106

Índice de archivos, 432

Iniciador, bus PCI, 185

Instrucciones

- comparaciones y ramificaciones, 352-353
- de control, 354-355
- de E/S, 356-359
- de movimiento, 348-349
- de Pentium II, 359-362
- de picoJava II, 364-369
- de UltraSPARC II, 362-364
- diádicas, 349-350
- llamadas a procedimientos, 353-354
- monádicas, 350-352
- operaciones, 349-352

Integración a muy grande escala, 23

Intel 4004, 29, 30

Intel 8008, 29, 30

Intel 8080, 29, 30

Intel 8086, 29, 30

Intel 8088, 29, 30

Intel 8255A, 194-195

Intel 8259A, 169-170

Intel 80286, 30, 30

Intel 80386, 30, 30

Intel 80486, 30, 30

Intel Corporation, 29

Intel IA-64, 388-397

Intel Pentium II (*véase* Pentium II)

Intel x86 (*véase también* Pentium II, IA-64), 29-31

Intel/Sandia Option Red, 590-592

Intercalado, 573

Interconexión completa, 534

Interfaz

- coherente escalable, 581-585
- con dispositivos gráficos, Windows NT, 453
- de paso de mensajes, 600-601
- de sistema de cómputo pequeño, 75-76
- del sistema, 453
- gráfica para el usuario, 449

Interpretación, 2

Intérprete, 2, 42-43

Interrupción, 90, 379-383

- imprecisa, 279
- precisa, 279
- transparente, 381

Inverso aditivo, 351

Inversor, 119

INVOKEVIRTUAL IJVM, instrucción, 222-225, 242-243

IOR IJVM, instrucción, 222, 223, 237

IQ-Link, tarjeta, 581

IR (*véase* Registro de instrucciones)

IRETURN IJVM, instrucción, 222, 223, 225-226, 242-243

ISA, nivel, 6, 303-402

- direccionamiento, 342-347
- flujo de control, 370-383
- formatos de instrucciones, 322-332
- generalidades, 305-318
- modelos de memoria, 307-309
- propiedades, 305-307
- tipos de datos, 318-321
- tipos de instrucciones, 348-370

ISA (*véase* Arquitectura de conjunto de instrucciones)

ISA (*véase también* Arquitectura industrial estándar)

- bus, 181-183

ISDN (*véase* Red digital de servicios integrados)

ISTORE IJVM, instrucción, 222, 223, 237

ISUB IJVM, instrucción, 222, 223, 237

IU (*véase* Unidad de enteros)

J

Jerarquía de memoria, 69-70

Jobs, Steve, 23

JOHNIAC, 17

Joy, Bill, 32

JVM (*véase* Máquina Virtual Java)

K

Kestrel, 591

Khosla, Vinod, 32

L

Land, 80

Latch, 141-143

Latch D, 143

- con reloj, 143

Latch SR, 141-142

- con reloj, 142-143

Latencia, 51

- rotacional, 71

Latin-1, 111

LBA (*véase* Direccionamiento por bloque lógico)

LCD (*véase* Pantalla de cristal líquido)

LDC_W IJVM, instrucción, 222, 240

LDT (*véase* Tabla de descriptores locales)

LED (*véase* Diodo emisor de luz)

Lenguaje

- de alto nivel, 7
- de máquina, 1
- ensamblador

 - características, 484-485
 - enunciados, 488-491
 - razones para usar, 485-488
 - seudoinstrucciones, 491-493

- fuelle, 483
- objetivo, 483

Levantamiento de código, 281

Ley

- de Amdahl, 541
- De Morgan, 125-126
- Moore, 25

Libro

- Amarillo, 81
- Anaranjado, 84
- Rojo, 80
- Verde, 83

Linda, 604-606

Línea de caché, 67, 266, 565

Líneas por pulgada, 104

Lista

- de control de acceso, 468
- libre, 433

Literal, 501

Little endian, 59

Localidad

- espacial, 266
- temporal, 266

Lógica

- de emisor acoplado, 120
- de transistor-transistor, 120
- negativa, 127
- positiva, 127

Longitud de trayectoria, 244

- reducción de, 245-252

Lovelace, Ada, 15

LPI (*véase* Líneas por pulgada)

LRU (*véase* Menos recientemente utilizado, algoritmo del)

LV (*véase* Apuntador a variable local)

Llamada

- a procedimiento, instrucción de, 353-354
- al sistema, 11, 403
- al supervisor, 11
- en UNIX, 459-465

M

Macro, 494-498

- definición de, 494
- expansión de, 494
- implementación de, 498
- llamada de, 494
- parámetro de, 496

Macroarquitectura, 218

Magnitud con signo, 637

Mainframe, 28

MAL (*véase* Microlenguaje ensamblador)

Manejador

- de bus, 158
- de dispositivo, en Windows NT, 452
- de interrupciones, 90
- de trampas, 379

MANIAC

Mantisa, 644

Mapa

- de bits, 320
- de memoria, 406

Máquina

- analítica, 14
- de diferencias, 13
- de estado, 204

ILC (*véase* Contador de ubicación de instrucciones)
ILOAD IJVM, instrucción, 222, 237
ILLIAC, 17
ILLIAC IV, 53-54, 554
Impresora, 101-106
 a color, 104-106
 CYMK, 104
 de cera, 106
 de inyección de tinta, 102
 de matriz, 101-102
 de sublimación de colorante, 106
 de tinta sólida, 105
 láser, 102-104
 monocromática, 101-104
 por sublimación de colorante, 106
Índice de archivos, 432
Iniciador, bus PCI, 185
Instrucciones
 comparaciones y ramificaciones, 352-353
 de control, 354-355
 de E/S, 356-359
 de movimiento, 348-349
 de Pentium II, 359-362
 de picoJava II, 364-369
 de UltraSPARC II, 362-364
 diádicas, 349-350
 llamadas a procedimientos, 353-354
 monádicas, 350-352
 operaciones, 349-352
Integración a muy grande escala, 23
Intel 4004, 29, 30
Intel 8008, 29, 30
Intel 8080, 29, 30
Intel 8086, 29, 30
Intel 8088, 29, 30
Intel 8255A, 194-195
Intel 8259A, 169-170
Intel 80286, 30, 30
Intel 80386, 30, 30
Intel 80486, 30, 30
Intel Corporation, 29
Intel IA-64, 388-397
Intel Pentium II (*véase* Pentium II)
Intel x86 (*véase también* Pentium II, IA-64), 29-31
Intel/Sandia Option Red, 590-592
Intercalado, 573
Interconexión completa, 534

Interfaz
 coherente escalable, 581-585
 con dispositivos gráficos, Windows NT, 453
 de paso de mensajes, 600-601
 de sistema de cómputo pequeño, 75-76
 del sistema, 453
 gráfica para el usuario, 449
Interpretación, 2
Intérprete, 2, 42-43
Interrupción, 90, 379-383
 imprecisa, 279
 precisa, 279
 transparente, 381
Inverso aditivo, 351
Inversor, 119
INVOKEVIRTUAL IJVM, instrucción, 222-225, 242-243
IOR IJVM, instrucción, 222, 223, 237
IQ-Link, tarjeta, 581
IR (*véase* Registro de instrucciones)
IRETURN IJVM, instrucción, 222, 223, 225-226, 242-243
ISA, nivel, 6, 303-402
 direccionamiento, 342-347
 flujo de control, 370-383
 formatos de instrucciones, 322-332
 generalidades, 305-318
 modelos de memoria, 307-309
 propiedades, 305-307
 tipos de datos, 318-321
 tipos de instrucciones, 348-370
ISA (*véase* Arquitectura de conjunto de instrucciones)
ISA (*véase también* Arquitectura industrial estándar)
 bus, 181-183
ISDN (*véase* Red digital de servicios integrados)
ISTORE IJVM, instrucción, 222, 223, 237
ISUB IJVM, instrucción, 222, 223, 237
IU (*véase* Unidad de enteros)

J

Jerarquía de memoria, 69-70
Jobs, Steve, 23
JOHNIAC, 17
Joy, Bill, 32
JVM (*véase* Máquina Virtual Java)

K

Kestrel, 591
Khosla, Vinod, 32

L

Land, 80
Latch, 141-143
Latch D, 143
 con reloj, 143
Latch SR, 141-142
 con reloj, 142-143
Latencia, 51
 rotacional, 71
Latin-1, 111
LBA (*véase* Direccionamiento por bloque lógico)
LCD (*véase* Pantalla de cristal líquido)
LDC_W IJVM, instrucción, 222, 240
LDT (*véase* Tabla de descriptores locales)
LED (*véase* Diodo emisor de luz)
Lenguaje
 de alto nivel, 7
 de máquina, 1
 ensamblador
 características, 484-485
 enunciados, 488-491
 razones para usar, 485-488
 seudoinstrucciones, 491-493
 fuente, 483
 objetivo, 483
Levantamiento de código, 281
Ley
 de Amdahl, 541
 De Morgan, 125-126
 Moore, 25
Libro
 Amarillo, 81
 Anaranjado, 84
 Rojo, 80
 Verde, 83
Linda, 604-606
Línea de caché, 67, 266, 565
Líneas por pulgada, 104
Lista
 de control de acceso, 468
 libre, 433
Literal, 501

Little endian, 59
Localidad
 espacial, 266
 temporal, 266
Lógica
 de emisor acoplado, 120
 de transistor-transistor, 120
 negativa, 127
 positiva, 127
Longitud de trayectoria, 244
 reducción de, 245-252
Lovelace, Ada, 15
LPI (*véase* Líneas por pulgada)
LRU (*véase* Menos recientemente utilizado, algoritmo del)
LV (*véase* Apuntador a variable local)
Llamada
 a procedimiento, instrucción de, 353-354
 al sistema, 11, 403
 al supervisor, 11
 en UNIX, 459-465

M

Macro, 494-498
 definición de, 494
 expansión de, 494
 implementación de, 498
 llamada de, 494
 parámetro de, 496
Macroarquitectura, 218
Magnitud con signo, 637
Mainframe, 28
MAL (*véase* Microlenguaje ensamblador)
Manejador
 de bus, 158
 de dispositivo, en Windows NT, 452
 de interrupciones, 90
 de trampas, 379
MANIAC
Mantisa, 644
Mapa
 de bits, 320
 de memoria, 406
Máquina
 analítica, 14
 de diferencias, 13
 de estado, 204

- de estados finitos
 - predicción de ramificación, 274-275
 - unidad de búsqueda de instrucciones, 250-251
- de impresión, 103
- multinivel, 4-7
- virtual, 2-4
- Máquina virtual Java, 34-35
 - formatos de instrucción, 330-332
 - generalidades, 317-318
 - instrucciones, 364-368
 - modos de direccionamiento, 346-347
 - tipos de datos, 321
 - torres de Hanoi, 386-389
- Máquina virtual paralela, 599-600
- MAR (véase Registro de dirección de memoria)
- Marcador, 277
- Marco
 - de página, 408
 - de variable local, 218, 220
- Máscara, 349
- MASM, 488
 - seudoinstrucciones, 491-493
- Matriz activa, presentación de, 95
- Mauchley, John, 16-17
- MBR (véase Registro de buffer de memoria)
- McNealy, Scott, 32
- MDR (véase Registro de datos de memoria)
- Medio sumador, 135-136
- Medios tonos, 104
- Mejor ajuste, algoritmo de, 420
- Memoria, 56-68, 141-154
 - asociativa, 420-421, 505
 - caché, 30, 65-67, 265-270
 - con mapeo directo, 267-269
 - de asignación de escritura, 270, 566
 - de conjunto asociativo, 269-270
 - de escritura diferida, 270
 - de escritura inmediata, 270, 565
 - de escritura una vez, 568
 - de múltiples niveles, 265-266
 - de nivel 2, 265
 - de protocolo MESI, 568-569
 - de reescritura, 270
 - dividida, 67, 265
 - espionaje de, 564-569
 - estrategia de actualización, 566
 - estrategia de invalidación, 566
 - unificada, 67
 - compartida a nivel de aplicaciones, 601-609

- compartida distribuida, DSM, 602
- de acceso aleatorio, 152-154
 - RAM dinámica, 152
 - RAM estática, 152
- de atracción, 585
- de salida de datos extendida, 152-153
- de sólo lectura, 153, 154
- de video, 96
- EDO, 152-153
 - (véase Memoria de salida de datos extendida)
- EEPROM, 152, 154
- EPROM, 153, 154
- flash, 153-154
- FPM, 152
 - (véase Modo de página rápida, memoria en)
- primaria, 56-68
- PROM, 153, 154
- RAM dinámica, 152, 154
- RAM estática, 152, 154
- ROM, 153, 154
- secundaria, 68-88
- semántica de la (véase Semántica de la memoria)
- virtual, 404-429
 - de Pentium II, 421-426
 - de UltraSPARC II, 426-428
 - en comparación con uso de cachés, 428-429
 - en UNIX, 455-456
 - en Windows NT, 456-458
- Menos recientemente utilizado, algoritmo del, 269, 412-413
- Mensajes, paso de, 598-601
- Merced (véase IA-64)
- MESI, protocolo de caché, 568-569
- Método, 354
 - de comunicación, 549-550
- MFT (véase Tabla principal de archivos)
- Mic-1, 213-252
 - trayectoria de datos, 214
 - implementación, 232-243
 - implementación IJVM, 232-243
 - microprograma, 234-236
 - optimación, 243-252
- Mic-2, 252-255
 - trayectoria de datos, 252
 - microprograma, 253-255
 - prebúsqueda, 253

- Mic-3, 253-260
 - filas de procesamiento, 253-260
 - trayectoria de datos, 256
- Mic-4, 260-264
 - trayectoria de datos, 261
- Mickey, 101
- Microarquitectura
 - Pentium II, 283-288, 296-298
 - picoJava II, 291-298
 - UltraSPARC II, 288-291, 296-298
- Microinstrucción, 45, 211-213
- MicroJava, 701
 - introducción, 35-36
 - vista a nivel de chip, 180-181
- Microkernel, Windows NT, 452
- Microlenguaje ensamblador, 227-232
- Microoperación, 262
- Micropaso, 257
- Microprograma, 6, 11-13
- Microprogramación, 8-9
- Microsoft, 24
- MIMD, computadora, 551-553
- MIPS (acrónimo), 48
 - chip, 46
- MIR (véase Registro de microinstrucción)
- MISD, computadora, 551-552
- MMU (véase Unidad de administración de memoria)
- MMX (véase Extensiones multimedios)
- Modelo(s)
 - de comunicación, 526-530
 - de consistencia, 560
 - del conjunto de trabajo, 412
- Módem, 98
- Modo
 - 8086 virtual, 312
 - de compartimiento falso, 603
 - de kernel, 307
 - de página rápida, memoria en, 152
 - de registro, 334
 - de transferencia asincrónica, 596-597
 - de usuario, 307
- Modos de direccionamiento, 333-342
 - explicación, 347
 - JVM, 346-347
 - Pentium II, 344-346
 - UltraSPARC II, 346
- Modulación, 106-107
 - de amplitud, 106-107
 - de fase, 107
- de frecuencia, 107
- dibit, 107
- por desplazamiento de frecuencia, 107
- Módulo
 - de memoria dual en línea, 68
 - de memoria individual en línea, 68
 - objeto, 511-512
- Monitor, 93
- Montículo, 317
- Moore, Gordon, 25, 29
- MOS, 120
- Motif, 449
- Motorola 68000, 45
- Movimiento de datos, instrucción de, 348-349
- MPC (véase Contador de microprograma)
- MPI (véase Interfaz de paso de mensajes)
- MPP (véase Enlace masivamente paralelo)
- Multicomputadora, 56, 527, 552-553, 586-609
 - COW, 592-598
 - Cray T3E, 589-590
 - escalable, 529
 - MPI, 600-601
 - NOW, 592-598
 - Option Red, 590-592
 - planificación de, 593-595
 - PVM, 599-600
 - software de comunicación, 598-601
- MULTICS
 - desarrollo de, 487
 - enlazado dinámico en, 515-516
 - memoria virtual en, 420-421
- Multidifusión, 550
- Multienlaces, 544
- Multihilos (Véase Multienlaces)
- Multiplexor, 130
- Multiprocesador, 55, 526-530, 559-586
 - basado en directorios, 575-585
 - DASH, 577-580
 - espionaje, 564-569
 - multietapas, 571-573
 - NUMA, 573-585
 - NUMA-Q, 581-585
 - simétrico, 559, 564-573
 - transversal, 569-571
 - UMA basada en bus, 564-569
- Multiprogramación, 22
- Mutex, 473
- Myhrvold, Nathan, 26
- Myrinet, 597-598

N

NaN (*véase* No es número)

NC-NUMA (*véase* NUMA no coherente)

Nemática torcida, 94

Nibble, 360

NIC (*véase* Chip de interfaz de red)

Nivel, 2-4

- de arquitectura del conjunto de instrucciones, 6
- de dispositivos, 5, 118
- de lenguaje ensamblador, 483-522
- de lógica digital, 5, 117-202
- de máquina del sistema operativo, 6, 403-482
- de microarquitectura, 5, 203-302

No es número, 650

Nodo i, 463

NOP IJVM, instrucción, 222, 236

NORMA (*véase* Sin acceso remoto a memoria)

Norma de punto flotante IEEE 754, 646-650

Normalizado, 646

Notación

- de microinstrucción, 227-232
- en exceso, 638
- infija, 338
- polaca, 338-342
- inversa, 338-342
- posfija, 338

NOW (*véase* Red de estaciones de trabajo)

Noyce, Robert, 21, 29

NTFS (*véase* Sistema de archivos NT)

Nueva tecnología de Windows, 450

NUMA (*véase* Acceso a memoria no uniforme)

- con caché coherente, 574, 575-585
- no coherente, 574

NUMA-Q, multiprocesador, 581-585

Número(s)

- binario, 631-642
- negativo, 637-640
- de doble precisión, 318
- de precisión finita, 631-633
- de punto flotante, 643-651
- desnormalizado, 649
- hexadecimal, 633

O

Obj archivo, 506

Objetivo, del bus PCI, 185

OC-12, 596

Ocultamiento de latencia, 544-545

Olsen, Kenneth, 19

Omega, red, 572

Ómnibus, 19-20

OPC, registro, 205, 232

Opcode, 204

Operación de bus, 167-170

Operando inmediato, 334

Option

- Blue, 591
- Red, 590-592
- White, 591

OR alambrado, 158

Orca, 606-608

Ordenamiento

- de bytes, 58-61
- perfecto, 572

Organización

- de computadoras, 7
- de la memoria, 146-150
- estructurada de computadoras, 2-4

Ortogonalidad de códigos de operación y direccionamiento, 342-344

Otorgamiento de bus, 165-167

P

Página, 406

- comprometida, 457
- de código, 111
- libre, 457
- reservada, 457

Paginación, 405-407

- implementación, 407-409
- por demanda, 409-412

Palabra, 58

- de código, 61
- de estado del programa, 310
- en Pentium II, 425

Paleta de colores, 97

Pantalla

- de cristal líquido, 94-95
- de matriz pasiva, 94
- de panel plano, 94-95

Paquete, 531

- dual en línea, 128

Paradigma

- computacional, 548-549
- del trabajador repetido, 548, 606

Paralelismo

- a nivel de bloques, 547
- de grano fino, 525
- de grano grueso, 525
- granularidad, 547-548
- nivel de instrucciones, 49-53
- nivel de procesos, 53-56

Paro

- de la cola de procesamiento, 258, 272
- de la fila de procesamiento, 272

Pasada de ensamblador, 499

Pascal, Blaise, 13

PC (*véase* Contador de programa)

PDP-1, 19

PDP-8, 19

Peligro, 258

Pentium II

- bus en filas de procesamiento, 174-175
- conexiones de terminales, 172-174
- control de potencia, 171-172
- empaquetado, 171
- en modo real, 311
- formatos de instrucción, 327-328
- generalidades a nivel de chip, 170-175
- generalidades del nivel ISA, 311-313
- instrucciones, 359-362
- introducción, 29-31
- memoria virtual, 421-426
- microarquitectura, 283-288, 296-298
- modo 8086 virtual, 312
- modos de direccionamiento, 344-346
- registros, 312-313
- tipos de datos, 320
- torres de Hanoi, 384-385
- transacción de bus, 174-175
- unidad de búsqueda/decodificación, 285-286
- unidad de despacho/ejecución, 286-287
- unidad de retiro, 287-288

PicoJava I, diseño en, 35

PicoJava II, diseño en

- fila de procesamiento, 293-294
- formatos de instrucción de JVM, 330-332
- generalidades del nivel JVM ISA, 317-318
- instrucciones, 364-369
- introducción, 35
- microarquitectura, 291-298
- modos de direccionamiento de JVM, 346-347
- paso a paso, 293
- perspectiva a nivel de chip, 179-181
- plegado, 294-296
- plegado de instrucciones, 294-296
- tipos de datos de JVM, 321
- torres de Hanoi, 386-389

Pila, 218-219

- de operandos, 219-221, 226-227

PIO, chip, 194-195

Pista, 70

Pixel, 96

PLA (*véase* Arreglo lógico programable)

Planificación de multicomputadoras, 593-595

Plantilla, 605

Plegado de instrucciones, en picoJava II, 294-296

Política de reemplazo de páginas, 412-414

POP IJVM, instrucción, 222, 223, 233, 237

Porción de bit (*bit slice*), 139

Portadora, 106

POSIX, 447

Preámbulo, 70

Prebúsqueda, 544

Predicación, 393-395

Predicción de ramificaciones, 270-276

- dinámica, 273-275
- estática, 275-276

Primer ajuste, algoritmo de, 420

Primera

- generación de computadoras, 16-19
- ley del software de Nathan, 26

Primero en entrar, primero en salir, algoritmo, 413

Principio de localidad, 66

Principios de diseño RISC, 47-49

Problema del productor-consumidor, 438-445

Procedimiento, 372-376

- recursivo, 372

Procesador, 39-56

- masivamente paralelo, 553, 587-592
- vectorial, 54, 555-559

Procesamiento

- con instrucciones explícitamente paralelas, 391-393
- en paralelo, 436-445

Proceso

- hijo, 470
- ligero, 547

Programa, 1

- automodificable, 336
- binario ejecutable, 484, 506
- objeto, 484

Programador de sistemas, 7

Prólogo de procedimiento, 375

PROM
 borrable, 153, 154
 (véase ROM programable)
Propiedad de integridad, 123
Protocolo de bus, 157
Prueba comparativa (Benchmark), 486
PSW *(véase Palabra de estado del programa)*
Pthreads, 472
Punto(s)
 de código, 111
 de cruce, 570
 de entrada, 511
 por pulgada, 102
PVM *(véase Máquina virtual paralela)*

R

RAID, 76-80
RAM *(véase también Memoria de acceso aleatorio)*
 de video, 97
 dinámica, 152, 154
 estática, 152, 154
Ramificación, instrucción de, 352-353
Ranura
 de correo, 475
 de retraso, 272
Ratón, 99
RAW, dependencia, 258
Receptor-transmisor universal asincrónico, 98, 193
Recolector, 118
 de basura, 317
Recurción, 354
Red
 bloqueadora, 573
 conmutadora multietapas, 571-573
 cúbica, 534
 de anillo, 534
 de árbol, 534
 de árbol grueso, 534
 de conmutación, 535-538
 de cuadrícula, 534
 de doble cilindro, 534
 de estaciones de trabajo, 28, 553, 592-593
 de interconexión, 530-532
 bisección del ancho de banda, 532
 comercial, 595-598

 conmutación, 535-538
 topología, 532-535
 de malla, 534
 digital de servicios integrados, 108-109
 en estrella, 534
 no bloqueadora, 570
Redondeo, 645
Reed-Solomon, código, 71
Referencia
 externa, 509
 hacia adelante, problema de la, 499
Refresco de memoria, 152
Registro, 5, 145-146
 de buffer de memoria, 209-212, 232, 238, 250-252
 de datos de memoria, 209-211
 de desplazamiento, para almacenamiento de saltos, 275
 de dirección de memoria, 209-211
 de instrucciones, 40
 de microinstrucción, 215
 E, 589
 en el nivel ISA, 309-310
 H, 205, 228-229
 lógico, 431
 vectorial, 54
 virtual, 218
Reloj, 139-141
Reserva constante, 220
Retraso de compuerta, 129
Reubicación
 dinámica, 512-515
 problema de, 508
RISC *(véase Computadora con conjunto de instrucciones reducido)*
ROB *(véase Buffer de resurtido)*
Robo de ciclos, 90, 359
ROM
 programable, 153, 154
 (véase Memoria de sólo lectura)
RS-232-C, terminal, 98-99
Ruta absoluta, 461
Rutina de servicio de interrupción, 380-383

S

Salida estándar, 461
Saludo completo, (Full handshake), 164
SBus, 177

SCI *(véase Interfaz coherente escalable)*
SCSI *(véase Interfaz de sistema de cómputo pequeño)*
SDRAM *(véase DRAM sincrónica)*
SEC *(véase Cartucho de una sola arista)*
Sección crítica, 476
Sector, 70
Secuenciador, 213
Segmentación, 415-418
 implementación, 418-421
 por mejor ajuste, 420
 por primer ajuste, 420
Segmento, 416-418
 de enlace, 515
Segunda generación de computadoras, 19-21
Selección de acarreo, sumador con, 137
Semáforo, 442-445
Semántica de la memoria, 559-563
 consistencia de liberación, 563
 consistencia del procesador, 561-562
 consistencia estricta, 560
 consistencia secuencial, 560-561
Señal
 activada, 150
 de control, 209
 negada, 151
Separación entre sectores, 71
Sequent NUMA-Q, multiprocesador, 581-585
Servicios del sistema, Windows NT, 453
Sesgo, 536
 de bus, 160
Sesión, CD-ROM de, 85
Seudoinstrucción, 491
Shell, 449, 589
SIB *(véase Escala, índice, base)*
SID *(véase Identificador de seguridad)*
Significando, 648
Signo, extensión de, 210
Símbolo externo, 512
SIMD, computadora, 551-552, 554-559
SIMM *(véase Módulo de memoria individual en línea)*
Sin acceso remoto a memoria, 553
Sincronización
 de procesos con semáforos, 442-445
 del bus, 160-165
 primitiva, 550-551
Sintonización de programas, 486
SISD, computadora, 551

Sistema
 básico de entrada/salida, 74
 de archivos NT, 465
 de archivos, Windows NT, 452
 de memoria compartida *(véase Multiprocesador)*
 de memoria distribuida, 529, 602-604
 hardware del, 574
 débilmente acoplado, 525
 escalable, 543
 fuertemente acoplado, 526
 operativo, 9-11, 403
 por lotes, 11
SLED *(véase Disco único, grande y caro)*
SMP *(véase Multiprocesador simétrico)*
Socket, 447
SO-DIMM *(véase DIMM de contorno pequeño)*
Software para computadora paralela, 545-546
 de comunicación, 598-609
 de planificación, 593-595
Solaris, 447
Solicitud de bus, 165-167
SP *(véase Apuntador a la pila)*
SPARC, 32
SPMD *(véase Un solo programa, múltiples datos)*
SRAM *(véase RAM estática)*
Stibbitz, George, 15-16
Stream, 448
Striping (multiplexaje de datos), 77
Strobe, 143
Subrutina, 353
Subsistema de entorno, Windows NT, 453
 subsistema POSIX, Windows NT, 453
Sumador, 135-137
 completo, 136-137
 con propagación de acarreo, 137
 con selección de acarreo, 137
 medio, 135 136
Sun Enterprise 10000, 570-571
Sun Microsystems, 32-33
Sun UltraSPARC *(véase UltraSPARC)*
Supercomputadora, 21, 28
 ASCI distribuida, 593
 Cray-1, 557-559
Superposición, 405
Superusuario, 463
SWAP IJVM, instrucción, 222, 223, 237, 257-259

T

Tabla

- de asignación de archivos, 465
- de contenido del volumen, 85
- de descriptores globales, 421
- de descriptores locales, 421
- de memoria local, 582
- de páginas, 406
- de símbolos, 499
- de traducción, 428
- de verdad, 120
- principal de archivos, 469

Tamaño

- de grano, 525
- de página, 414-415

Tarjeta

- de línea, 596
- madre, 89
- Quad, 581

Tasa de fallos, 66

TAT-12/13, 26

Taxonomía de computadoras paralelas, 551-553

Teclado, 92

Temporización de bus, 160-163

Tercera generación de computadoras, 21-23

Terminal, 91-99

- de mapa de bits, 96-98
- de mapa de caracteres, 96

Terminales de los circuitos integrados, 154

Tiempo

- compartido, sistema de, 11
- de ciclo de reloj, 139
- de ligado, 512-515

Tinta

- basada en colorantes, 105
- basada en pigmentos, 105
- de impresora a color, 105

Tipos de datos

- no numéricos, 319-321
- numéricos, 319-321

Tipos de instrucciones, en el nivel ISA, 348-370

TLB (véase Buffer de consulta para traducción)

Tope del registro de pila, 205, 232

Topología, 532-535

- virtual, 601

Torres de Hanoi, 372-376, 383-388

- con Pentium II, 384-385
- con picoJava II, 386-389
- con UltraSPARC II, 384-387

TOS (véase Tope del registro de pila)

Traducción, 2

- de Java a IJVM, 226-227

Traductor, 483

- de dos pasadas, 499

Trampa, 379

Transacción de bus, 174

- PCI, bus, 189

Transceptor de bus, 158

Transferencia

- de bloques, bus de, 168
- de mensajes punto a punto, 550

Transición, máquina de estados finitos, 250

Transistor, 19-21

- bipolar, 118-119, 120
- MOS, 120

Transmisión

- de mensajes sincrónico, 598
- dúplex, 108
- símples, 108

Trayectoria, 461

- de datos, 6, 40, 204-210

Mic-1, 214

Mic-2, 252

Mic-3, 256

Mic-4, 261

temporización del, 207-209

Tres estados, dispositivo de, 149

TSB (véase Buffer de almacenamiento para traducción)

TTL (véase Lógica de transistor-transistor)

Tubo de rayos catódicos, 93

Tupla, 604

TX-0, 19

U

UART (véase Receptor Transmisor Universal Asincrónico)

UDB (véase Buffer de datos de UltraSPARC)

Ultra puerto, arquitectura de, 177

UltraSPARC I, 33

UltraSPARC II

- buffer de datos, 177
- cola de procesamiento, 290-291

formatos de instrucción, 328-330

generalidades del nivel ISA, 313-316

instrucciones, 362-364

introducción, 31-34

memoria virtual, 426-428

microarquitectura, 288-291, 296-298

modos de direccionamiento, 346

perspectiva a nivel de chip, 176-179

tipos de datos, 321

torres de Hanoi, 384-387

ventanas de registro, 315-316

UMA (véase Acceso uniforme a la memoria)

Un solo programa, múltiples datos, 548

UNICODE, 111-112

Unidad

- aritmética lógica, 5, 40, 138-139, 204-209
- central de procesamiento, 39-56
- de administración de memoria, 409
- de almacenamiento temporal, 262
- de búsqueda de instrucciones, 249-252
- de búsqueda/decodificación, Pentium II, 285-286
- de despacho/ejecución, Pentium II, 285-286
- de enteros, 33
- de retiro, Pentium II, 287-280
- decodificadora, 261

UNIX

- administración de procesos, 470-473
- Berkeley, 447
- cola de procesamiento, 470
- descriptor de archivo, 459
- directorío, 461-463
- E/S virtual, 459-465
- enlace, 471-473
- implementación del sistema de archivos, 461-465
- introducción, 446-449
- llamadas al sistema, 459-465
- Solaris, 447
- System V, 447

UPA (véase Arquitectura ultrapuerto)

USART, 193

USB (véase Bus Serie Universal)

Uso de buffer común, 537

V

Variable de condición, 473

VAX, 45, 46

Vector, 555

- de interrupción, 169, 380

Ventana, 97

Ventanas de registros, 315-316

VIS (véase Conjunto de instrucciones visuales)

VLSI (véase Integración a muy grande escala)

Von Neumann

- John, 17-18

- máquina de, 18, 41

VTOC (véase Tabla de contenido del volumen)

W

WEIZAC, 17

Whirlwind I, 18

WIDE IJVM, instrucción, 222, 239-240

Wilkes, Maurice, 44

Win32, subsistema, 453

Win32API, 454

Winchester, disco, 71

Windows 95, 449

Windows 98, 450

Windows NT

- administración de procesos, 473-476
- administrador de memoria virtual, 452
- administrador de objetos, 452
- capa de abstracción de hardware, 451
- E/S virtual, 465-470
- interfaz con dispositivos gráficos, 452
- introducción, 450-455
- llamadas Win32API, 466-468
- microkernel, 452
- seguridad, 452, 468
- sistema de archivos, 469-470

Wozniak, 23

X

X Windows, 449

Xeon, 31

Z

Zilog Z8000, 45

Zuse, Konrad, 15

ACERCA DEL AUTOR

Andrew S. Tanenbaum estudió la licenciatura en ciencias en el MIT y un doctorado en la University of California en Berkeley. Actualmente es profesor de ciencias de la computación en la Vrije Universiteit en Amsterdam, Países Bajos, donde encabeza el Grupo de Sistemas Computacionales; también es decano de la Advanced School for Computing and Imaging, una escuela de posgrado interuniversitaria dedicada a la investigación de sistemas paralelos, distribuidos y de imágenes avanzados. No obstante, está esforzándose mucho por no convertirse en un burócrata.

El profesor Tanenbaum ha realizado investigaciones sobre compiladores, sistemas operativos, redes y sistemas distribuidos de área local. Sus investigaciones actuales se enfocan principalmente en el diseño de sistemas distribuidos de área extensa que pueden ampliarse a millones de usuarios. Estos proyectos de investigación han dado pie a más de 85 artículos arbitrados en revistas y memorias de congresos, y a cinco libros.

El profesor Tanenbaum también ha producido un volumen considerable de software. Él fue el arquitecto principal del Amsterdam Compiler Kit, una herramienta ampliamente usada para escribir compiladores portátiles, así como de MINIX, un pequeño clon de UNIX creado para ser usado por estudiantes en laboratorios de programación. Junto con sus estudiantes de doctorado y programadores, contribuyó en el diseño de Amoeba, un sistema operativo distribuido de alto rendimiento basado en microkernel. Los sistemas MINIX y Amoeba ya se encuentran disponibles gratuitamente a través de Internet.

Los estudiantes de doctorado del profesor Tanenbaum han cosechado grandes triunfos después de obtener sus grados. Él está muy orgulloso de ellos; en este sentido se asemeja a una mamá gallina.

El profesor Tanenbaum es Socio de la ACM, Socio del IEEE, miembro de la Academia Real de Artes y Ciencias de los Países Bajos, ganador del Karl. V. Karlstrom Outstandig Educator Award de la ACM en 1994 y del ACM/SIGCSE Award for Outstanding Contributions to Computer Science Education; además, está listado en *Who's Who in the World*. La dirección de su página en internet es <http://www.cs.vu.nl/~ast/>.

005.133
T164.444e

c.2.